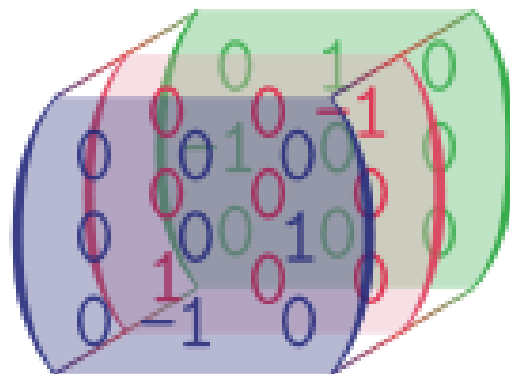


Exploring Math Σ_{math} with EIGENMATH

Some Tapas of Tensors

Introductory Tensor Calculus with EIGENMATH

$$\epsilon_{ijk} =$$


Dr. Wolfgang Lindner

dr.w.g.Lindner@gmail.com

Leichlingen, Germany

²2026

Contents

1	Coordinate Systems and Transformations	4
1.1	Examples	5
2	What is a tangent vector?	16
3	The Metric Tensor	21
3.1	Definition: metric tensor	21
3.2	Examples	22
4	Contravariant Basis	31
4.1	Definition: the contravariant basis	31
4.2	Examples	31
5	What is a tensor?	37
5.1	Definition: Tensor	37
5.2	Example: This is <i>not</i> a tensor!	38
5.3	Examples: These <i>are</i> tensors	41
6	The KRONECKER delta tensor	55
6.1	Definition: δ_i^j	55
6.2	Example: The case of the Scalar product of Vectors	59
7	The Permutation symbol ε	63
7.1	Definition	63
7.2	Examples	63
7.3	Applications	68
8	The KRONECKER product and the outer product	74
8.1	outer	74
8.2	kronecker	78
8.3	Applications of kronecker	80
9	contract	83
9.1	Definition of contract	83
9.2	Examples of contract	83
9.3	Case study : construction of the <i>curl</i>	88
10	The LEVI-CIVITA Tensor ϵ	93
10.1	Definition	93
10.2	Examples	94

<i>CONTENTS</i>	2
11 The CHRISTOFFEL Symbol Γ_{ij}^k	96
11.1 Definition	96
11.2 Examples	97
12 Appendix: source code of tensorBox	108
13 grad-div-curl in different coordinate systems	110
14 Bibliography	113

Preface

Some *tapas* of tensors: this booklet would like to whet your appetite to immerse yourself into the world of tensors with small bites of special tensors to take a closer look at tensor calculus. For the novice, tensor notations and operations are somehow clumsy and uncomfortable and accompanied by heavy calculations. Therefore here is no forbidding theory presented but instead the focus is on praxis with selected examples - using EIGENMATH as your computer algebraic companion to unburden calculation in this field. We only cite some of the necessary underlying mathematical definitions and facts to be able to show the corresponding implementation of the concepts into the language of CAS EIGENMATH.

For the theory of Tensor Analysis I strongly recommend the treatment by Pavel GRINFELD. His book [6] focusses on comprehension and illuminates mathematical concepts by well chosen worked examples and 354 exercises.

Looking back at my first contact with EIGENMATH I was very impressed by the ingenious and sophisticated treatment of the RIEMANN tensor in the calculation of the Schwarzschild metric by George WEIGT in [36]. This script is an eye opener and a convincing powerful example of the amazing efficiency of EIGENMATH.

The collection of 25 short EIGENMATH example scripts and 27 exercises in this booklet to cope with tensor operations not only want to help the reader to dive into the theory of tensors e.g. along FLEISCH's book, but also to become comfortable with the use of the CAS EIGENMATH in this field. It aims to be a source of help and inspiration for another round of thoughtful activity with respect to tensors - supplementary to paper & pencil calculations, now from the perspective of a compact CAS. So have fun on our tour from coordinate systems to the omnipresent metric tensor, to KRONECKER delta and the CHRISTOFFEL symbols. My booklet on *Elementary Differential Geometry* [8] will e.g. address the RIEMANN tensor R and the curvature tensor B to calculate the GAUSS resp. the mean curvature of surfaces in $\mathbb{R}^3$.

For the inspection or running an EIGENMATH script no installation is necessary, *everything runs directly online*: a click on a link like [\[Run script\]](#) in this text is enough to invoke the corresponding script - and by a click on the RUN button inside EIGENMATH window the calculation is made, allowing further free inputs from the user. If you own a Mac or Linux PC, there is the option to install the app EIGENMATH free of charge and run the scripts by *mark-copy-paste* into the EIGENMATH window.

I want to thank George WEIGT for his friendly support with tips and hints while writing these notes.

Wolfgang Lindner
Leichlingen, Germany
June 2026

1 Coordinate Systems and Transformations

The description of tensors relies largely on transformations between coordinate systems. Therefore the reader should be familiar with the basics of linear algebra (vector spaces, transformations) and analysis (partial derivative). FLEISCH [5] offers a very understandable introduction. We begin with some examples of coordinate systems and the switch between different basis with transformations (linear mappings), and also introduce the terminology and notations used in this book.

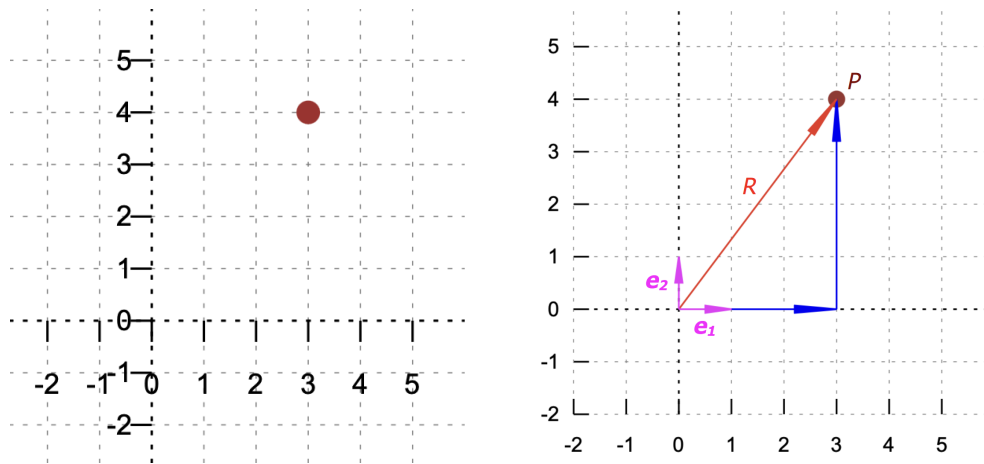


Figure 1: **left:** point $P(3,4)$ with coordinates $x = 3$ and $y = 4$ in $\mathbb{R}^2$.
right: the standard basis $\mathbf{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\mathbf{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, the position vector $R = \begin{bmatrix} 3 \\ 4 \end{bmatrix}$ and its decomposition '→↑' of R into $3 \cdot \mathbf{e}_1 + 4 \cdot \mathbf{e}_2$

In general, we associate to a point $P \in \mathbb{R}^n$ an n -tuple $(x_P^1, x_P^2, \dots, x_P^n)$, where *the superscripts does not denote the powers* of the numbers, but the i -th entry in the given coordinate system tuple. As we will see, this habit is in harmony with the usual tensor notation.

- For example, EINSTEIN wrote the usual sum $a_1e_1 + a_2e_2 + a_3e_3$ with superscript indices as $a^1e_1 + a^2e_2 + a^3e_3$ and abbreviated this summation to a^ie_i , where summation is to be performed over indices with the same name above and below.
- Or: $b^1e_1 + b^2e_2 \equiv b^ie_i$.
- In general we have: $a^1e_1 + a^2e_2 + \dots + a^ne_n = \sum_{k=1}^n a^ke_k =: a^ke_k$.

We see, that this notation is independent of the dimension of the space $\mathbb{R}^n$.

This abbreviation is called the

(EINSTEIN summation convention): *A summation in a tensor expression is implicit done, when an index appears twice, once as a subscript ('lower index') and once as a superscript ('upper index').*

- If one writes $A_i^k \cdot B_k^j$, this is *interpreted in tensor language as a hidden summation behind the scene*, e.g. we suppress the summation sign $\sum_{k=1}^n$ in such tensor expressions.

1.1 Examples

Example 1. (Same point, but different coordinates w.r.t. a basis)

We look at two coordinate systems in the 2D Euclidean vectorspace $\mathbb{R}^2$ with canonical scalar product $\bullet$, i.e. we have two different basis vectors $\vec{e}_1, \vec{e}_2$ resp. $\vec{b}_1, \vec{b}_2$ written in the physicist notation with an overline arrow $\vec{}$.

Let by example $\vec{e}_1 = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\vec{e}_2 = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ and $\vec{b}_1 = \begin{bmatrix} 2.5 \\ 1 \end{bmatrix}$, $\vec{b}_2 = \begin{bmatrix} -2 \\ 3 \end{bmatrix}$.

We ask: what are the coordinates of point $C(3, 5)_e$ with it's coordinates w.r.t. basis $e = (e_1, e_2)$ in the other coordinate system $b = (b_1, b_2)$, i.e $C(?, ?)_b$?

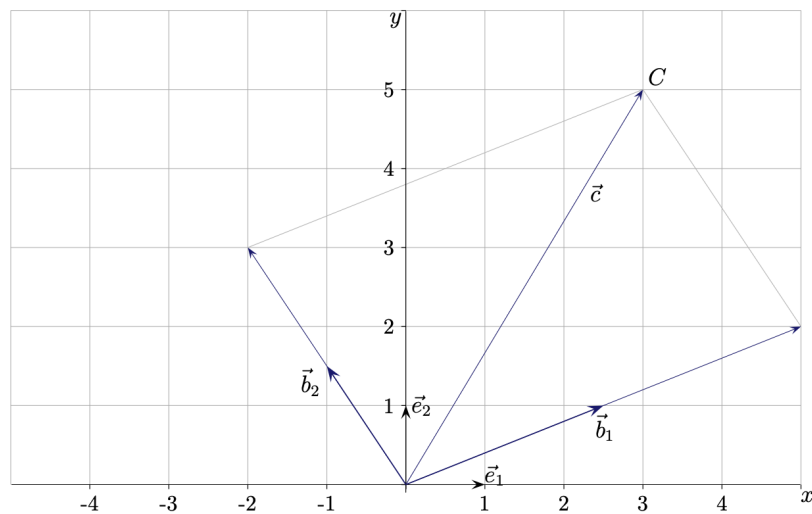


Figure 2: $\vec{e}_1, \vec{e}_2$: the standard Euclidean basis in $\mathbb{R}^2$.
 $\vec{b}_1, \vec{b}_2$: another nonstandard basis in $\mathbb{R}^2$.
 C : a point in $\mathbb{R}^2$ with different coordinates w.r.t. e resp. b .

First we drop the physicist notation in fig.1¹ and write vectors in **boldface** or in CAS EIGENMATH² using capital letters, i.e. we write $\vec{e}_1$ as $\mathbf{e}_1$ or $E1$.

1st we solve the problem using elementary (linear) algebra.

We express the point C more precise and exact by its development in the basis vectors (see fig.1) and then do the following steps:

1. $C = (3, 5) \equiv 3 \cdot E_1 + 5 \cdot E_2 = u \cdot B_1 + v \cdot B_2$ unknown coord's u, v of C w.r.t basis b
2. $B_1 = 2.5 \cdot E_1 + 1 \cdot E_2$ express B_1 w.r.t. basis e
 $B_2 = -1 \cdot E_1 + 1.5 \cdot E_2$ express B_2 w.r.t. basis e
3. $3 \cdot E_1 + 5 \cdot E_2 = u \cdot (2.5 \cdot E_1 + 1 \cdot E_2) + v \cdot (-1 \cdot E_1 + 1.5 \cdot E_2)$ substitute 2. in 1.

¹The figure is found in ROOLFS, [21].

²which don't have boldface or overline arrows

4. $3 \cdot E_1 + 5 \cdot E_2 = 2.5u \cdot E_1 + u \cdot E_2 + (-1)v \cdot E_1 + 1.5v \cdot E_2$ evaluate 3.
5. $3 \cdot E_1 + 5 \cdot E_2 = (2.5u - v) \cdot E_1 + (1u + 1.5v) \cdot E_2$ collect
6. $(3, 5) = (2.5u - v, 1u + 1.5v)$ write component wise
7. $\begin{bmatrix} 3 \\ 5 \end{bmatrix} = \begin{bmatrix} 2.5 & -1 \\ 1 & 1.5 \end{bmatrix} \bullet \begin{bmatrix} u \\ v \end{bmatrix}$ write as matrix equation
8. $\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \frac{6}{19} & \frac{4}{19} \\ -\frac{4}{19} & \frac{10}{19} \end{bmatrix}^{-1} \bullet \begin{bmatrix} 3 \\ 5 \end{bmatrix}$ use inverse matrix (check!)
9. $\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 2 \\ 2 \end{bmatrix}$ read off the solution

Result: the coordinates of point $C(3, 5)_e$ in the other coordinate system $b = (b_1, b_2)$ are $C(2, 2)_b$. This is verified by looking in fig.2.

2nd we solve the problem using EIGENMATH following the above 9 steps.

```

e1 = (1, 0)
e2 = (0, 1)
b1 = (2.5, 1)
b2 = (-1, 1.5)

C = (3,5)      -- w.r.t. base e, i.e.:

-- 3*e1 + 5*e2 = u*b1 + v*b2
-- (3,5) = (2.5*u-v, 1.5*v+u)
-- we solve using inverse matrix

A = ((5/2, -1), (1, 3/2))
A
inv(A)
dot( inv(A), C)

```

▷ Run script.

EIGENMATH output:

$$\begin{bmatrix} \frac{6}{19} & \frac{4}{19} \\ -\frac{4}{19} & \frac{10}{19} \end{bmatrix}$$

$$\begin{bmatrix} 2 \\ 2 \end{bmatrix}$$

3rd we generalize the problem using general values for point C and basis b .

```

e1 = (1, 0)
e2 = (0, 1)
C  = (x, y)                                -- 1 */

-- other basis vectors b1, b2
b1 = a11*e1 + a21*e2
b2 = a12*e1 + a22*e2

-- ansatz for same point C(x,y) w.r.t. e and b
-- x*e1 + y*e2 = u*b1 + v*b2;              -- 2 */

-- read coordinates of C w.r.t. basis b
x = a11*u + a12*v
y = a21*u + a22*v

-- collect new coordinates of C as columns(!) in a matrix A
-- i.e. A is transpose of (b1,b2)

A = ((a11, a12), (a21, a22))

AU = dot( A, (u,v))                        -- means matrix product A*U
AU

-- kill content of x and y
x=quote(x)
y=quote(y)

-- (x,y) = A . (u,v)                       -- 3 */
--      X = A . U      => U = inverse(A) . X
U = dot(inv(A) , (x,y))                    -- 4 */
U

```

▷ Run script.

Remark. In 1: we represent the point C as a vector in EIGENMATH. In 2: you read the coordinates x and y of C w.r.t. e now expressed via the new coordinates (u, v) w.r.t. basis b . Line 3: writes 2: in matrix form, so that we can solve afterwards for the new coordinates (u, v) with matrix methods (inverse matrix) in 4:.

Nevertheless: you read off the solution formula for the new coordinates of C .

EIGENMATH output:

$$A_U = \begin{bmatrix} a_{11} u + a_{12} v \\ a_{21} u + a_{22} v \end{bmatrix}$$

$$U = \begin{bmatrix} -\frac{a_{12} y}{a_{11} a_{22} - a_{12} a_{21}} + \frac{a_{22} x}{a_{11} a_{22} - a_{12} a_{21}} \\ \frac{a_{11} y}{a_{11} a_{22} - a_{12} a_{21}} - \frac{a_{21} x}{a_{11} a_{22} - a_{12} a_{21}} \end{bmatrix}$$

Exercise 1. Use the general formula for U to redo the calculation in 1st or 2nd in one step.

Exercise 2. Calculate the length of C in the selected basis e resp. b , i.e. determine $|C_e|$ and $|C_b|$, where $|X| := \sqrt{X \bullet X}$, $X \in \mathbb{R}^2$.

Exercise 3. Calculate the angle between $\vec{OC}$ and the first 'axis' e_1 resp. b_1 , i.e. determine: $\angle(C_e, e_1)$ and $\angle(C_b, b_1)$.

Example 2. (Coordinate change by rotation)

We look at a counterclockwise rotation e.g. by 30° of the standard basis in the 2D Euclidean vectorspace $\mathbb{R}^2$. We ask: how does the coordinates of a point P change? What is the transformation rule?

Let by example $\vec{e}_1 := \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $\vec{e}_2 := \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ and $\vec{e}_1^* := T_{30^\circ}(\vec{e}_1)$, $\vec{e}_2^* := T_{30^\circ}(\vec{e}_2)$, where T_α represents the counterclockwise rotation with an angle of α .

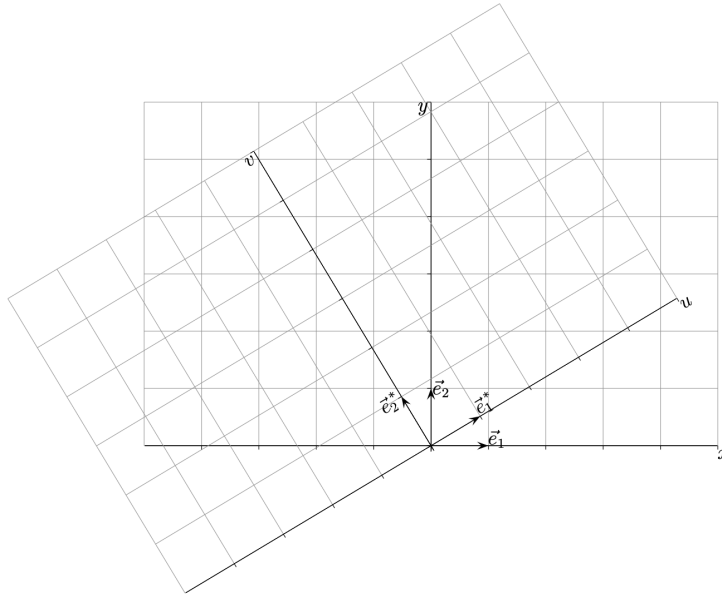


Figure 3: $\vec{e}_1, \vec{e}_2$: the standard Euclidean basis in $\mathbb{R}^2$.
 $\vec{e}_1^*, \vec{e}_2^*$: the rotated standard basis. [Picture by ROOLFS].

Determine

- the matrix of the transformation T_α , the counterclockwise rotation with an angle of α and T_{30° .
- the coordinates of point $P(2, 3)_e$ with it's coordinates w.r.t. basis $\vec{e} = (\vec{e}_1, \vec{e}_2)$ in the rotated coordinate system $\vec{e}^* = (\vec{e}_1^*, \vec{e}_2^*)$, i.e $P^*(?, ?)$?
- Be warned: *Not every matrix represents a tensor.*

Show: Let $A = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{bmatrix}$ be orthogonal³ and $T = \begin{bmatrix} T^{11} & T^{21} \\ T^{12} & T^{22} \end{bmatrix}$.

If $T^* = A^t \bullet T \bullet A$, then T represents a tensor

and in EINSTEIN's tensor notation we have $T_{ij}^* = a_{ki} a_{lj} T^{kl}$

³i.e. $A^{-1} = A^t$, which is the case for the transformation T_α from part a., where A is the invertible matrix of a coordinate change.

Solution:

Let's drop again the physicist notation and write vectors simplified noted as $\mathbf{e}_1$ or $E1$ etc.

1st we solve problems a. and b. using elementary Linear Algebra.

$$1. \quad T_\alpha = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} \quad \text{known from Linear Algebra}$$

$$2. \quad T_{30^\circ} = \begin{bmatrix} \cos(\pi/6) & -\sin(\pi/6) \\ \sin(\pi/6) & \cos(\pi/6) \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix}$$

$$3. \quad P = (2, 3) \equiv 2 \cdot E_1 + 3 \cdot E_2 = u \cdot E_1^* + v \cdot E_2^* \quad \text{unknown } u, v \text{ of } P \text{ w.r.t. basis } e^*$$

general approach for transformation from E to E^* :

$$4. \quad \begin{aligned} E_1^* &= a_{11} \cdot E_1 + a_{12} \cdot E_2 && \text{express } e_1^* \text{ w.r.t. basis } e \\ E_2^* &= a_{21} \cdot E_1 + a_{22} \cdot E_2 && \text{express } e_2^* \text{ w.r.t. basis } e \end{aligned}$$

From 1. we have $a_{11} = \cos(\alpha)$, $a_{12} = \sin(\alpha)$, etc., ergo

$$5. \quad \begin{bmatrix} x \\ y \end{bmatrix} = T_\alpha \left(\begin{bmatrix} u \\ v \end{bmatrix} \right) = \begin{bmatrix} \cos(\alpha) & -\sin(\alpha) \\ \sin(\alpha) & \cos(\alpha) \end{bmatrix} \cdot \begin{bmatrix} u \\ v \end{bmatrix} \quad \text{transformation rule } (u, v) \mapsto (x, y)$$

$$6. \quad \begin{bmatrix} u \\ v \end{bmatrix} = T_\alpha^{-1} \left(\begin{bmatrix} x \\ y \end{bmatrix} \right) = \begin{bmatrix} \cos(\alpha) & \sin(\alpha) \\ -\sin(\alpha) & \cos(\alpha) \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix} \quad \text{inverse transformation } (x, y) \mapsto (u, v)$$

$$7. \quad \begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{3}}{2} & -\frac{1}{2} \\ \frac{1}{2} & \frac{\sqrt{3}}{2} \end{bmatrix}^{-1} \cdot \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

$$8. \quad \begin{bmatrix} u \\ v \end{bmatrix} \approx \begin{bmatrix} 3.2 \\ 1.6 \end{bmatrix} \quad \text{read the solution}$$

Result: the coordinates of point $P(2, 3)_e = 2 \cdot \vec{e}_1 + 3 \cdot \vec{e}_2$ in the other coordinate system e^* are $P^*(3.2, 1.6)$, i.e. $P^* = 3.2 \cdot \vec{e}_1^* + 1.6 \cdot \vec{e}_2^*$. This is verified by looking in fig.3.

2nd we solve problems a. and b. using EIGENMATH .

```
T(alpha) = ( (cos(alpha),-sin(alpha)), (sin(alpha), cos(alpha)))
T(alpha)
T(pi/6)
inv(T(alpha))
Tinv = simplify(inv(T(alpha)))
Tinv
Tinv = simplify(inv(T(pi/6)))
dot(Tinv , (2,3))
float
```

▷ Run script.

EIGENMATH output:

$$\begin{bmatrix} \frac{\cos(\alpha)}{\cos(\alpha)^2 + \sin(\alpha)^2} & \frac{\sin(\alpha)}{\cos(\alpha)^2 + \sin(\alpha)^2} \\ -\frac{\sin(\alpha)}{\cos(\alpha)^2 + \sin(\alpha)^2} & \frac{\cos(\alpha)}{\cos(\alpha)^2 + \sin(\alpha)^2} \end{bmatrix}$$

$$\begin{bmatrix} 3^{1/2} + \frac{3}{2} \\ \frac{3}{2} 3^{1/2} - 1 \end{bmatrix}$$

$$\begin{bmatrix} 3.23205 \\ 1.59808 \end{bmatrix}$$

Exercise 4. Verify, that the length of P is independent of the selected basis e resp. e^* , i.e. show: $|P| = |P^*|$, where $|X| := \sqrt{X \bullet X}$, $X \in \mathbb{R}^2$. Control in the sketch with a ruler.

Exercise 5. Verify, that the angle between $\vec{OP}$ and the first 'axis' $E1$ resp. $E1^*$ is independent of the selected basis e resp. e^* , i.e. show: $\angle(P, E1) = \angle(P^*, E1^*)$. Control in the sketch with the protractor.

3^{rd} we solve c. using EIGENMATH.

```
A = ((a11, a12), (a21, a22))
-- A^-1 = A^t, assumed A orthogonal!
At = transpose(A)
At
T = ((T11,T12), (T21, T22))
dot(At , T , A)           -- 4 */
```

▷ Run script.

EIGENMATH output:

$$A_t = \begin{bmatrix} a_{11} & a_{21} \\ a_{12} & a_{22} \end{bmatrix}$$

$$\begin{bmatrix} T_{11} a_{11}^2 + T_{12} a_{11} a_{21} + T_{21} a_{11} a_{21} + T_{22} a_{21}^2 & T_{11} a_{11} a_{12} + T_{12} a_{11} a_{22} + T_{21} a_{12} a_{21} + T_{22} a_{21} a_{22} \\ T_{11} a_{11} a_{12} + T_{12} a_{12} a_{21} + T_{21} a_{11} a_{22} + T_{22} a_{21} a_{22} & T_{11} a_{12}^2 + T_{12} a_{12} a_{22} + T_{21} a_{12} a_{22} + T_{22} a_{22}^2 \end{bmatrix}$$

Now you may gather from here the RHS of the EINSTEIN notation $T_{ij}^* = a_{ki} a_{lj} T^{kl}$.

Comment. We elaborate a little on line (4*). Suppose we have the coordinates $U = [u, v]^t$ in basis e^* and the corresponding coordinates $X = [x, y]^t$ w.r.t. basis e . Then (4) reflects the coordinate change $T^* \cdot U = (A^t \cdot T \cdot A) \cdot U$, which is the composite mapping $U \xleftarrow{A^t} X^* \xleftarrow{T} X \xleftarrow{A} U$ and shows, that T leaves the coordinates invariant.

Example 3. (Inertia Tensor)

We follow ROOLFS [20]. We consider a system of rigidly coupled point masses of identical mass $m = 1$.

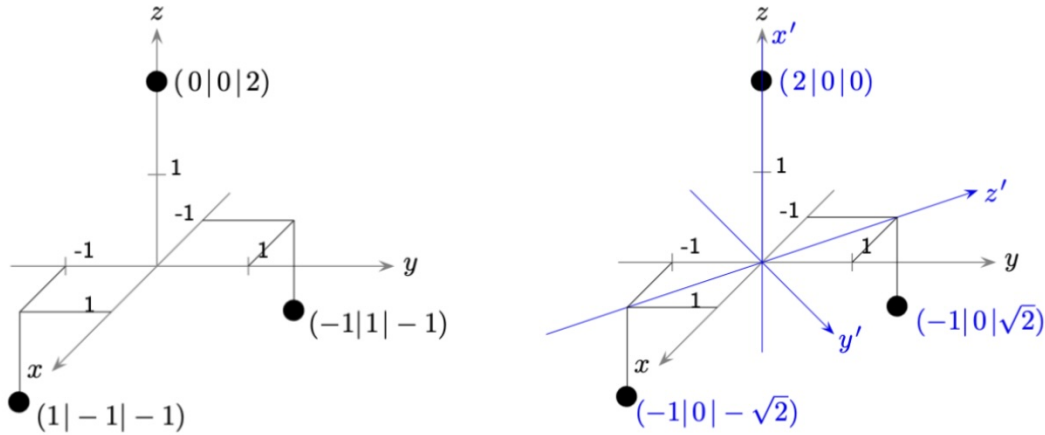


Figure 4: ●_{left}: three coupled point masses in $\mathbb{R}^3$ w.r.t. standard basis.
●_{right}: the points coordinates in the basis of the eigenvectors.

Verify the following facts about the configuration of fig.2:

- a. Verify, that the inertia tensor Θ of the configuration in fig.2^{left} can be determined as

$$\Theta = \begin{bmatrix} 8 & 2 & 0 \\ 2 & 8 & 0 \\ 0 & 0 & 4 \end{bmatrix}$$

w.r.t. the Euclidian basis $e_1 = (1, 0, 0)^t, e_2 = (0, 1, 0)^t, e_3 = (0, 0, 1)^t$ of $\mathbb{R}^3$, see [20].

- b. Verify, that the eigenvalues of matrix Θ are $\lambda_1 = 4, \lambda_2 = 10, \lambda_3 = 6$ with corresponding eigenvectors $w_1 = (1, -1, 0)^t, w_2 = (1, 1, 0)^t, w_3 = (0, 0, 1)^t$. Calculate the normalized eigenvectors (v_1, v_2, v_3) from (w_1, w_2, w_3) via $v_i := w_i/|w_i|$.
- c. Use the matrix $A := [v_1, v_2, v_3]$ of the normalized eigenvectors⁴ to calculate the new coordinates of the mass points w.r.t. the basis $v = (v_1, v_2, v_3)$. Check your result in fig.4.^{right}. [Tip: remember step 8. resp. formulas *3* and *4* of example.1.]

⁴the normalized eigenvectors v_i are the *columns* of matrix A .

- d. Verify, that the matrix Θ is a *tensor*, i.e. Θ transforms via

$$\Theta' = \begin{bmatrix} 4 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 6 \end{bmatrix} = A^t \cdot \Theta \cdot A$$

To quote ROOLFS:

“The diagonalization is not the remarkable aspect, but rather that $\Theta' = A^t \cdot \Theta \cdot A$ correctly describes the rotation around the new coordinate system. Therefore, it is not simply a renaming of the coordinates.”

1st We (re)do the calculations a. ... d. with EIGENMATH.

```

Theta = ((8,2,0),(2,8,0),(0,0,4))
Theta
Ev = eigenvec(Theta )
Ev
ev = dot( transpose(Ev), Theta, Ev)
-- eigenvalues on diagonal of EV
ev

--* b.:  the eigenvectors of Theta
v1 = (1, -1,  0)
v2 = (1,  1,  0)
v3 = (0,  0,  1)

--* Check if Theta . v_i = lambda_i * v_i
dot(Theta , v1)
dot(Theta , v2)
dot(Theta , v3)

unitvector(v) = v/abs(v)  -- abs: vector length of v

v1u = unitvector(v1)
v2u = unitvector(v2)
v3u = unitvector(v3)

--* c.:  A is matrix of normed eigenvectors as columns
A = transpose( (v1u,v2u,v3u) )

--*      calculate the new coordinates U via
--*      X = A . U => U = inverse(A) . X

dot( inv(A), ( 1,-1,-1))
dot( inv(A), (-1, 1,-1))
dot( inv(A), ( 0, 0, 2))

```

```
dot(Theta , A)

"d.: verify tensor property of Theta"
dot(inv(A) , Theta , A)
```

▷ Run script.

EIGENMATH output:

$$\begin{bmatrix} 2^{1/2} \\ 0 \\ -1 \end{bmatrix}$$

$$\begin{bmatrix} -2^{1/2} \\ 0 \\ -1 \end{bmatrix}$$

$$\begin{bmatrix} 0 \\ 0 \\ 2 \end{bmatrix}$$

$$\begin{bmatrix} 3 \cdot 2^{1/2} & 5 \cdot 2^{1/2} & 0 \\ -3 \cdot 2^{1/2} & 5 \cdot 2^{1/2} & 0 \\ 0 & 0 & 4 \end{bmatrix}$$

$$\begin{bmatrix} 6 & 0 & 0 \\ 0 & 10 & 0 \\ 0 & 0 & 4 \end{bmatrix}$$

2nd We now write the tensor transformation formula $\Theta' = A^t \cdot \Theta \cdot A$ using index notation.

Suppose we have two matrices $A = (A_{ij})_{i=1..m}^{j=1..n}$ and $B = (B_{jk})_{j=1..n}^{k=1..p}$, then the ik 's element C_{ik} of the product of two matrices A and B is calculated/defined as

$$\begin{aligned} C_{ik} &= A_{i,1} \cdot B_{1,k} + A_{i,2} \cdot B_{2,k} + \cdots + A_{i,m} \cdot B_{m,k} \\ &= \sum_{k=1}^n A_{ik} \cdot B_{kj} \\ &=: A_i^{\textcolor{violet}{k}} \cdot B_{\textcolor{violet}{k}}^j \quad \leftarrow \text{EINSTEIN summation convention} \end{aligned}$$

Remark: In the last expression we have discriminated precise between the *row indices* and the *column indices* of a matrix using a subscript ('lower index') for the row indices and a superscript ('upper index') for the column indices. Then we can apply the EINSTEIN summation convention, see section 1.0.

◦ Ergo, summation symbols $\sum$ can be 'eliminated or forgotten' by using EINSTEIN's convention, giving very compact tensorial notations - but with the drawback to decode the terms in your mind.

If we have a third matrix C with $C = (C_{k\ell})_{k=1..p}^{\ell=1..q}$, then the triple product $A \cdot B \cdot C^5$ is calculated via

$$\begin{aligned} ((A \cdot B) \cdot C)_{i=1..m}^{\ell=1..q} &= \sum_{k=1}^p (A \cdot B)_{ik} \cdot C_{k\ell} = \sum_{k=1}^p \left(\sum_{j=1}^n A_{ij} \cdot B_{jk} \right) \cdot C_{k\ell} \\ &= \sum_{k=1}^p \left(\sum_{j=1}^n A_i^j \cdot B_j^k \right) \cdot C_k^\ell \stackrel{\text{EINSTEIN}}{=} A_i^j \cdot B_j^k \cdot C_k^\ell \end{aligned}$$

We now specialize this to our matrices Θ and A and conclude

$$\begin{aligned} (\Theta')_{i=1..3}^{\ell=1..3} &= \sum_{k=1}^3 \left(\sum_{j=1}^3 (A^t)_i^j \cdot \Theta_j^k \right) \cdot A_k^\ell \\ &= \sum_{k=1}^3 \sum_{j=1}^3 ((A^t)_i^j \cdot \Theta_j^k) \cdot A_k^\ell \\ &= (A^t)_i^j \cdot \Theta_j^k \cdot A_k^\ell \end{aligned}$$

Therefore we get the tensor formula for the transformation rule

$$(\Theta')_i^\ell = (A^t)_i^j \cdot \Theta_j^k \cdot A_k^\ell$$

with A as invertible transformation, i.e. $A^t = A^{-1}$.

◦ Θ is a tensor of type $(1, 1)$, because we have one subscript and one superscript index.

3rd We formulate the tensor transformation formula $\Theta' = A^t \cdot \Theta \cdot A$ using index notation and the tensor package `itensor` from CAS MAXIMA:

```
load("itensor")$
ishow( At([i],[j]) * Theta([j],[k]) * A([k],[l]) )$
indices(%);
```

We see: in the term `A([k],[l])` the first bracket `[ ]` collects the *sub*scripted indices and the second `[ ]` the *super*script indices. YAMWI output:

```
[%i1] load("itensor")$
[%i2]
[%o2]  j  k  l
      At  Θ  A
      i  j  k
[%i3] indices(%);
[%o3]  [[1, i], [j, k]]
```

▷ Click here to RUN the code.
newpage

⁵Remember: the matrix product is associative.

2 What is a tangent vector?

We begin with three examples of tangent vectors to gain a visual mental image of this concept, which plays a central role beside the position vectors. Imagine a tangent vector as a vector that is tangent at a given point to a curve, a surface or a higher dimensional space, i.e. a 'manifold'. *Tangent vectors are examples of tensors of rank 1, type (1,0).*

Example 4. (Tangent vector on a Parabola)

Imagine a mass point that travels along a parabolic path over time at a certain speed. If the mass point leaves the path at a certain position, it leaves the parabolic path *tangentially*. We denote the path by M ('manifold'), the position vector of the point at time t by $R(t)$ and the velocity by v .

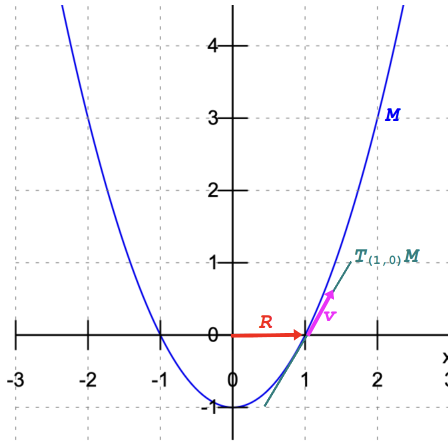


Figure 5:
 M : parabola $\{(x, y) \in \mathbb{R}^2 | y = x^2 - 1\}$.
 R : the position vector $R(1) = (0, 1)$.
 v : the velocity vector $R'(t) = (1, 2t)$.
 $T_{(0,1)}M$: tangent line at point $P = R(1)$ on M .

Then the velocity is defined as follows:

$$\mathbf{v} \quad \underset{\text{Math}}{:=} \quad \frac{\partial R}{\partial t} \quad \underset{\text{Eigenmath}}{=} \quad d(R(t), t) \quad (2.1)$$

- We calculate the velocity v in Fig.5. first mathematically.

Let $R : I \rightarrow \mathbb{R}^2$ with $t \mapsto (t, t^2)$ be differentiable on the open interval $I \subset \mathbb{R}$.

Then $R'(t) = \frac{\partial}{\partial t}(t, t^2 - 1) = (1, 2t)$ and $R'(1) = (1, 2) = v$ is the *tangential vector* at the time $t = 1$ alias at the point $P = (0, 1)$.

- Then we calculate v using EIGENMATH:

```

R(t) = (t,t^2-1)      --* parametrization of parabola
R(t)
v = d(R(t),t)         --* velocity
v
Tp = eval(v,t,1)      --* tangent vector at [0,1]
Tp

```

▷ Run script.

EIGENMATH output:

$$\begin{aligned} & \begin{bmatrix} t \\ t^2 - 1 \end{bmatrix} \\ v &= \begin{bmatrix} 1 \\ 2t \end{bmatrix} \\ T_p &= \begin{bmatrix} 1 \\ 2 \end{bmatrix} \end{aligned}$$

Exercise 6. (tangent vector on a circle)

The unit circle is a 1-dimensional manifold ('curve') in $\mathbb{R}^2$.

We recommend to describe this circle $S^1 := \{(x, y) \in \mathbb{R}^2 | x^2 + y^2 = 1\}$ via a parametrization using the position vector $R(t) := (\cos(t), \sin(t))$.

Calculate the tangent vector at $P = R(45^\circ)$.

You should reproduce the following scene:

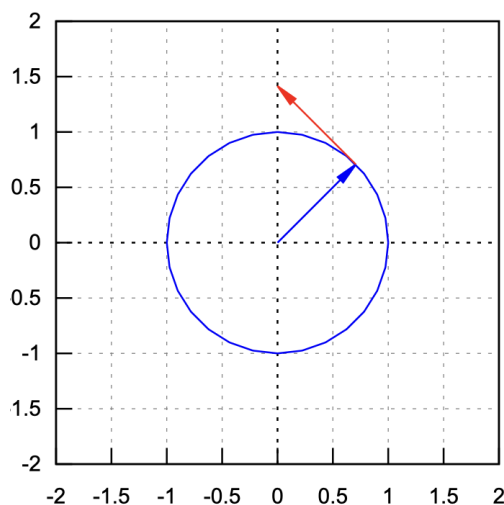


Figure 6: $R(\pi/4)$: position vector at time $t = \pi/4$ pointing on S^1 .
 $\partial_t R$: tangent vector $R'(\pi/4)$ at $R(\pi/4) \in S^1$.

Example 5. (Tangent vector on a Sphere)

The unit sphere S^2 is a 2-dimensional manifold ('surface') in the surrounding space $\mathbb{R}^3$. Let's describe the sphere $S^2 := \{(x, y, z) \in \mathbb{R}^3 | x^2 + y^2 + z^2 = 1\}$ by means of a parametrization using the position vector $R(\theta, \phi)$.

Let $R : U \rightarrow \mathbb{R}^3$ be defined by

$$(\theta, \phi) \mapsto (\cos(\theta) \cos(\phi), \cos(\theta) \sin(\phi), \sin(\theta))$$

and differentiable on an open subset $U \subset \mathbb{R}^2$ with $(\theta, \phi) \in U$.

Then we have two tangent vectors: $\mathbf{Z}_1 := \frac{\partial R}{\partial \theta}$ and $\mathbf{Z}_2 := \frac{\partial R}{\partial \phi}$ are the *tangential vectors* at the point $R(\theta, \phi) \in S^2$ with coordinates (θ, ϕ) , which span the tangential plane $T_P M$.

Task:

- Calculate the tangent vectors at $P = R(\theta, \phi)$.
- Calculate the tangent vectors at $P = R(45^\circ, 0)$.

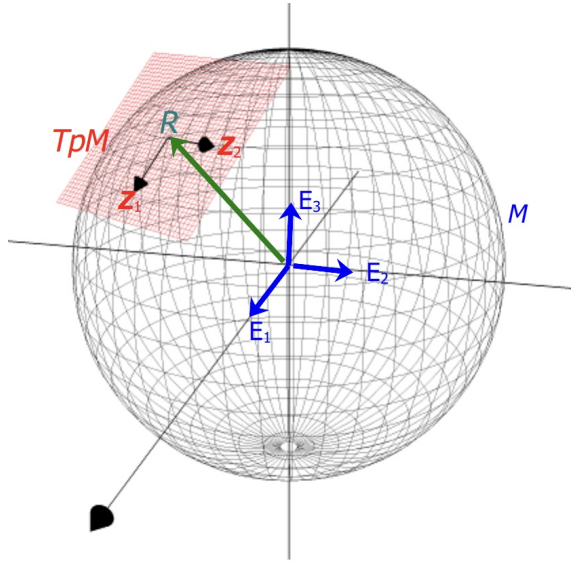


Figure 7: (E_1, E_2, E_3) : canonical Euclidian basis in $\mathbb{R}^3$.
 (Z_1, Z_2) : basis of tangent plane $T_P S^2$ at $R(\theta, \phi) \in S^2$.
 R : the position vector from the origin O to $R(\theta, \phi)$.

Solution:

1st mathematically.

a:

$$\begin{aligned}\mathbf{Z}_1 &= \frac{\partial R}{\partial \theta} \\ &= \frac{\partial}{\partial \theta}(\cos(\theta) \cos(\phi), \cos(\theta) \sin(\phi), \sin(\theta)) \\ &= (-\cos(\phi) \sin(\theta), -\sin(\phi) \sin(\theta), \cos(\theta))\end{aligned}$$

$$\begin{aligned}\mathbf{Z}_2 &= \frac{\partial R}{\partial \phi} \\ &= \frac{\partial}{\partial \phi}(\cos(\theta) \cos(\phi), \cos(\theta) \sin(\phi), \sin(\theta)) \\ &= (-\sin(\phi) \cos(\theta), \cos(\phi) \cos(\theta), 0)\end{aligned}$$

b:

$$\mathbf{Z}_1 \mid \left(\frac{\pi}{4}, 0\right) = \left(\frac{-1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}}\right)$$

$$\mathbf{Z}_2 \mid \left(\frac{\pi}{4}, 0\right) = \left(0, \frac{1}{\sqrt{2}}, 0\right)$$

2nd solution with EIGENMATH.

```
R(theta,phi) = (cos(theta)*cos(phi), cos(theta)*sin(phi), sin(theta))
```

```
Z1 = d( R(theta,phi), theta)
```

```
Z1
```

```
Z2 = d( R(theta,phi), phi)
```

```
Z2
```

```
eval(Z1, theta,pi/4, phi,0)
```

```
eval(Z2, theta,pi/4, phi,0)
```

▷ Run script.

EIGENMATH output:

$$\begin{aligned}Z_1 &= \begin{bmatrix} -\cos(\phi) \sin(\theta) \\ -\sin(\phi) \sin(\theta) \\ \cos(\theta) \end{bmatrix} & \begin{bmatrix} -\frac{1}{2^{1/2}} \\ 0 \\ \frac{1}{2^{1/2}} \end{bmatrix} \\ Z_2 &= \begin{bmatrix} -\cos(\theta) \sin(\phi) \\ \cos(\phi) \cos(\theta) \\ 0 \end{bmatrix} & \begin{bmatrix} 0 \\ \frac{1}{2^{1/2}} \\ 0 \end{bmatrix} \end{aligned} \text{ at } (\pi/2, 0) :$$

Remark. A tangent vector is a so-called *contravariant vector* and should conventionally better written in column form, e.g.

$$\mathbf{Z}_1 = (-\cos(\phi)\sin(\theta), -\sin(\phi)\sin(\theta), \cos(\theta))^t = \begin{bmatrix} -\cos(\phi)\sin(\theta) \\ -\sin(\phi)\sin(\theta) \\ \cos(\theta) \end{bmatrix}$$

Admittedly, we've been a bit lax in this regard up until now, but EIGENMATH respected it.

3 The Metric Tensor

The so-called *metric tensor*⁶ g (or simply metric) plays an crucial role in Differential Geometry and General Relativity. It allows defining and calculate distances and angles on surfaces ('manifolds') in space, just as the inner product do it on a Euclidean space $\mathbb{R}^n$.

3.1 Definition: metric tensor

Let $R : U \rightarrow \mathbb{R}^n$ be the position vector aka a 'coordinate system' of a manifold M and differentiable on the open subset $U \subset \mathbb{R}^n$ with coordinates $(Z_1, Z_2, \dots) \in U$.

- a. Define the tangent vectors $\mathbf{Z}_i$ by differentiation of R w.r.t. each of the coordinates

$$\mathbf{Z}_i := \frac{\partial R}{\partial Z_i} = \text{EigenMath} \, \mathbf{d}(R(x, y), x) \quad (3.1)$$

These tangent vectors $(\mathbf{Z}_1, \dots, \mathbf{Z}_n)$ ⁷ are also called the *covariant basis* of the n -dimensional tangent space $T_P M$ with inner product $\bullet$.

- b. Then, by definition, the (covariant) *metric tensor* $g = (g_{ij})$ consists of the pairwise dot products of the covariant basis vectors, i.e. for $i, j = 1, \dots, n$

$$g_{ij} := \mathbf{Z}_i \bullet \mathbf{Z}_j \quad (3.2)$$

The metric tensor is also called GRAM's matrix, therefore the notions g and g_{ij} .

- c. The corresponding *contravariant metric tensor* $ug := g^{-1}$ is defined by $g^{ij} := (g^{-1})_{ij}$.

Remark.

- The metric tensor is heavily used to do so-called *index juggling*, cf. [6, p.88], i.e. doing the tensor operations of *raising* or *lowering* an index. In this context is is essential to keep track of the position of an index, i.e. if the index is *superscripted* ('upper index') or *subscripted* ('lower index') in a tensor expression. For this purpose, the terms *covariant* ('lower') resp. *contravariant* ('upper') index are introduced to distinguish between them and to use the EINSTEIN summation convention correctly.

- The following lexicon helps to memorize these conventions.

`itensor` is a reference to this MAXIMA package for tensor calculus.

	LEXICON	Math	EIGENMATH	itensor
	<i>covariant tensor</i> g of type $(2,0)$:	g_{ij}	gdd	<code>g([i],[j],[])</code>
	<i>contra</i> variant tensor g of type $(0,2)$:	g^{ij}	guu	<code>g([],[i],[j])</code>
	<i>mixed</i> -variant tensor T of type $(1,1)$:	T_i^j	Tud	<code>T([i],[j])</code>

⁶aka *first fundamental form* or *fundamental tensor*

⁷We follow the notations in [6].

◦ The components of a metric tensor g in a coordinate basis is a symmetric matrix (tensor), whose entries transform 'covariantly' under changes of the coordinate system. Thus a metric tensor is a 'covariant' symmetric tensor, cf.[39].

Remark. In EIGENMATH we try to mimic the tensor notion of $_{ij}$ resp. ij resp. i_j by the identifier `dd` (read: 'down down') resp. `uu` (read: 'up up') resp. `ud` (read: 'up down').⁸

3.2 Examples

3.2.1 The Metric g in Cartesian coordinates in $\mathbb{R}^2$

The most elementary example is that of the two-dimensional Euclidean geometry, the Euclidean metric tensor g w.r.t. the canonical basis $E = (E_1, E_2) = (\begin{bmatrix} 1 \\ 0 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix})$ with *cartesian coordinates*. Calculate g .

Solution:

1st mathematically.

We have

$$g_{11} = E_1 \bullet E_1 = 1, \quad g_{12} = E_1 \bullet E_2 = 0, \quad g_{21} = E_2 \bullet E_1 = 0, \quad g_{22} = E_2 \bullet E_2 = 1$$

so we get the GRAM matrix alias the *metric tensor* g

$$g = (g_{ij}) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

and their inverse matrix

$$g^{-1} = (g^{ij}) = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} =: ug$$

2nd we use EIGENMATH.

```
# EIGENMATH
-- basis in Euclidean vector space R^2
E=((1,0),(0,1))

-- metric tensor in R^2
gdd = zero(2,2)
for(i,1,2,
  for(j,1,2,
    gdd[i,j]= dot(E[i],E[j]) ))
```

⁸This cool convention was introduced by G. WEIGT in his script about the SCHWARZSCHILD metric tensor, cf. [36].

```

gdd

-- Calculate guu, i.e. the inverse of gij

guu = inv(gdd)
guu

```

▷ Run script.

Remark. Remember: the tensor notion g_{ij} resp. g^{ij} is noted here as **gdd** (read: 'g down down') resp. **guu** (read: 'g up up').

EIGENMATH output:

$$g_{dd} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$g_{uu} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

3.2.2 Affine coordinates in $\mathbb{R}^2$

From GRINFELD [6, p.64]. Suppose we have affine coordinates with covariant basis $Z_1 = \begin{bmatrix} 2 \\ 0 \end{bmatrix}$ and $Z_2 = (\cos(\frac{\pi}{3}), \sin(\frac{\pi}{3}))^t = \begin{bmatrix} \frac{1}{2} \\ \frac{\sqrt{3}}{2} \end{bmatrix}$. Calculate the metric tensor and its inverse.

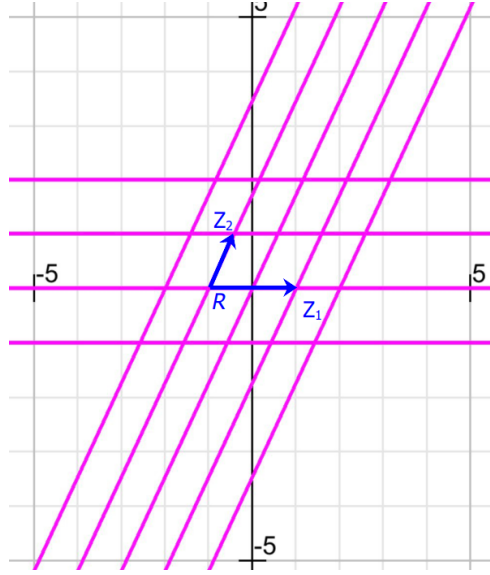


Figure 8: $R(x = -1, y = 0)$: position vector in ambient $\mathbb{R}^2$.
 Z_1 : 1st covariant basis vector at R .
 Z_2 : 2nd covariant basis vector at R .
 $/// \dots$: the coordinate grid corresponding basis Z .

Solution.

1st mathematically using Calculus.
 We have

$$g_{11} = Z_1 \bullet Z_1 = |Z_1|^2 = 4, \quad g_{12} = Z_1 \bullet Z_2 = 1, \quad g_{21} = Z_1 \bullet Z_2 = 1, \quad g_{22} = Z_2 \bullet Z_2 = 1$$

so we get the GRAM matrix g

$$g = (g_{ij}) = \begin{bmatrix} 4 & 1 \\ 1 & 1 \end{bmatrix}$$

and their inverse matrix

$$ug := g^{-1} = (g^{ij}) = \begin{bmatrix} \frac{1}{3} & \frac{-1}{3} \\ \frac{-1}{3} & \frac{4}{3} \end{bmatrix}$$

BTW, hence, in anticipation of §4.1, we get the so-called *contravariant* basis vectors $\mathbf{Z}^1$, $\mathbf{Z}^2$ via

$$\begin{aligned} Z^1 &:= \frac{1}{3}Z_1 - \frac{1}{3}Z_2 = \begin{bmatrix} \frac{1}{2} \\ \frac{-1}{2\sqrt{3}} \end{bmatrix} \\ Z^2 &:= -\frac{1}{3}Z_1 + \frac{4}{3}Z_2 = \begin{bmatrix} 0 \\ \frac{2}{\sqrt{3}} \end{bmatrix} \end{aligned}$$

2nd use EIGENMATH.

```
# EIGENMATH
-- we introduce affine coordinates in R^2 by
Z1 = (2,0)
Z2 = (cos(pi/3),sin(pi/3))
Z1
Z2
-- the covariant basis in affine coordinates
Z=(Z1,Z2)

abs(Z1)
abs(Z2)

-- metric tensor in affine coordinates
gdd = zero(2,2)
for(i,1,2,
  for(j,1,2,
    gdd[i,j]= dot(Z[i],Z[j]) ))

gdd

-- Calculate guu, i.e. the inverse of gij

guu = inv(gdd)
guu

-- the contravariant basis is
E=((1,0),(0,1))
Zu1 = dot(guu, E[1])
Zu1

Zu2 = dot(guu, E[2])
Zu2
```

▷ Run script.

EIGENMATH output:

$$g_{dd} = \begin{bmatrix} 4 & 1 \\ 1 & 1 \end{bmatrix}$$

$$g_{uu} = \begin{bmatrix} \frac{1}{3} & -\frac{1}{3} \\ -\frac{1}{3} & \frac{4}{3} \end{bmatrix}$$

$$Z_{u1} = \begin{bmatrix} \frac{1}{3} \\ -\frac{1}{3} \end{bmatrix}$$

3.2.3 The metric tensor g for the polar coordinate system

The Euclidean metric in polar coordinates (r, θ) can be calculated as follows, cf. [6, p. 65]. Look at the position vector $R(r, \theta) = (r \cos(\theta), r \sin(\theta))$. Then the basis vectors $\mathbf{Z}_i$ are the tangential vectors on the two coordinate lines, i.e.

$$\mathbf{Z}_r = \frac{\partial R}{\partial r} =: \partial_r \quad \mathbf{Z}_\theta = \frac{\partial R}{\partial \theta} =: \partial_\theta$$

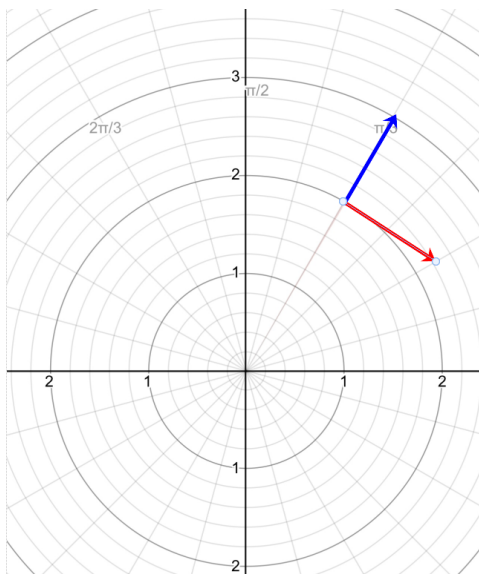


Figure 9: $R(r=2, \theta=\pi/3)$: position vector in $\mathbb{R}^2$.
 ∂_r : 1st covariant basis vector Z_r at R .
 ∂_θ : 2nd covariant basis vector Z_θ at R .

Let's calculate the basis and the metric using

```
# EIGENMATH
-- we introduce polar coordinates in R^2 by
R(r,theta) = (r cos(theta), r sin(theta))
R

-- the covariant basis in polar coordinates
Z = d(R,(r,theta))  -- both basis vectors are in the columns
Z = transpose(Z)     -- therefore we have to transpose
Z

-- metric tensor gij in polar coordinates
```

```

gdd = zero(2,2)
for(i,1,2,
    for(j,1,2,
        gdd[i,j]= dot(Z[i],Z[j]) ))

gdd
gdd = simplify(gdd)
gdd

-- Calculate guu, i.e. the inverse of gij
guu = inv(gdd)
guu

```

▷ Run script.

EIGENMATH output:

$$Z = \begin{bmatrix} \cos(\theta) & \sin(\theta) \\ -r \sin(\theta) & r \cos(\theta) \end{bmatrix}$$

$$g_{dd} = \begin{bmatrix} \cos(\theta)^2 + \sin(\theta)^2 & 0 \\ 0 & r^2 \cos(\theta)^2 + r^2 \sin(\theta)^2 \end{bmatrix}$$

$$g_{dd} = \begin{bmatrix} 1 & 0 \\ 0 & r^2 \end{bmatrix}$$

$$g_{uu} = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{r^2} \end{bmatrix}$$

3.2.4 The metric tensor g for the cylindrical coordinate system

The metric tensor in the cylinder coordinate systems in $\mathbb{R}^3$ can be calculated as follows, cf. [6, p.66, p.179]. Align a background Cartesian grid (x, y, z) with the cylindrical coordinates (r, θ, z) . The position vector R , pointing to a position on the cylinder, is then given by

$$R(r, \theta, z) = (r \cos(\theta), r \sin(\theta), z)$$

The covariant basis vectors $\mathbf{Z}_i$ are the tangential vectors on the three coordinate lines, i.e. the covariant basis is obtained by partial differentiation of R with respect to r, θ and z :

$$\mathbf{Z}_r = \frac{\partial R}{\partial r} \quad \mathbf{Z}_\theta = \frac{\partial R}{\partial \theta} \quad \mathbf{Z}_z = \frac{\partial R}{\partial z}$$

Remember: at each point $R(r, \theta, z)$ we get *another* 'new' basis $(\mathbf{Z}_r, \mathbf{Z}_\theta, \mathbf{Z}_z)$!

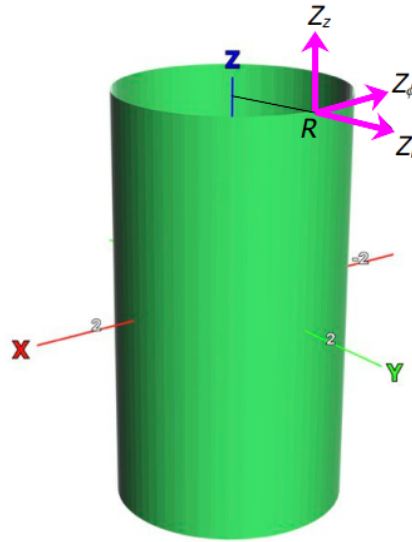


Figure 10: $R(r = 1, \theta = \pi/2, z = 2)$: position vector in $\mathbb{R}^3$.
 ∂_r : 1st covariant basis vector Z_r at R .
 ∂_θ : 2nd covariant basis vector Z_θ at R .
 ∂_z : 3rd covariant basis vector Z_z at R .

Task: calculate the metric tensor g and its inverse g^{-1} for the cylindrical coordinate system.

Solution:

We skip the mathematical calculation using Calculus (left to the reader) and do the Mathematics directly in EIGENMATH.

```
# EIGENMATH
-- we introduce cylindrical coordinates in R^2 by

R(r,theta,z) = (r cos(theta),r sin(theta),z)
R

-- the covariant basis in cylindrical coordinates
Z = d(R,(r,theta,z))  -- basis vectors are in the columns
Z = transpose(Z)      -- therefore we have to transpose
Z
Z[1]
Z[2]
Z[3]

-- metric tensor in cylindrical coordinates
gdd = zero(3,3)
for(i,1,3,
  for(j,1,3,
    gdd[i,j]= dot(Z[i],Z[j]) ))

gdd
gdd = simplify(gdd)
gdd

-- Calculate guu, i.e. the inverse of gij

guu = inv(gdd)
guu
```

▷ Run script.

EIGENMATH output:

$$g_{dd} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & r^2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

$$g_{uu} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{r^2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

4 Contravariant Basis

With the help of the tangent vectors $\mathbf{Z}_i$ and the covariant metric tensor $g = (g_{ij})$ we define the so-called *contravariant* basis $\mathbf{Z}^i$. These objects also play a major role in Tensor Analysis and Differential Geometry.

4.1 Definition: the contravariant basis

Let $R : U \rightarrow \mathbb{R}^n$ be the position vector of a manifold M and differentiable on the open subset $U \subset \mathbb{R}^n$ with coordinates $(Z_1, Z_2, \dots) \in U$.

Then we defined in §3.1

- the tangent vectors $\mathbf{Z}_i := \frac{\partial R}{\partial Z_i}$ by differentiation of R w.r.t. each of the coordinates, also called the *covariant basis* vectors of the tangent space $T_P M$.
- the covariant *metric tensor* g by $g_{ij} := \mathbf{Z}_i \bullet \mathbf{Z}_j$
- the *contravariant metric tensor* $ug := g^{-1}$ by $g^{ij} := (g^{-1})_{ij}$.

Now we define the i th *contravariant basis* vector $\mathbf{Z}^i$ by the linear combination⁹

$$\mathbf{Z}^i := g^{ij} \cdot \mathbf{Z}_j \stackrel{\text{Einstein}}{=} \sum_{j=1}^n g^{ij} * \mathbf{Z}_j \quad (4.1)$$

Remark. Remember the EINSTEIN summation convention: The *covariant* tangent vectors $\mathbf{Z}_i$ are written with one index as *subscript*, the corresponding *contravariant* basis vector $\mathbf{Z}^i$ with a *superscript* index. Then a summation in such a tensor expression is implicit done, when an index appears twice, once as a subscript ('lower index') and once as a superscript ('upper index').

4.2 Examples

4.2.1 The contravariant basis for polar coordinate system

We introduce polar coordinates in $\mathbb{R}^2$ by the position vector

$$R(r, \theta) := (r \cos(\theta), r \sin(\theta))$$

Then the covariant basis $\mathbf{Z}$ in polar coordinates is calculated by¹⁰

$$\begin{aligned} \mathbf{Z}_1 &:= \frac{\partial R}{\partial r} = (\cos(\theta), \sin(\theta)) \\ \mathbf{Z}_2 &:= \frac{\partial R}{\partial \theta} = (-r \sin(\theta), r \cos(\theta)) \\ \mathbf{Z} &:= (\mathbf{Z}_1, \mathbf{Z}_2) \end{aligned}$$

⁹cf. [6, p.58]

¹⁰sometimes we write $\mathbf{Z}_r$ or E_r instead of $\mathbf{Z}_1$ and $\mathbf{Z}_\theta$ or E_θ for $\mathbf{Z}_2$, see fig. 10.

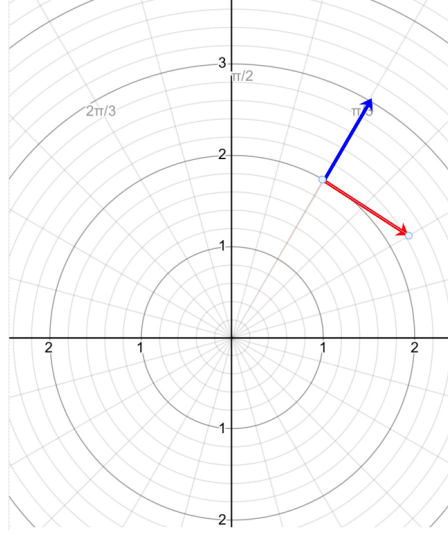


Figure 11: $\mathbf{Z}_r$: 1st covariant basis vector at $R(r=2, \theta=\pi/3)$.
 $\mathbf{Z}_\theta$: 2nd covariant basis vector at R .

We determine the *contravariant* basis in three ways.

1st we calculate the contravariant basis $\mathbf{uZ}$ using Calculus.

First we calculate the metric tensor g :

$$\begin{aligned}
 g_{11} &:= \mathbf{Z}_1 \bullet \mathbf{Z}_1 = (\cos(\theta), \sin(\theta)) \bullet (\cos(\theta), \sin(\theta)) = \cos^2(\theta) + \sin^2(\theta) = 1 \\
 g_{12} &:= \mathbf{Z}_1 \bullet \mathbf{Z}_2 = (\cos(\theta), \sin(\theta)) \bullet (-r \sin(\theta), r \cos(\theta)) = 0 \\
 g_{21} &:= \mathbf{Z}_2 \bullet \mathbf{Z}_1 = (-r \sin(\theta), r \cos(\theta)) \bullet (\cos(\theta), \sin(\theta)) = 0 \\
 g_{22} &:= \mathbf{Z}_2 \bullet \mathbf{Z}_2 = (-r \sin(\theta), r \cos(\theta)) \bullet (-r \sin(\theta), r \cos(\theta)) = r^2 \\
 g &= \begin{bmatrix} 1 & 0 \\ 0 & r^2 \end{bmatrix}
 \end{aligned}$$

Next we calculate the inverse metric tensor

$$g^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{r^2} \end{bmatrix} = \begin{bmatrix} g^{11} & g^{12} \\ g^{21} & g^{22} \end{bmatrix}$$

and get the contravariant basis p.def.

$$\begin{aligned}
 \mathbf{Z}^1 &= g^{11}\mathbf{Z}_1 + g^{12}\mathbf{Z}_2 \stackrel{\text{Einstein}}{=} g^{1i}\mathbf{Z}_i = 1 \cdot (\cos(\theta), \sin(\theta)) + 0 \cdot (-r \sin(\theta), r \cos(\theta)) \\
 &= (\cos(\theta), \sin(\theta)) \\
 \mathbf{Z}^2 &= g^{21}\mathbf{Z}_1 + g^{22}\mathbf{Z}_2 \stackrel{\text{Einstein}}{=} g^{2i}\mathbf{Z}_i = 0 \cdot (\cos(\theta), \sin(\theta)) + \frac{1}{r^2} \cdot (-r \sin(\theta), r \cos(\theta)) \\
 &= \left(-\frac{\sin(\theta)}{r}, \frac{\cos(\theta)}{r}\right) \\
 \mathbf{uZ} &:= (\mathbf{Z}^1, \mathbf{Z}^2)
 \end{aligned}$$

This calculation is a little expensive, so we
2nd calculate the contravariant basis with plain EIGENMATH.

```

-- we introduce polar coordinates in R^2 by
R(r,theta) = (r cos(theta),r sin(theta))

-- the covariant basis in cylindrical coordinates
Z1 = d(R(r, theta), r)
Z2 = d(R(r, theta), theta)
Z  = (Z1,Z2)
Z      -- Z is also transpose(d(R(r, theta), (r, theta)))

-- metric tensor in polar coordinates
gdd = zero(2,2)
      for(i,1,2,
          for(j,1,2, gdd[i,j]= dot(Z[i],Z[j])) )) -- Z[1] is 1st row of Z, ie. Z1
gdd
gdd = simplify(gdd)

-- Calculate guu, i.e. the inverse of gij
guu = inv(gdd)
guu

-- calculate contravariant basis
uZ1 = guu[1,1]*Z1 + guu[1,2]*Z2
uZ1
uZ2 = guu[2,1]*Z1 + guu[2,2]*Z2
uZ2
--* or do */
uZ1 = dot( guu[1] , Z)
uZ1
uZ2 = dot( guu[2] , Z)
uZ2

```

▷ Run script.

EIGENMATH output:

$$g_{uu} = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{r^2} \end{bmatrix}$$

$$u_{Z1} = \begin{bmatrix} \cos(\theta) \\ \sin(\theta) \end{bmatrix}$$

$$u_{Z2} = \begin{bmatrix} -\frac{\sin(\theta)}{r} \\ \frac{\cos(\theta)}{r} \end{bmatrix}$$

Remark. To discriminate between the covariant resp. contravariant basis we used the words 'co..' resp. 'contra..' and subscript resp. superscript indices in the mathematical notation. Using plain EIGENMATH we used the attributes 'd..' resp. 'u..' for down alias 'lower' resp. upper position of the indices. Special packages like `ctensor` or `itensor` from MAXIMA^{online} reflect that logic by means of index notation in brackets: `[covariant][contravariant]`, i.e. in the first `[..]` we collect all covariant indices and in the second `[..]` we collect all contravariant indices. . So we have:

$$\begin{aligned} \text{covariant metric : } g_{ij} &= \text{gdd} = \text{g}([\textcolor{red}{i}, \textcolor{red}{j}], []) \\ \text{contravariant metric : } g^{ij} &= \text{guu} = \text{g}([], [\textcolor{blue}{i}, \textcolor{blue}{j}]) \end{aligned}$$

Remark. The 1st contravariant basis vector $\mathbf{Z}^1$ in polar coordinates equals the first covariant basis vector $\mathbf{Z}_1$, whereas the 2nd contravariant basis vector $\mathbf{Z}^2$ is collinear with the second covariant basis vector and has length $|\mathbf{Z}^2| = 1/r$, see fig.11.

4.2.2 The contravariant basis on a sphere

The sphere S_r^2 with radius r is described by means of a parametrization using the position vector $R : U \rightarrow \mathbb{R}^3$ defined by

$$(r, \theta, \phi) \mapsto (r \sin(\theta) \cos(\phi), r \sin(\theta) \sin(\phi), r \cos(\theta))$$

which is differentiable on an open subset $U \subset \mathbb{R}^2$ with $(r, \theta, \phi) \in U$.

Task:

- Calculate the covariant basis $(\mathbf{Z}_1, \mathbf{Z}_2, \mathbf{Z}_3)$ and the contravariant basis $(\mathbf{Z}^1, \mathbf{Z}^2, \mathbf{Z}^3)$ ¹¹ :
 - a. at a general position $P = R(r, \theta, \phi)$.
 - b. at the special position $P = R(2, 45^\circ, 0)$.

¹¹the latter is denoted `(uZ1,uZ2,uZ3)` in EIGENMATH script below, 'u' means 'upper' index.

Solution: Because the calculation is very laborious, we use right away EIGENMATH.

a. We formulate the position vector R as transformation rule.

```
-- we introduce spherical coordinates in R^3 by
R(r,theta,phi) = (r*sin(theta)*cos(phi),
r*sin(theta)*sin(phi),
r*cos(theta))

-- the covariant basis in spherical coordinates
Z = transpose(d(R(r, theta,phi), (r, theta,phi)))
Z = simplify(Z) -- Z[1] is 1st row of Z, i.e. Z1
Z

-- metric tensor in polar coordinates
gdd = zero(3,3)
for(i,1,3,
for(j,1,3, gdd[i,j]= simplify(dot(Z[i],Z[j]))))
gdd
gdd = simplify(gdd)

-- We help Eigenmath: 1-2*cos(2x) = 2*sin^2(x)
-- Therefore we simplify
gdd[3,3] = r^2*sin(theta)^2
gdd

-- Calculate guu, i.e. the inverse of gij
guu = inv(gdd)
guu

-- calculate contravariant basis
uZ1 = guu[1,1]*Z[1] + guu[1,2]*Z[2] + guu[1,3]*Z[3]
uZ1
uZ2 = guu[2,1]*Z[1] + guu[2,2]*Z[2] + guu[2,3]*Z[3]
uZ2
uZ3 = guu[3,1]*Z[1] + guu[3,2]*Z[2] + guu[3,3]*Z[3]
uZ3

-- or in one stroke, basis vectors in the rows
uZ = zero(3,3)
for(i,1,3, uZ[i] = sum(j,1,3, guu[i,j]*Z[j]))
uZ
uZ1 = uZ[1]
uZ1
```

▷ Run script.

EIGENMATH output:

$$\begin{aligned}
 g_{dd} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & r^2 & 0 \\ 0 & 0 & \frac{1}{2}r^2 - \frac{1}{2}r^2 \cos(2\theta) \end{bmatrix} & u_{Z1} &= \begin{bmatrix} \cos(\varphi) \sin(\theta) \\ \sin(\varphi) \sin(\theta) \\ \cos(\theta) \end{bmatrix} \\
 g_{dd} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & r^2 & 0 \\ 0 & 0 & r^2 \sin(\theta)^2 \end{bmatrix} & u_{Z2} &= \begin{bmatrix} \frac{\cos(\varphi) \cos(\theta)}{r} \\ \frac{\cos(\theta) \sin(\varphi)}{r} \\ -\frac{\sin(\theta)}{r} \end{bmatrix} \\
 g_{uu} &= \begin{bmatrix} 1 & 0 & 0 \\ 0 & \frac{1}{r^2} & 0 \\ 0 & 0 & \frac{1}{r^2 \sin(\theta)^2} \end{bmatrix} & u_{Z3} &= \begin{bmatrix} -\frac{\sin(\varphi)}{r \sin(\theta)} \\ \frac{\cos(\varphi)}{r \sin(\theta)} \\ 0 \end{bmatrix}
 \end{aligned}$$

b. Substituting the concrete values of $R = (2, 45^\circ, 0) = (2, \frac{\pi}{4}, 0)$ into Z_1, Z_2, Z_3 we get the following figure:

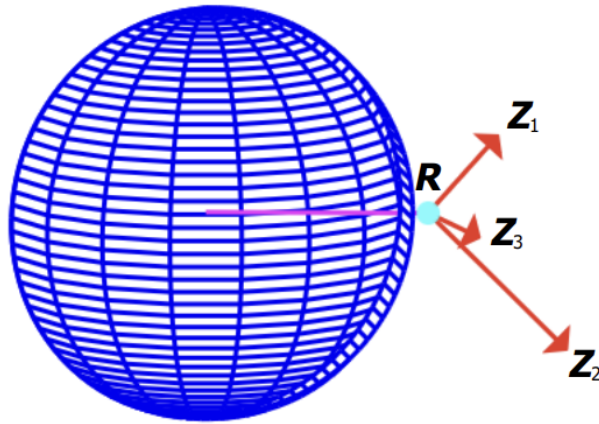


Figure 12: R : the position vector at $R(r = 2, \theta = \pi/4, \phi = 0)$.
 Z_1, Z_2, Z_3 : the covariant basis vectors at R .

◦ If you want to change the viewpoint or make other changes to the sketch, you find the source code here: ▷ [Click here to RUN the code](#).¹²

¹²I am very grateful to Michael GOSSE for fixing my code.

5 What is a tensor?

From wikipedia: ▷ Tensor. 'Each type of tensor comes equipped with a transformation law that details how the components of the tensor respond to a change of basis. The components of a vector can respond in two distinct ways to a change of a basis, see covariance and contravariance of vectors.'

A very understandable decription is by V. TOTH.

Here is the correct and precise mathematical Wikipedia definition of a tensor, in case you want to be shocked and will feel incomprehensible - we therefore cite this definition in tiny print, you may skip it:

5.1 Definition: Tensor

- (Wikipedia, ibid.) A *tensor of type* (p, q) is an assignment of a multidimensional array $T_{j_1 \dots j_q}^{i_1 \dots i_p}[\mathbf{f}]$ to each basis $\mathbf{f} = (\mathbf{e}_1, \dots, \mathbf{e}_n)$ of an n -dimensional vector space such that,

if we apply the change of basis $\mathbf{f} \mapsto \mathbf{f} \cdot R = (\mathbf{e}_i R_1^i, \dots, \mathbf{e}_i R_n^i)$

then the multidimensional array obeys the transformation law

$$T_{j'_1 \dots j'_q}^{i'_1 \dots i'_p}[\mathbf{f} \cdot R] = (R^{-1})_{i_1}^{i'_1} \dots (R^{-1})_{i_p}^{i'_p} T_{j_1 \dots j_q}^{i_1 \dots i_p}[\mathbf{f}] R_{j_1}^{j'_1} \dots R_{j_q}^{j'_q}.$$

Instead, we cite Peter COLLIER's description [4, p.140] to get a feeling what a tensor 'is':

- a *mathematical object* represented by a collection of components that transform a certain way.
- a *linear machine* that produces numbers when fed with covariant and contravariant vectors. Each lower index represent a slot that can be fed by a covariant vector, each upper index represents a slot that can be fed by a contravariant vector. When all the slots are filled, out pops a number.

Remark.

- V. Toth¹³ gives the following overview for the representation of tensors:

Representation of a contravariant vector	$\begin{bmatrix} 1 \\ 2 \end{bmatrix}$
Representation of a covariant vector	$\begin{bmatrix} 1 & 2 \end{bmatrix}$
Representation of a (2,0) tensor	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
Representation of a (1,1) tensor	$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$
Representation of a (0,2) tensor	$\begin{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 3 \\ 4 \end{bmatrix} \end{bmatrix}$
The Christoffel-symbol in 2D	$\Gamma_{ij}^k = \begin{bmatrix} [\Gamma_{00}^0 \Gamma_{01}^0] & [\Gamma_{10}^0 \Gamma_{11}^0] \\ [\Gamma_{00}^1 \Gamma_{01}^1] & [\Gamma_{10}^1 \Gamma_{11}^1] \end{bmatrix}$

¹³in a private communication

◦ TOTH distinguish three ways of looking at tensors: in terms of its representation (i.e. a matrix), in terms of the algebraic rules of matrix manipulation and third with focus on the purely algebraic properties of tensors, see ▷ Tensors.

◦ We often denote an Euclidean vector $v = (a_1, \dots, a_n) \in \mathbb{R}^n$ simply as a list of entries, e.g. $v = (3, 2) \in \mathbb{R}^2$. In the language of tensors we should precise discriminate between a **c**ovariant vector $v = [3 \ 2] = [v_1 \ v_2]$ i.e. a tensor of type $(1, 0)$ with *one lower index* vs. a **c**ontravariant vector $v = \begin{bmatrix} v^1 \\ v^2 \end{bmatrix} = \begin{bmatrix} 3 \\ 2 \end{bmatrix}$ i.e. a tensor of type $(0, 1)$ with *one upper index* and reflect this distinction as far as possible in the description in the CAS.

◦ So we may say: a tensor with respect to a basis is represented by a multidimensional array. In a figurative rough sense, tensors 'are' 'multidimensional' matrices.

To make this complicated definition §5.1 more grounded, we will show some simple cases.

5.2 Example: This is *not* a tensor!

We define and test an indexed object T , cf. the example by GRINFELD [6, p.94].

$$T_{ijk} := i^2 j k$$

First we represent T with the EIGENMATH, because the output is easy to interpret:

```
## EIGENMATH
i = quote(i)                -- clear 'i', the imaginary unit ..

-- define and calculate elements of T as function values

Tijk(i,j,k) = i^2*j*k       -- would be -jk, if 'i' wasn't cleared
Tijk                --(1)
Tijk(1,2,1)           --(2) 1st matrix, 2nd row, 1st entry

-- T as multidimensional array

Tijk = zero(2, 2,2)         -- empty container for object T
for(i,1,2,                -- three loops to fill container Tijk
  for(j,1,2,
    for(k,1,2,  Tijk[i,j,k] = i^2*j*k)))
Tijk                -- (3)
Tijk[1,2,1]          -- (4) 1st matrix, 2nd row, 1st entry
```

▷ Run script.

EIGENMATH output:

$$T_{ijk} = i^2 j k$$

2

$$T_{ijk} = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \\ \begin{bmatrix} 4 & 8 \\ 8 & 16 \end{bmatrix} \end{bmatrix}$$

Figure 13: 1: T defined as function $T_{ijk} := i^2 j k$.
 2: value of T at index triple $(i, j, k) = (1, 2, 1)$ is 2.
 3: T displayed as multidimensional array $[(::), (::)]$.

Comment. We define object T in two ways. First T is defined by computing each individual value by the term $i^2 j k$ in (1). Here you have to address the value via `Tijk (..)` for the function call. Alternatively T is defined in (2) as a multidimensional array whose values are read in with matrix-access brackets `Tijk[. .]`. In this way the complete table of T -values is available at once.

Second we use the `ctensor` package from YAMWI= MAXIMA^{online} to do the same job:

```
load(ctensor);
for i thru dim do
  for j thru dim do
    for k thru dim do    T [i,j,k] : i^2*j*k;
T[1,2,1];
cdisplay(T);
```

▷ Click here to RUN the code in MAXIMA^{online}.

YAMWI output:

```
[%i3] T[1,2,1];
[%o3] 2
[%i4] cdisplay(T);
[%o4] T1 =  $\begin{pmatrix} 1 & 2 & 3 & 4 \\ 2 & 4 & 6 & 8 \\ 3 & 6 & 9 & 12 \\ 4 & 8 & 12 & 16 \end{pmatrix}$ 
[%o4] T2 =  $\begin{pmatrix} 4 & 8 & 12 & 16 \\ 8 & 16 & 24 & 32 \\ 12 & 24 & 36 & 48 \\ 16 & 32 & 48 & 64 \end{pmatrix}$ 
```


Comment. We see that `ctensor` interpret the term $T[i, j, k] : i^2 * j * k$ in another way, i.e. tensoriell. So this is a hint that original object is *not* a tensor. GRINFELD names such indexed objects *variants*, because they does not follow the transformation rules. When the same rule is applied in different coordinate systems, the results of a variant differ, i.e. they are not the same. *Not a tensor, but only a variant!*

Exercise 7. (Variant T_{ijkl} with prescribed values.)

$$T_{ijkl} = \left[\begin{array}{cc} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} & \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix} \\ \begin{bmatrix} 4 & 8 \\ 8 & 16 \end{bmatrix} & \begin{bmatrix} 8 & 16 \\ 16 & 32 \end{bmatrix} \end{array} \right]$$

1. Determine the function and the table-oriented formula for the given variant T .
2. Do access for the value 16 in the 3th matrix inside variant T .
3. Represent T in EIGENMATH. Think about the results.

Solution:

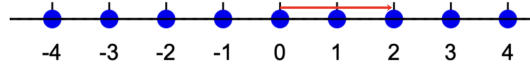
1. We give the defining relation as function value, [▷ Click here to see the solution.](#)
2. We define T as a tensor 'table', [▷ Click here to look at the ssolution.](#)
3. to be left as an exercise.

You should see the variant T as shown in the beginning of the exercise.

5.3 Examples: These *are* tensors

5.3.1 1D scaling transformation

In a one dimensional space (imagine the number line), a scaling transformation is a linear operation that stretches or compresses all points relative to a fixed origin 0. The coordinate system is changed by e.g. doubling the length of the unit, i.e. changing the unit from 1-cm to 2-cm units.



1. *Visualization*: Imagine a number line labeled with markers at $..-3, -2, -1, 0, 1, 2, 3, ..$. If you apply a scale factor of $s = 2$, the marker at 1 moves to 2, 2 moves to 4, -1 moves to -2. The space expands outward from zero. If you apply $s=0.5$, the space shrinks toward zero.
2. *Coordinate transformation*: Every vector (a number) x is mapped to a new number x' by a scale factor $s = 2$: $x' = s \cdot x$. The new coordinate x' is double of the old coordinate x .
3. *Jacobian*: $J_x^{x'} := \frac{\partial x'}{\partial x} = \frac{\partial}{\partial x}(2x) = 2 = s$.
4. *Vector transformation*: Let the one dimensional vector (number) be $v = 3$ units in the old system, then $v' = J_x^{x'} \cdot v = 2 \cdot 3 = 6$. In the new system the vector v has new component $v' = 6$.

We now use EIGENMATH .

```

R(x) = (2*x)           -- position vector i.e. coordinate change
Z     = d(R(x), x)     -- the Jacobian, i.e. the covariant basis
Z
g11 = dot(Z,Z)
g11
g    = ((g11))         -- matrix with one element
g
ug   = inv(g)
ug
uZ   = dot(ug,Z)       -- the contravariant basis
uZ

```

▷ Run script.

$$\begin{aligned}
 Z &= 2 \\
 g_{11} &= 4 \\
 g &= 4 \\
 u_g &= \frac{1}{4} \\
 u_Z &= \frac{1}{2}
 \end{aligned}$$

YAMWI output:

Example 6. The vectors of the covariant basis $(\mathbf{Z}_1, \mathbf{Z}_2, \mathbf{Z}_3)$ and the contravariant basis $(\mathbf{Z}^1, \mathbf{Z}^2, \mathbf{Z}^3)$ are tensors. The proof uses the chain rule and is shown e.g. in [6, p.77]. The result is: if we relate $\mathbf{Z}_{i'}$ with its primed coordinates and $\mathbf{Z}_i$ with the unprimed, we get

$$\mathbf{Z}_{i'} = \mathbf{Z}_i \cdot J_{i'}^i$$

where $J_{i'}^i$ is the so-called *Jacobian* of the coordinate transformation defined by $J_{i'}^i := \frac{\partial Z^i(Z')}{\partial Z^{i'}}$. See V. TOTH in ▷ On tensors and their matrix representations.

5.3.2 Example: the line element ds^2 transforms *contravariantly*

In the following example, we want to show that *a vector, viewed as a tensor*, is more than just an object with magnitude and direction. In particular, we show that the *line element* ds^2 transforms *contravariantly* from one coordinate system to another.

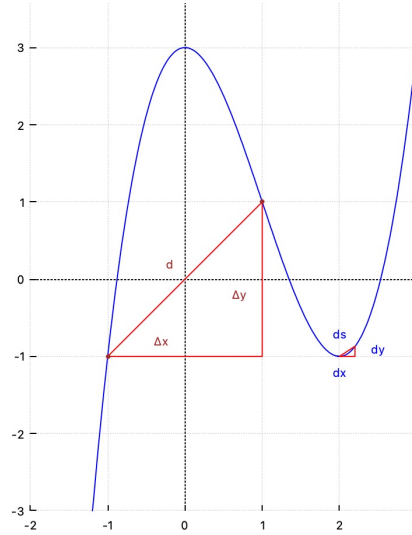


Figure 14: △: distance between two points via $d^2 = (\Delta x)^2 + (\Delta y)^2$
△: and two infinitesimal close points via $(ds)^2 = (dx)^2 + (dy)^2$

To calculate the squared distance d^2 between the two points $P(-1, -1) = (x^1, y^1)$ and $Q(1, 1) = (x^2, y^2)$ we do $d^2 = (x^2 - x^1)^2 + (y^2 - y^1)^2 = (1 + 1)^2 + (1 + 1)^2 = 8$, ergo $d = 2\sqrt{2}$. This motivates the definition of the *line element* ds resp. its square ds^2 as the distance of two infinitesimal close points with tiny displacements dx, dy in a two dimensional space:

$$ds^2 = dx^2 + dy^2$$

This definition extends immediately to higher dimensions.

Looking at the multiplication table of the infinitesimal displacements

$$g = (g_{ij}) = \begin{array}{cc} & \begin{array}{cc} dx & dy \end{array} \\ \begin{array}{c} dx \\ dy \end{array} & \begin{array}{cc} 1 & 0 \\ 0 & 1 \end{array} \end{array}$$

we may accept, that the coefficients of the metric tensor g_{ij} are often described using the line element ds^2 .

Task: Find the line element and the metric of $\mathbb{R}^2$ in the polar coordinate system.

Solution:

1st We know that $x = r \cos(\theta) =: x(r, \theta)$ and $y = r \sin(\theta) =: y(r, \theta)$ depends explicit of r and θ . Therefore, on an infinitesimal small scale, the differentials dx resp. dy depend linearly on the differentials of r and θ .

With the abbreviation $ds^2 \equiv (ds)^2, dx^2 \equiv (dx)^2, dr^2 \equiv (dr)^2$ etc. we argue

$$\begin{aligned} dx &= \cos(\theta) \cdot dr - r \cdot \sin(\theta) \cdot d\theta \\ dy &= \sin(\theta) \cdot dr + r \cdot \cos(\theta) \cdot d\theta \\ ds^2 &= dx^2 + dy^2 \\ &= (\cos(\theta) \cdot dr - r \cdot \sin(\theta) \cdot d\theta)^2 + (\sin(\theta) \cdot dr + r \cdot \cos(\theta) \cdot d\theta)^2 \\ &= \dots \\ &= \cos^2(\theta) \cdot dr^2 + r^2 \cdot \sin^2(\theta) \cdot d\theta^2 + \sin^2(\theta) \cdot dr^2 + r^2 \cdot \cos^2(\theta) \cdot d\theta^2 \\ &= dr^2 + r^2 \cdot d\theta^2 \end{aligned}$$

From the line element $ds^2 = 1 \cdot dr^2 + r^2 \cdot d\theta^2$ we may read off the metric tensor

$$g = (g_{ij}) = \begin{bmatrix} 1 & 0 \\ 0 & r^2 \end{bmatrix}$$

2nd We transform the line element ds^2 in $\mathbb{R}^2$ from polar (r, θ) to Cartesian (x, y) coordinates a little more abstract in order to show the transformation rule.

From 1st we get

$$\begin{aligned} 1. \quad dx &= \frac{\partial x}{\partial r} dr + \frac{\partial x}{\partial \theta} d\theta && \text{because } x \text{ depends on } r \text{ and } \theta \\ dy &= \frac{\partial y}{\partial r} dr + \frac{\partial y}{\partial \theta} d\theta && \text{also } y(r, \theta) \text{ is function of } r \text{ and } \theta \\ 2. \quad \begin{bmatrix} dx \\ dy \end{bmatrix} &= \begin{bmatrix} \frac{\partial x}{\partial r} & \frac{\partial x}{\partial \theta} \\ \frac{\partial y}{\partial r} & \frac{\partial y}{\partial \theta} \end{bmatrix} \bullet \begin{bmatrix} dr \\ d\theta \end{bmatrix} && \text{write as matrix equation} \end{aligned}$$

To get a tensorial notation of 1. resp. 2. we switch to index notation using the lexicon $\begin{smallmatrix} x \\ y \end{smallmatrix} \mid \begin{smallmatrix} \bar{x}^1 \\ \bar{x}^2 \end{smallmatrix}$ for primed (bar'ed) coordinates and unprimed (unbar'ed) coordinates $\begin{smallmatrix} r \\ \theta \end{smallmatrix} \mid \begin{smallmatrix} x^1 \\ x^2 \end{smallmatrix}$. Now 2. is written as

$$\begin{aligned} 3. \quad \begin{bmatrix} d\bar{x}^1 \\ d\bar{x}^2 \end{bmatrix} &= \begin{bmatrix} \frac{\partial \bar{x}^1}{\partial x^1} & \frac{\partial \bar{x}^1}{\partial x^2} \\ \frac{\partial \bar{x}^2}{\partial x^1} & \frac{\partial \bar{x}^2}{\partial x^2} \end{bmatrix} \bullet \begin{bmatrix} dx^1 \\ dx^2 \end{bmatrix} && \text{write with un/primed coordinates} \\ 4. \quad d\bar{x}^1 &= \sum_{k=1}^2 \frac{\partial \bar{x}^1}{\partial x^k} \cdot dx^k, \quad d\bar{x}^2 = \sum_{k=1}^2 \frac{\partial \bar{x}^2}{\partial x^k} \cdot dx^k && \text{write down as 2 individual equations} \end{aligned}$$

5. $d\bar{x}^j = \sum_{k=1}^2 \frac{\partial \bar{x}^j}{\partial x^k} \cdot dx^k$ write as *one* compact equation using free index $j = 1, 2$

6.

$$d\bar{x}^j = \frac{\partial \bar{x}^j}{\partial x^k} \cdot dx^k$$

even more compact using EINSTEIN summation convention

In summa: the tensor equation 6. is equivalent to the *both* equations 3. resp. 4.

If we rename $\frac{\partial \bar{x}^j}{\partial x^k} =: a_{jk}$ and $d\bar{x}^j =: \bar{T}^j$ resp. $dx^j =: T^j$ we arrive at the familiar transformation term

$$\bar{T}^j = a_{ij} T^j$$

where the factors a_{ij} represents the elements of a transformation matrix for *contravariant vector components* between the unprimed 'old' $(r, \theta) = (x^1, x^2)$ and the primed 'new' $(x, y) = (\bar{x}^1, \bar{x}^2)$ coordinates.

3rd But these factors $\frac{\partial \bar{x}^j}{\partial x^k}$ in 6. are also the components of the basis vectors tangent to the original (unprimed) coordinate axes, expressed in the new (primed) coordinate system, e.g. the 1st column of the transformation matrix in 3. is the first tangential vector E_r w.r.t. the position vector $R(r, \theta) := (r \cos(\theta), r \sin(\theta)) = (x, y)$

$$\begin{aligned} E_r &= \frac{\partial \bar{x}^1}{\partial x^1} \cdot E_x + \frac{\partial \bar{x}^2}{\partial x^1} \cdot E_y = \cos(\theta) \cdot E_x + \sin(\theta) \cdot E_y = (\cos(\theta), \sin(\theta)) \stackrel{!}{=} \frac{\partial R}{\partial r} \\ E_\theta &= \frac{\partial \bar{x}^1}{\partial x^2} \cdot E_x + \frac{\partial \bar{x}^2}{\partial x^2} \cdot E_y = -r \sin(\theta) \cdot E_x + r \cos(\theta) \cdot E_y = (-r \sin(\theta), r \cos(\theta)) \stackrel{!}{=} \frac{\partial R}{\partial \theta} \end{aligned}$$

with the Cartesian basis $E_x := \begin{bmatrix} 1 \\ 0 \end{bmatrix}, E_y := \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, i.e. we have

$$E_r = \frac{\partial x}{\partial r} \cdot E_x + \frac{\partial y}{\partial r} \cdot E_y, \quad E_\theta = \frac{\partial x}{\partial \theta} \cdot E_x + \frac{\partial y}{\partial \theta} \cdot E_y$$

Task:

- Redo the discussion using EIGENMATH. Calculate $E_{r=2}$ and $E_{\theta=30^\circ}$.
- Determine the normalized polar unit vectors $\hat{E}_r$ and $\hat{E}_\theta$.¹⁴
- Determine the polar line element s^2 using infinitesimal argumentation, see fig. 13.¹⁵

¹⁴We note the unit vectors with 'hat'-sign, i.e. $\hat{a} := a/|a|$.

¹⁵from MARSDEN, [14, p.500]

Solution:

ad **a.** and **b.** :

```

x(r, theta) = r*cos(theta)
y(r, theta) = r*sin(theta)
U = (r, theta)                                -- the polar coordinates .. and

d( x(r,theta), r)                             -- the 4 elements of the Jacobean matrix:
d( x(r,theta), theta)                         -- alias the transformation matrix
d( y(r,theta), r)                             -- from polar (r,theta)
d( y(r,theta), theta)                         -- to Cartesian (x,y) coordinates.

"Jacobian of (x,y) w.r.t. each polar variable"
f(r,theta) = (r*cos(theta), r*sin(theta))
J(r,theta) = d(f(r,theta), U)
J(r,theta)
-- or: J(r,theta) = (d( x(r,theta), r), d( x(r,theta), theta))
eval(J(r,theta), r,2, theta, pi/6)

Ex = (1,0)
Ey = (0,1)

Er = d( x(r,theta), r) * Ex + d( y(r,theta), r) * Ey
Er

Etheta = d( x(r,theta), theta) * Ex + d( y(r,theta), theta) * Ey
Etheta

unitvector(v)= v/abs(v)

Eru = simplify( unitvector(Er))
Eru
Ethetau = simplify( unitvector(Etheta))
Ethetau

Er2 = eval(Er, r,2, theta,pi/6)
Er2
Etheta30 = eval(Etheta, r,2, theta,pi/6)
Etheta30

```

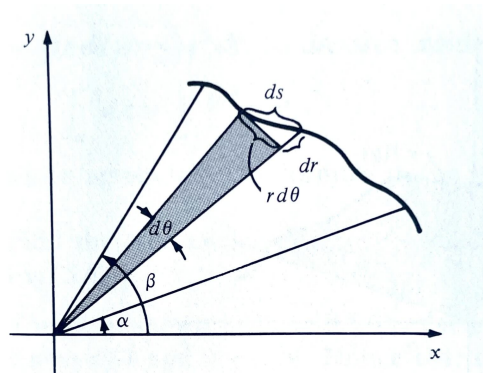
▷ Run script.

EIGENMATH output: $J(r, \theta)$ E_{r2} and $E_{\theta30}$

$$\begin{array}{c} \cos(\theta) \\ -r \sin(\theta) \\ \sin(\theta) \\ r \cos(\theta) \\ \left[\begin{array}{cc} \cos(\theta) & -r \sin(\theta) \\ \sin(\theta) & r \cos(\theta) \end{array} \right] \end{array} \quad \begin{array}{c} E_{r2} = \left[\begin{array}{c} \frac{1}{2} 3^{1/2} \\ \frac{1}{2} \end{array} \right] \\ E_{\theta30} = \left[\begin{array}{c} -1 \\ 3^{1/2} \end{array} \right] \end{array}$$

ad c. ¹⁶

"Consider now ordinary 2D polar coordinates. Let us set up an orthonormal frame at (r, ϕ) , so that its x -axis points in the r -direction and the y -axis is perpendicular to it. Now consider the polar coordinate components of an infinitesimal displacement vector $d\mathbf{v} = (dr, d\phi)$ evaluated at (r, ϕ) . In the orthonormal frame itself, the coordinates of this same infinitesimal displacement vector are $d\mathbf{v} = (dr, r \cdot d\phi)$, cf. fig.14. What are the components of this vector in a global Cartesian coordinate system (x, y) ? They would be $(dr \cdot \cos \phi - d\phi \cdot \sin \phi, dr \cdot \sin \phi + d\phi \cdot \cos \phi)$, a plain rotation!"



PYTHAGORAS' theorem in rectangular triangle $\triangle(ds, dr, r \cdot d\theta)$

Figure 15: says $ds^2 = dr^2 + (r \cdot d\theta)^2$, ergo we get the line element
 $ds = \sqrt{dr^2 + (r \cdot d\theta)^2}$ as infinitesimal length, [14, p.500].

Let's do it in EIGENMATH with package `ctensor`.

```
fpprintprec: 5;          /* Show only 5 significant digits */
postfix("^t")$ "^t"(x) := transpose(x)$ /*1 */
load(ctensor)$
cframe_flag : true$
```

¹⁶following the message of V. TOTTH in 59309247

```

ct_coordsys(polar)$
cmetric();
dv : [dr, r*diff(phi)]^t;          /* 2 */
trigsimp(fr);                      /* 3 */
trigsimp(fr . dv);                 /* 4 */

```

▷ Click here to RUN the code.

Comment. In 1: we define local a transpose operator $..^t$ to make writing of column vectors easier. 2: defines the line element $ds^2 = 1 \cdot dr^2 + r^2 \cdot d\phi^2$ in polar coordinates in vector representation. We transpose this vector for reasons of mathematical correctness, although EIGENMATH would allow to write $dv : [dr, r * d\phi]$. 3: calls ‘fr’ which represents the *contravariant Cartesian coordinate components* of the set of basis vectors, aligned with polar coordinates in the Euclidean plane. We can see that one basis vector in a background Cartesian frame is $(\cos \phi, \sin \phi)$ and the other is $(-\frac{1}{r} \sin \phi, \frac{1}{r} \cos \phi)$. In 4: the transformation is done, cf. 1st.

YAMWI output:

```

[%i7] dv : [dr, r*dφ];
[%o7] [dr, dφ r]
[%i8] trigsimp (fr);
[%o8] [ cos φ   - ( sin φ / r )
       sin φ   cos φ / r ]
[%i9]
[%o9] [ dr cos φ - dφ sin φ
       dr sin φ + dφ cos φ ]

```

Exercise 8. (components of a contravariant resp. covariant vector)

Do the following three exercises from the book of E. KREYSZIG [7, p.129]

DIFFERENTIALGEOMETRIE

VON

DR. ERWIN KREYSZIG

PROFESSOR AN DER TECHNISCHEN HOCHSCHULE GRAZ

Aufgaben

30.1. $a^1 = 2, a^2 = 1$ seien die Komponenten eines **kontravarianten** Vektors im Punkt P mit den Koordinaten $u^1 = 1, u^2 = 1$. Man bestimme die Komponenten $\bar{a}^1, \bar{a}^2$ dieses Vektors bezüglich der Koordinaten $\bar{u}^1 = \sqrt{(u^1)^2 + (u^2)^2}, \bar{u}^2 = \arctan(u^2/u^1)$.

30.2. Man bestimme die Komponenten des **kovarianten** Vektorfeldes $a_1 = (u^1)^2 - (u^2)^2, a_2 = 2u^1u^2$ bezüglich der Koordinaten $\bar{u}^1, \bar{u}^2$ in Aufg. 30.1.

30.3. Man bestimme die Komponenten des **kontravarianten** Vektorfeldes $a^1 = (u^1)^2 - (u^2)^2, a^2 = 2u^1u^2$ bezüglich der Koordinaten $\bar{u}^1, \bar{u}^2$ in Aufg. 30.1.

Lösungen der Aufgaben mit ungerader Nummer

$$\mathbf{30.1.} \quad \bar{a}^m = a^i \frac{\partial \bar{u}^m}{\partial u^i} = 2 \frac{\partial \bar{u}^m}{\partial u^1} + \frac{\partial \bar{u}^m}{\partial u^2} \quad \text{usw.,} \quad \bar{a}^1 = \frac{3}{\sqrt{2}}, \quad \bar{a}^2 = -\frac{1}{2}$$

$$\mathbf{30.3.} \quad \bar{a}^1 = (\bar{u}^1)^2 \cos \bar{u}^2, \quad \bar{a}^2 = \bar{u}^1 \sin \bar{u}^2$$

5.3.3 Example: the gradient ∇f transforms *covariantly*

Imagine a square hotplate $U \subset \mathbb{R}^2$ where you can measure the corresponding temperature value $f(x, y)$ at each point (x, y) with a function $f : U \rightarrow \mathbb{R}^2$.

Then we define the gradient $\text{grad}(f) \equiv \nabla f$ of the function f by

$$\text{grad}(f) := \nabla f := \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right) = \frac{\partial f}{\partial x} \cdot E_x + \frac{\partial f}{\partial y} \cdot E_y \quad (5.1)$$

with the Cartesian basis $E_x := \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, $E_y := \begin{bmatrix} 0 \\ 1 \end{bmatrix}$.

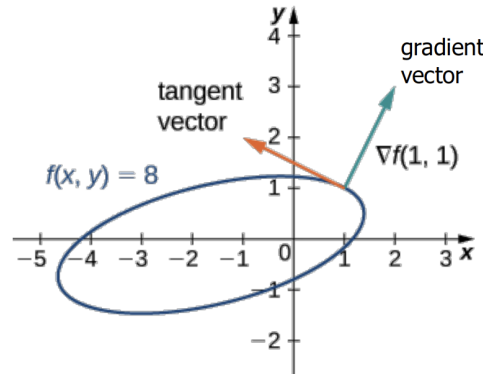


Figure 16:
○: the graph (level surface) of $f(x, y) = 8$ for $f(x, y) = x^2 - 2xy + 5y^2 + 3x - 2y + 3$ in $\mathbb{R}^2$.
∇ f : the $\text{grad}_{(1,1)} f = \nabla f(1, 1) = (3, 6)$ at point $(1, 1)$.

Exercise 9. Verify the calculation in fig. 15¹⁷ and determine $\nabla_{(x,y)} f$, i.e. in short ∇f .

Solution: ▷ Click here to SEE the solution.

We want to show that *the gradient vector ∇f , viewed as a tensor, transforms covariantly* from one coordinate system to another. To see, what this means, we start abstract this time and transform the gradient *from unprimed coordinates x^1, x^2 to primed $\bar{x}^1, \bar{x}^2$ coordinates*. If we introduce the new coordinates using the position vector $R(x^1, x^2) := (\bar{x}^1, \bar{x}^2)$, we have $\bar{x}^1 \equiv \bar{x}^1(x^1, x^2)$ and $\bar{x}^2 \equiv \bar{x}^2(x^1, x^2)$, and therefore make the dependence of $\bar{x}^1$ resp. $\bar{x}^2$ from x^1 and x^2 explicit. Now we can use the chain rule to calculate the gradient w.r.t the new coordinates. We take the same steps as in §5.3.2. Remember: the notation x^2 means the second coordinate of $\mathbf{x} = (x^1, x^2)$, *not* the square of x !

1. because $\bar{x}^1$ depends on x^1 and x^2 etc.

$$\frac{\partial f}{\partial \bar{x}^1} = \frac{\partial f}{\partial x^1} \cdot \frac{\partial x^1}{\partial \bar{x}^1} + \frac{\partial f}{\partial x^2} \cdot \frac{\partial x^2}{\partial \bar{x}^1}$$

$$\frac{\partial f}{\partial \bar{x}^2} = \frac{\partial f}{\partial x^1} \cdot \frac{\partial x^1}{\partial \bar{x}^2} + \frac{\partial f}{\partial x^2} \cdot \frac{\partial x^2}{\partial \bar{x}^2}$$

¹⁷Figure from LibreTexts, Ex.14.6.4

2. write as matrix equation using *row notation* for covariant vectors

$$\left[\frac{\partial f}{\partial \bar{x}^1}, \frac{\partial f}{\partial \bar{x}^2} \right] = \left[\frac{\partial f}{\partial x^1}, \frac{\partial f}{\partial x^2} \right] \bullet \begin{bmatrix} \frac{\partial x^1}{\partial \bar{x}^1} & \frac{\partial x^1}{\partial \bar{x}^2} \\ \frac{\partial x^2}{\partial \bar{x}^1} & \frac{\partial x^2}{\partial \bar{x}^2} \end{bmatrix}$$

3. or write both equations of 1. *as a sum*

$$\frac{\partial f}{\partial \bar{x}^1} = \sum_{k=1}^2 \frac{\partial f}{\partial x^k} \cdot \frac{\partial x^k}{\partial \bar{x}^1}, \quad \frac{\partial f}{\partial \bar{x}^2} = \sum_{k=1}^2 \frac{\partial f}{\partial x^k} \cdot \frac{\partial x^k}{\partial \bar{x}^2}$$

4. write both equations in 3. *as one term* with free index $j = 1..2$

$$\frac{\partial f}{\partial \bar{x}^j} = \sum_{k=1}^2 \frac{\partial f}{\partial x^k} \cdot \frac{\partial x^k}{\partial \bar{x}^j}$$

5. write even more compact using EINSTEIN summation convention

$$\frac{\partial f}{\partial \bar{x}^j} = \frac{\partial f}{\partial x^k} \cdot \frac{\partial x^k}{\partial \bar{x}^j}$$

6. write the transformation formula 5. as abstract pattern using $\bar{T}_j := \frac{\partial f}{\partial \bar{x}^j}$ and $T_j := \frac{\partial f}{\partial x^j}$

$$\bar{T}_j = \frac{\partial x^k}{\partial \bar{x}^j} \cdot T_j$$

- Note: in 6. are the coordinate transformation equations for the components of a covariant vector, where T_j are the components of the tensor (vector) in the original coordinate system x^j and $\bar{T}_j$ are the components of the same tensor (vector) expressed in the new coordinate system $\bar{x}^j$.

- Note also, in this case the transformation matrix $\left(\frac{\partial x^k}{\partial \bar{x}^j} \right)$ is the *inverse of the transformation matrix of the line element in $\left(\frac{\partial \bar{x}^j}{\partial x^k} \right)$* in §5.3.2-6.! Hence in this case the weighting factors are the components of the contravariant basis vectors, which means that the components of the gradient vector transform as covariant components.

- Therefore many authors define the covariant components of a vector as components that transform according to equation 6.

Task: Let $f(x, y) = x^2 - 2xy + 5y^2 + 3x - 2y + 3$ in $\mathbb{R}^2$ be the function to be used. Redo the discussion using EIGENMATH for the transformations $(x, y) \mapsto (r, \theta)$ resp. $(x, y) \leftarrow (r, \theta)$ and calculate the gradient of f in both coordinate systems.

Solution:

1st To calculate gradients and Jacobians of f in both coords, we could do, e.g., this:

```
f(x,y) = x^2-2*x*y+5*y^2+3*x-2*y+3

"----- gradient of f in both coords"

"grad of f in Cartesian coords"
E = (x, y)                -- Cartesian coordinates

gradf(x,y) = d(f, E)
gradf
## or: gradf1(x,y) = (d(f(x,y),E[1]) , d(f(x,y),E[2]))

"grad of f in polar coords"
U = (r, theta)            -- polar coords:  r, theta

f1(r,theta) = eval(f(x,y), x, r*cos(theta), y, r*sin(theta))    #(1)
f1
-- function f in polar coords

gradf1(r,theta) = d( f1(r,theta), U)
gradf1
gradf1 = simplify(gradf1)
gradf1

"----- Jacobian  of coord transformations in both coords"

toPolar(x,y) = (sqrt(x^2+y^2), arctan(y,x))                      #(2)
J1 = d(toPolar, E)
J1
J1 = simplify(J1)
J1

toCartesic(r,theta) = (r*cos(theta), r*sin(theta))              #(3)
J2 = d(toCartesic, U)
J2

"proving that J1 and J2 are inverses"
J1J2 = dot(J1, J2)
J1J2

J1J2 = eval(dot(J1, J2),
              sin(theta),  y/r,      -- substitutions
              cos(theta),  x/r,
              sqrt(x^2+y^2), r)
```

```

J1J2

"to verify: J1J2[1,1] = 1 and J1J2[2,2] = 1"

a11 = x^2/r^2 + y^2/r^2      -- we know: x^2+y^2 = r^2.
a11
a11 = (x^2+y^2)/r^2
a11
a11 = r^2/r^2
a11

a22 = y^2/(x^2*(y^2/x^2+1)) + 1/(y^2/x^2+1)      -- element (2,2) of J1J2
a22
a22 = y^2/(x^2*((y^2+x^2)/x^2)) + 1/(y^2/x^2+1)
a22
a22 = y^2/(x^2*((y^2+x^2)/x^2)) + 1/((y^2+x^2)/x^2)
a22
a22 = (x^2+y^2)/(x^2+y^2)
a22

"ergo:"
J1J2 = unit(2)                                     #(5)
J1J2

```

▷ Run script.

Comment. In line #(1) we substitute in $f(x, y)$ for $x \mapsto r * \cos(\theta)$ and for $y \mapsto r * \sin(\theta)$ to get the term for f in polar coords. The result is coded in $f1$. This function is then derivated w.r.t. the polar corrd $U = (r, \theta)$.

In In #(3) we construct the transformation matrix

$$J1 : \overset{polar}{U} \mapsto \overset{Cartesian}{E}$$

using the `jacobian` function $d(...)$ of EIGENMATH. In #(3) we reverse the process:

$$J2 : \overset{cartesian}{E} \mapsto \overset{polar}{U}$$

We then help EIGENMATH simplifying the matrix product $J1.J2 = \text{dot}(J1, j2)$ to reach the identity matrix.

EIGENMATH output:

grad of f in Cartesian coords

$$g_{radf} = \begin{bmatrix} 2x - 2y + 3 \\ -2x + 10y - 2 \end{bmatrix}$$

grad of f in polar coords

$$f_1 = r^2 \cos(\theta)^2 - 2r^2 \cos(\theta) \sin(\theta) + 5r^2 \sin(\theta)^2 + 3r \cos(\theta) - 2r \sin(\theta) + 1$$

$$g_{radf1} = \begin{bmatrix} 2r \cos(\theta)^2 - 4r \cos(\theta) \sin(\theta) + 10r \sin(\theta)^2 + 3 \cos(\theta) - 2 \sin(\theta) \\ -2r^2 \cos(\theta)^2 + 8r^2 \cos(\theta) \sin(\theta) + 2r^2 \sin(\theta)^2 - 2r \cos(\theta) - 3r \sin(\theta) \end{bmatrix}$$

$$g_{radf1} = \begin{bmatrix} 6r - 4r \cos(2\theta) - 2r \sin(2\theta) + 3 \cos(\theta) - 2 \sin(\theta) \\ -2r^2 \cos(2\theta) + 4r^2 \sin(2\theta) - 2r \cos(\theta) - 3r \sin(\theta) \end{bmatrix}$$

proving that J1 and J2 are inverses

$$J_{1,2} = \begin{bmatrix} \frac{x \cos(\theta)}{[x^2 + y^2]^{1/2}} + \frac{y \sin(\theta)}{[x^2 + y^2]^{1/2}} & -\frac{r x \sin(\theta)}{[x^2 + y^2]^{1/2}} + \frac{r y \cos(\theta)}{[x^2 + y^2]^{1/2}} \\ \frac{x \sin(\theta)}{x^2 + y^2} - \frac{y \cos(\theta)}{x^2 + y^2} & \frac{r x \cos(\theta)}{x^2 + y^2} + \frac{r y \sin(\theta)}{x^2 + y^2} \end{bmatrix}$$

$$J_{1,2} = \begin{bmatrix} \frac{x^2}{r[x^2 + y^2]^{1/2}} + \frac{y^2}{r[x^2 + y^2]^{1/2}} & 0 \\ 0 & \frac{x^2}{x^2 + y^2} + \frac{y^2}{x^2 + y^2} \end{bmatrix}$$

Verify: J1J2[1,1] = 1 and J1J2[2,2] = 1

$$a_{11} = \frac{x^2}{r^2} + \frac{y^2}{r^2}$$

$$a_{11} = \frac{x^2}{r^2} + \frac{y^2}{r^2}$$

$$a_{11} = 1$$

$$a_{22} = \frac{y^2}{x^2 \left[\frac{y^2}{x^2} + 1 \right]} + \frac{1}{\frac{y^2}{x^2} + 1}$$

$$a_{22} = \frac{y^2}{x^2 + y^2} + \frac{1}{\frac{y^2}{x^2} + 1}$$

Exercise 10. (the velocity vector of a curve in polar coordinates)

Let $y = x^2$ alias $C(t) := (t, t^2)$ be the curve ('parabola') in the Cartesian plane $\mathbb{R}^2$.

- a. Show, that $C(t)$ has the equation $c(\Theta) = (\frac{\sin(\Theta)}{\cos(\Theta)^2}, \Theta)$ in polar coordinates.
Remember, both curves $C(t)$ and $c(\Theta)$ parameterize the same set ('manifold').
- b. Verify: the Cartesian point $(x = 1, y = 1)$ is the same point as $(r = \sqrt{2}, \theta = \frac{\pi}{4})$ in polar coordinates.
- c. Calculate and verify in the figure: the velocity vector v at $(1, 1)|_{\text{cartesian}}$ is $(1, 2)|_{\text{cartesian}}$,
i. e. in the Cartesian basis $E_x := \begin{bmatrix} 1 \\ 0 \end{bmatrix}, E_y := \begin{bmatrix} 0 \\ 1 \end{bmatrix}$ we have $P = 1 \cdot E_x + 1 \cdot E_y$ and $v = 1 \cdot E_x + 2 \cdot E_y$.

What are the polar coordinates of v ? Check your result at the picture.

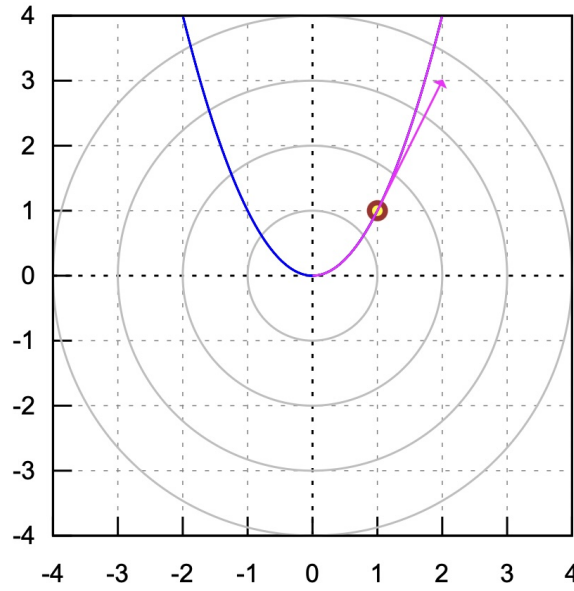


Figure 17:
 $\odot$: the polar coordinates aligned in the Euclidean plane.
 $\bullet$: the Cartesian point $(1, 1) \xrightarrow{\text{polar}} (\sqrt{2}, \frac{\pi}{4})$ (brown ring)
 magenta : the Cartesian curve $y = x^2$ on $[0, 2]$.
 blue : the polar curve $c(\Theta) = (\frac{\sin(\Theta)}{\cos(\Theta)^2}, \Theta)$ on $[0, 2\pi]$.

6 The KRONECKER delta tensor

In Linear Algebra, the KRONECKER delta (named after Leopold KRONECKER) is a *function* of two variables ('indices'), usually non-negative integers. This function has value 1 if the inputs are equal, and 0 otherwise. It is *not* a tensor.¹⁸

6.1 Definition: δ_i^j

The KRONECKER delta *tensor* of Tensor Calculus is defined by

$$\delta_i^j := \begin{cases} 0 & \text{if } i \neq j, \\ 1 & \text{if } i = j. \end{cases} \quad (6.1)$$

Remark.

1. The KRONECKER delta *function* of Linear Algebra is defined by

$$\delta_{ij} := \begin{cases} 0 & \text{if } i \neq j, \\ 1 & \text{if } i = j. \end{cases} \quad (6.2)$$

2. We have two versions of the KRONECKER δ symbol, a function and a tensor:

LEXICON	<i>Math</i>	EIGENMATH	MAXIMA
KRONECKER δ function	$\delta_{i,j}$	δij	<code>kron_delta(i,j)</code>
KRONECKER δ tensor	δ_a^b	δab	<code>kdelta([a],[b])</code>

3. δ_i^j is the identity transformation that transforms a vector into itself. That is a coordinate-free statement: the components of the KRONECKER delta remain the same in every coordinate system. The KRONECKER delta δ_i^j always has the same number of upstairs and downstairs indices, and so respects the transformation rules.
4. The `kdelta(L1, L2)` function of the `itensor` package of MAXIMA is the *generalized Kronecker delta tensor* with *L1* the list of covariant indices and *L2* the list of contravariant indices. In particular δ_i^j is a *tensor*, but δ_{ij} is *not*. For a proof cf. [6, 82], "which shows that the values of the KRONECKER delta symbol in different coordinate systems relate by the same tensor rule."

Considered as a type (1,1) tensor, the Kronecker tensor is written as δ_i^j , with one covariant index *i* and contravariant index *j*. We say: δ_i^j is a rank-2 mixed tensor.

¹⁸In contrast, the KRONECKER delta *tensor* of the MAXIMA package `itensor` is a true tensor.

Example 7. (some checks of δ_{ij})

We implement the KRONECKER δ 'tensor' as a function in EIGENMATH.

```
" Define KRONECKER delta function "
delta(i,j) = test( i=j, 1, 0)      -- (1)

delta11 = delta(1,1)                -- (2)
delta11                                -- (3)

delta10 = delta(1,0)
delta10

" Define KRONECKER delta as table "
deltaij = zero(2,2)                -- (4)
    deltaij[1,1] = 1
    deltaij[2,2] = 1

deltaij                                -- the whole 'tensor'
deltaij[1,2]                        -- (5), watch the [.] brackets
```

Comment. Definition (1) is a direct formulation of the mathematical definition. In (2) we test the function `delta` on the input pair (1,1) and catch the result in the identifier `delta11`. Its return value in (3) is automatically written in pretty-print *LaTeX* shape δ_{ij} in the EIGENMATH-output region - which has nothing to do with the tensorial definition in (4), which is formatted by EIGENMATH also as δ_{ij} and therefore looks like (3).

▷ Run script.

EIGENMATH output:

$$\begin{array}{l} \delta_{11} = 1 \\ \delta_{10} = 0 \\ \delta_{ij} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} \\ 0 \end{array}$$

Exercise 11. Argue, whether it is possible to define the KRONECKER delta tensor using the self-written function 'signum' `sign`. Do some tests in EIGENMATH's script window.

```
sign(x) = test(x>0, 1, -1)
deltaij(i,j) = 1 - abs(sign(i-j))
```

Example 8. (some uses of δ_i^j)

For concreteness let's assume 3D space; any index runs over the three dimensions. We use Cartesian coordinates x, y, z or more general x^1, x^2, x^3 . The indices varies through 1, 2, 3.

- a. What is $\frac{\partial x^i}{\partial x^j}$?
- b. What gives $A_i \delta_j^i = A_j$?
- c. What means $A_j^i B_k^j = \delta_k^i$?
- d. $\delta_i^j A_i^j = ?$
- e. $\delta_i^i = ?$

Answer:

a: δ_i^j , because different independents x^i have no dependence on each other.

b: $A_i \delta_j^i \stackrel{Einstein}{=} \sum_{i=1}^3 A_i \delta_j^i$, where $j = 1, 2, 3$ is free.

So we have 3 equations incorporated in this single tensorial term:

$$A_1 \delta_1^1 + A_2 \delta_1^2 + A_3 \delta_1^3 = A_1$$

$$A_1 \delta_2^1 + A_2 \delta_2^2 + A_3 \delta_2^3 = A_2$$

$$A_1 \delta_3^1 + A_2 \delta_3^2 + A_3 \delta_3^3 = A_3$$

c: A and B are inverse matrices, i.e. $A.B = E$.

d: A_i^i , which is a 'contraction' i.e. a renaming to make a common lower and upper index which we then sum over.

e: 3, because we assumed 3 as dimension of the space, therefore we have:

$$\delta_i^i \stackrel{Einstein}{=} \sum_{i=1}^3 \delta_i^i = \delta_1^1 + \delta_2^2 + \delta_3^3 = 1 + 1 + 1 = 3$$

Example 9. (δ_i^j is a true tensor)

To *prove*: δ_i^j with one upper and one lower index is a rank-2 mixed tensor.

Proof: From COLLIER [4, p.143].

Let $(x^1, \dots, x^n)$ be the original 'old' coordinate system and $(y^1, \dots, y^n)$ the primed 'new' one. Using index notation we have to verify the tensor property for the transform of the partial derivatives, i.e. δ_i^j should transform according to

$$\bar{\delta}_k^\ell = \frac{\partial y^\ell}{\partial x^j} \cdot \frac{\partial x^i}{\partial y^k} \cdot \delta_i^j$$

We have

$$\begin{aligned} \bar{\delta}_k^\ell &= \frac{\partial y^\ell}{\partial x^j} \cdot \frac{\partial x^i}{\partial y^k} \cdot \delta_i^j \\ &\stackrel{1}{=} \frac{\partial y^\ell}{\partial x^j} \cdot \frac{\partial x^j}{\partial y^k} \\ &\stackrel{2}{=} \frac{\partial y^\ell}{\partial y^k} \\ &= \delta_k^\ell \end{aligned}$$

1: the right-hand side KRONECKER delta relabels the i index as j .

2: the primed new coordinates are independent of each other.

Comment. In this proof we realize that the KRONECKER delta tensor δ_k^ℓ is used as a relabeling of indices and therefore simplifying tensorial expressions.

6.2 Example: The case of the Scalar product of Vectors

The inner product (aka scalar product) of vectors in Linear Algebra is often written using the symbol δ_{ij} (as function, not as tensor!) as

$$\mathbf{a} \bullet \mathbf{b} \stackrel{!}{=} \sum_{i=1}^n \sum_{j=1}^n a_i \delta_{ij} b_j = \sum_{i=1}^n a_i b_i \quad (6.3)$$

where i and j take the values $1, 2, \dots, n$ and the vectors $\mathbf{a}$ and $\mathbf{b}$ are defined as arbitrary n -tuples $\mathbf{a} = (a_1, a_2, \dots, a_n)$ and $\mathbf{b} = (b_1, b_2, \dots, b_n)$ in $\mathbb{R}^n$.

We see how in the above equation the values of the KRONECKER delta function δ_i^j reduce the double summation over i and j to a single summation over i only ...

▷ But this is an *abuse of the EINSTEIN summation convention*: A summation in a tensor expression is only implicit done, when an index appears twice, once as a *subscript* ('lower index') and once as a *superscript* ('upper index'). This is not the case here!

Task: define a correct tensorial scalar product using EIGENMATH

Remember, that the scalar product in Linear Algebra is defined by the GRAMmatrix g , i.e. the metric tensor by the tensorial pattern

$$g_{ij} A^i B^j \quad (6.4)$$

(instead of $\delta_{ij} a_i b_j$), where A and B are column vectors! Herein the case $g_{ij} = \delta_{ij}$ is only correct, if the corresponding basis is orthonormal in the Euclidean space.

In the following case study we present three implementations of the inner product.

1st We use the function definition of δ_{ij} .

We program the defining term (6.3¹) and use, that the values of the KRONECKER delta reduce the double summation over i and j to a single summation over i only. This is a first indication of the 'contract'ing effect of δ_{ij} .

```
delta(i,j) = test( i=j, 1, 0)

ip(a,b) = sum(i,1,dim(a),      -- inner product via double sum
               sum(j,1,dim(b),  a[i]*delta(i,j)*b[j]))    -- (1)

a=(1,2,3)
b=(3,2,1)

ip(a,b)          -- (2)
dot(a,b)         -- (3)
```

EIGENMATH output: 10

i.e. both calculations using function `ip(.)` in (2) or the build-in dot-product `dot(.)` in (3) give the same value.

▷ Run script.

2nd definition of the inner product using matrix multiplication.

We first choose a specific coordinate system, i.e. 'cartesian3d' with the Euclidian metric g . Then we may do ordinary matrix-vector multiplication a la " $\cdot \square \cdot \rightsquigarrow \cdot \rightsquigarrow \bullet$ " to calculate the inner product, which resembles again equation (6.3¹):

```
"coordsys is cartesian3d"
g = ((1,0,0),(0,1,0),(0,0,1))      -- cartesian metric
A = (a1,a2,a3)
B = (b1,b2,b3)
sp(g,A,B) = dot(A,g,B)              -- define sp
sp(g,A,B)                            -- and use it
```

▷ Run script.

EIGENMATH output: $a_1b_1 + a_2b_2 + a_3b_3$

Comment. The metric g is represented using a matrix, which would imply a mixed-rank tensor, which the metric most specifically isn't. Rows and columns correspond to covariant vs. contravariant quantities, but admittedly, we can't represent this for the inner product via $A \cdot lg \cdot \text{transpose}(B)$ in EIGENMATH, which would realize $g_{ij}A^iB^j$ of (6.4) correctly.

3rd definition of the inner product using the metric.

The most approximately generalizable version uses the cartesian metric g , which could be replaced by a metric of another manifold.

```
g = ((1,0,0),(0,1,0),(0,0,1))
n = dim(g,1)                      -- length(g)
sp(g,A,B) = sum(i,1,n, sum(j,1,n, g[i,j]*A[i]*B[j]))

A = (a1,a2,a3)                    -- is column vector
B = (b1,b2,b3)
sp(g,A,B)
```

▷ Run script.

EIGENMATH output: $a_1b_1 + a_2b_2 + a_3b_3$

Comment. We implement the term $g_{ij}A^iB^j$ as `g[i,j]*A[i]*B[j]`, defining the metric g "by hand", represented as a 3-by-3-matrix and use the size¹⁹ of the metric g tensor explicitly

¹⁹the `length(g)` of a matrix is the number of the rows

in the summation. So the definition is independent of the dimension of the space. The double sum resembles the Linear Algebra definition for the scalar product in Cartesian vector space $\mathbb{R}^3$ from the beginning. Function `sp()` expects as arguments the metric g and two column vectors.

To recap:

INNER PRODUCT <code>sp()</code>	<i>Tensor Math</i>	realization in EIGENMATH
1st :	$g_{ij}A^iB^j$	<code>delta_ij*A[i]*B[j]</code>
2nd :		<code>A . g . transpose(B)</code>
3rd :		<code>g[i,j] * A[i] * B[j]</code>

Only the **3rd** version would be the fully correct realization of the inner product in the tensorial setting in a notation e.g. `g([i,j],[i]) * A([i],[i]) * B([i],[j])` allowing to discriminate between covariant and contravariant indices. We could use a notion like `gdd[i,j] * Au[i] * Bu[j]` to mimic the correct notation in **EIGENMATH**. *d* means down index, *u* means upper index, e.g. read `Au[i]` as 'A upper i' or 'A up i'.

♥ We end this section with

$$\delta_i^i \equiv \sum_{i=1}^3 \delta_i^i \quad \text{in } \mathbb{R}^3$$

$\downarrow$ *contract*

$\downarrow$ *i.e. sum*

3

the EINSTEIN summation convention applied. ♥

Remark. Remember the Einstein summation convention!

Not always the Kronecker delta δ_{ij} is written correctly with one index as superscript and the other as subscript, i.e. δ_j^i . If one uses δ_i^i , this is interpreted in tensor language in the sense of EINSTEIN as a hidden summation behind the scene in the ambient space, e.g. in $\mathbb{R}^3$ to suppress the summation sign $\sum$ in tensor expressions.

Ergo, summation symbols can be 'eliminated or forgotten' by using Einstein's convention, giving very compact tensorial notations - but with the drawback to decode the terms in your mind. The application of the Einstein-summation-convention is often called a *contraction*.

Here is the EINSTEIN summation convention shown in EIGENMATH:

```
" EINSTEIN summation convention "
delta(i,j) = test( i=j, 1, 0)

deltaii = sum(i,1,3, delta(i,i))  -- Einstein's implicit sum in R^3
deltaii
```

▷ Run script.

EIGENMATH output: 3

Remark. The dot product $\mathbf{a} \cdot \mathbf{b} = \sum_{i=1}^n a_i b_i$ of two vectors in (6.3) may be thought of as a contraction a la EINSTEIN's convention. The vectors are first multiplied element-wise and then 'contracted', i.e. 'summed up'.

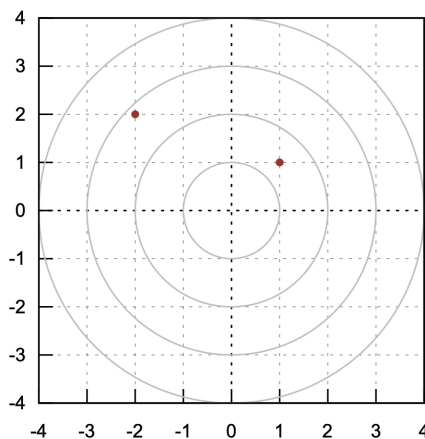
Therefore, we may write the short tensor notation

$$\mathbf{a} \bullet \mathbf{b} = a_i b_i$$

meaning expanded and decoded

$$\mathbf{a} \bullet \mathbf{b} = \sum_{i=1}^n a_i b_i$$

Exercise 12. (inner product in polar coordinate system)



- Read off the polar coordinates of the two points A and B .
- Calculate the inner product $A \bullet B$ using the polar metric g_{polar} .
- Determine the cartesian coordinates of the two points and calculate the cartesian inner product $A \bullet B$ using the Cartesian metric $g_{cartesian}$.

7 The Permutation symbol ε

The *permutation symbol* is a multi-index function sometimes also called the *alternating symbol* or the *signature*. There are several common notations for the symbol: some authors uses the Greek epsilon character ϵ_{ijk} , others uses the curly variant ε_{ijk} or the Latin lower case e_{ijk} . We will use the the latter, because we will follow GRINFELD and reserve the Greek epsilon notation for the LEVI-CIVITA symbol, cf. [6, p.134]. The permutation symbol is heavily used to define advanced tensorial objects such as the determinant, the curl or the cross product of vectors, cf. [38].

7.1 Definition

In two dimensions, the *permutation symbol* e_{ij} is defined by:

$$e_{ij} := \begin{cases} +1 & \text{if } (i, j) = (1, 2) \\ -1 & \text{if } (i, j) = (2, 1) \\ 0 & \text{if } i = j \end{cases} \quad (7.1)$$

- In three dimensions, the permutation tensor is defined by:

$$e_{ijk} = \begin{cases} +1 & \text{if } (i, j, k) \text{ is } (1, 2, 3), (2, 3, 1), \text{ or } (3, 1, 2), \\ -1 & \text{if } (i, j, k) \text{ is } (3, 2, 1), (1, 3, 2), \text{ or } (2, 1, 3), \\ 0 & \text{if } i = j, \text{ or } j = k, \text{ or } k = i \end{cases}$$

- In four dimensions (and also in 2D or 3D), the permutation tensor is defined by:

$$e_{ijkl} = \begin{cases} +1 & \text{if } (i, j, k, l) \text{ is an even permutation of } (1, 2, 3, 4) \\ -1 & \text{if } (i, j, k, l) \text{ is an odd permutation of } (1, 2, 3, 4) \\ 0 & \text{otherwise} \end{cases}$$

Remark. Be warned: some authors call the permutation *tensor* the *Levi – Civita symbol*.

LEXICON	<i>Math</i>	EIGENMATH
permutation symbol:	e_{ijk}	eijk or Eijk

7.2 Examples

To get a feeling of the construction of the permutation symbol e_{ijk} ... we discuss some special cases.

7.2.1 ... the two dimensional permutation symbol e_{ij}

```

-- permutation symbol als function Eij
Eij(i,j) = j-i
Eij(1,2)
Eij(1,1)

-- permutation symbol eij as table

eij = zero(2,2)                -- container
for(i,1,2, for(j,1,2, eij[i,j]=j-i ))  -- fill container

eij                            -- the permutation symbol
eij[1,2]                       -- result on index pair (1,2)

```

EIGENMATH output:

$$e_{ij} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

1

▷ Run script.

Exercise 13. Define function $Eij(i,j)$ using the math definition for e_{ij} from above.

7.2.2 ... the three dimensional permutation symbol e_{ijk}

```

# EIGENMATH
-- permutation symbol as function, i.e.
-- eijk is 1 if (i,j,k) is an even permutation of (1,2,3),
--      -1 if (i,j,k) is an odd permutation of (1,2,3),
--      0 if any index is repeated.

Eijk(i,j,k) = test(
  or( (i,j,k)==(1,2,3), (i,j,k)==(2,3,1), (i,j,k)==(3,1,2)), +1,
  or( (i,j,k)==(3,2,1), (i,j,k)==(1,3,2), (i,j,k)==(2,1,3)), -1,
  or( i==j, j==k, k==i), 0 )

Eijk(1,2,3)
Eijk(2,1,3)
Eijk(2,2,1)

```

▷ Run script.

EIGENMATH output:

$$\begin{bmatrix} 1 \\ -1 \\ 0 \end{bmatrix}$$

◦ Now the version with table access:

```
# EIGENMATH
eijk = zero(3,3,3)
n = 3
for(i,1,n,
  for(j,1,n,
    for(k,1,n, eijk[i,j,k]=(j-i)*(k-i)*(k-j)/2 ) ))

eijk

--test:
eijk[1,2,1]          -- access to entry with index list (1,2,1)
a = eijk[1,2,3] + eijk[2,1,3]
a
b = eijk[1,2,3] * eijk[2,3,1]
b
```

EIGENMATH output:

$$e_{ijk} = \begin{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & -1 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & -1 \\ 0 & 0 & 0 \\ 1 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{bmatrix}$$

0
a = 0
b = 1

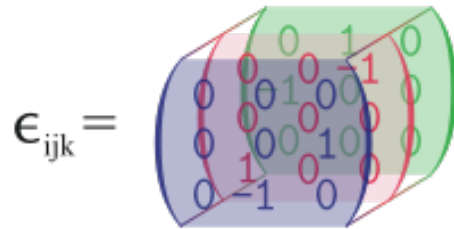
▷ Run script.

Remark. The EIGENMATH output of tensor `eijk` is verified by the two visualizations of this tensor:

Exercise 14. Define function `Eijk(i,j,k)` using the defining term $(j-i)*(k-i)*(k-j)/2$. Argue for the correctness of the formula and test it on some examples e.g. `eijk(3,2,1)=-1`.

Remark. The above formulas use the alternative explicit expression of the permutation symbol as a product of numbers using the *signum* function (denoted `sign`) as helper function:

$$\varepsilon_{a_1 a_2 a_3 \dots a_n} = \prod_{1 \leq i < j \leq n} \text{sign}(a_j - a_i)$$

Figure 18: 3D image of $\mathbf{eijk}$ with 3 layered matrix components, [38].

Exercise 15. G. Weigt, the author of EIGENMATH, defined $\mathbf{eijk}$ explicit in this way:

```
# EIGENMATH (G. Weigt)
eijk = zero(3,3,3)
  eijk[1,2,3] = 1
  eijk[2,3,1] = 1
  eijk[3,1,2] = 1
  eijk[3,2,1] = -1
  eijk[1,3,2] = -1
  eijk[2,1,3] = -1

eijk  -- the symbol
```

Explain.

7.2.3 ... the 4D permutation symbol $\mathbf{eijkl}$

In what follows we will not use more than the 4 dimensional version of the permutation tensor.

```
# EIGENMATH
-- permutation tensor eijkl as TABLE
eijkl=zero(4,4,4,4)
n=4
for(i,1,n, for(j,1,n, for(k,1,n, for(l,1,n,
    eijkl[i,j,k,l] = (i-j)*(i-k)*(i-l)*(j-k)*(j-l)*(k-l)/12 ) )))

eijkl
eijkl[3,4,2,1]  -- -1
eijkl[1,2,3,4]  -- 1
eijkl[1,2,1,4]  -- 0   ok

-- permutation tensor eijkl as FUNCTION
Eijkl(i,j,k,l) = (i-j)*(i-k)*(i-l)*(j-k)*(j-l)*(k-l)/12
```

```

Eijkl(3,4,2,1)  -- -1
Eijkl(1,2,3,4)  --  1
Eijkl(1,2,1,4)  --  0   ok

```

EIGENMATH output:

$$e_{ijkl} = \begin{bmatrix} \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & -1 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & -1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & -1 \\ 0 & 0 & 1 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & -1 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 & -1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 1 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \end{bmatrix}$$

▷ Run script.

Exercise 16. Define the permutation symbol `eijkl` in EIGENMATH using the G. WEIGT-way, i.e. use a table of just enough prescribed values at quadruple index positions - and to make 'tensors' printed visually to inspect their action.

[Hint: look carefully into the last EIGENMATH output. How many entrys do you need?]

7.2.4 a reconstruction of the permutation symbol

We now show a general procedure to calculate the permutation symbol using the determinant of permuted columns of the unit matrix, cf. ▷ $\varepsilon_{i_1 i_2 \dots}$. Here is the idea in $\mathbb{R}^3$:

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \xrightarrow{231} \begin{bmatrix} 0 & 0 & 1 \\ 1 & 0 & 0 \\ 0 & 1 & 0 \end{bmatrix} \xrightarrow{\det} 1 \longrightarrow \varepsilon_{231} = \text{permutation}([2, 3, 1])$$

Starting with the unit matrix U , pick their columns 2, 3 and 1 in this sequence, build with these columns a new matrix, get their determinant - and that's the value of ε_{231} .

Here is this process in EIGENMATH:

```

U = unit(3)
eijk(i,j,k) = det((U[i],U[j],U[k]))
eijk(2,3,1)

```

EIGENMATH output: 1

Exercise 17. Redo exercise 7.2.3.b using the above procedure.

7.3 Applications

As a proof of concept we show how the mathematical concepts of the determinant of a square matrix, the cross product of two vectors and the curl of a vector field can be defined and calculated using tensor technics with EIGENMATH.

7.3.1 The Determinant

If $A = (a_{ij}) \in \mathbb{R}^{n \times n}$ is a n-by-n square matrix, the determinant is defined and calculated through the so-called **LEIBNIZ formula** using sigma notation²⁰,

$$\det(A) = \sum_{\sigma \in S_n} \text{sgn}(\sigma) a_{1,\sigma_1} \cdots a_{n,\sigma_n}$$

Using 'Π' notation, this is in short $\det(A) = \sum_{\sigma \in S_n} (\text{sgn}(\sigma) \prod_{i=1}^n a_{i,\sigma(i)})$.
Written with the permutation symbol, the Leibniz formula is

$$\det(A) = \sum_{i_1, i_2, \dots, i_n} \varepsilon_{i_1 \dots i_n} a_{1,i_1} \cdots a_{n,i_n} \quad (7.2)$$

where the sum is taken over all n-tuples of integers from $\{1, \dots, n\}$.

This is a horrible formula, which no one really wants to compute this way by paper'n pencil. So let's do EIGENMATH the work for us.

For the special case of a 3×3 matrix A we have for $\det : \mathbb{R}^{3 \times 3} \rightarrow \mathbb{R}^1$

$$\text{det}(\mathbf{A}) = \sum_{i=1}^3 \sum_{j=1}^3 \sum_{k=1}^3 \varepsilon_{ijk} a_{1i} a_{2j} a_{3k} \quad (7.3)$$

Here the permutation symbol ε_{ijk} acts as a filter, which sort out the proper ones of the whole 27 summands. Let's realize this in EIGENMATH.

- definition of 3-by-3 determinant using LEIBNIZ formula and EIGENMATH.

We use our self-made permutation symbol Eijk and do the three sum's in (7.3) explicit.

```
Eijk(i,j,k) = (j-i)*(k-i)*(k-j)/2 -- permutation symbol

Det(A) = sum(i,1,3,
             sum(j,1,3,
             sum(k,1,3, Eijk(i,j,k)*A[1,i]*A[2,j]*A[3,k] ) ))

A=((1,2,3),(4,5,6),(7,8,8)) -- a 3-by-3 matrix
Det(A) -- determinant via LEIBNIZ formula
det(A) -- build-in det-function
```

²⁰see e.g. [10]

EIGENMATH output: 3

▷ Run script.

Exercise 18. Define an EIGENMATH function `Det2` analog to the example above and calculate the determinant of matrix $A = ((1, 2), (7, 8))$.

7.3.2 Cross product

Given two linearly independent vectors a and b in $\mathbb{R}^3$, the cross product $a \times b$ is a vector, that is perpendicular to both a and b , i.e. therefore perpendicular to the plane containing them. Using column vector notation, we have the explicit defining formula for $c = a \times b$:

$$\begin{bmatrix} c_1 \\ c_2 \\ c_3 \end{bmatrix} = \begin{bmatrix} a_2 b_3 - a_3 b_2 \\ a_3 b_1 - a_1 b_3 \\ a_1 b_2 - a_2 b_1 \end{bmatrix}$$

In any basis, the cross-product $a \times b$ is given by the tensorial formula

$$(a \times b)_k = \varepsilon_{ijk} a^i b^j$$

EINSTEIN ... $\downarrow$... expanded

$$(a \times b)_k = \sum_{i=1}^3 \sum_{j=1}^3 \varepsilon_{ijk} a^i b^j \quad (\ddagger)$$

i.e. with the suppressed summation expanded, we get the correct formula for the translation of the cross product term into EIGENMATH code. Therein E_{ijk} is the permutation symbol, which respects the positions of the indices.

1st definition of 3-by-3 determinant using LEIBNIZ formula and EIGENMATH.

The idea is to add up scalar multiplied altered 3×1 unit-vectors $E[i]$ to construct the *cross* vector:

```
# EIGENMATH
Eijk(i,j,k) = (j-i)*(k-i)*(k-j)/2          -- permutation symbol

do( E1=(1,0,0), E2=(0,1,0), E3=(0,0,1))    -- basis in R^3
E=(E1,E2,E3)

Cross(a,b) = sum(i,1,3,
                sum(j,1,3, sum(k,1,3,
                    Eijk(i,j,k) * E[i]*a[j]*b[k] ))) -- the tensorial formula

Cross((1,2,3),(4,5,6))                    -- test tensorial Cross product formula
cross((1,2,3),(4,5,6))                     -- check using build-in cross
```

EIGENMATH output: [-3, 6, -3]

▷ Run script.

Exercise 19. Program the cross product using the explicit formula given above.

2nd definition of 3-by-3 determinant using LEIBNIZ formula and E_{ijk} .

We use the permutation symbol E_{ijk} and do both sum's for each of the three component of the *cross* formula in (‡) explicit.

```
# EIGENMATH
Eijk(i,j,k) = (j-i)*(k-i)*(k-j)/2          -- permutation symbol

cross1(M) = sum(i,1,3, sum(j,1,3, Eijk(i,j, 1)*M[1,i]*M[2,j]))
cross2(M) = sum(i,1,3, sum(j,1,3, Eijk(i,j, 2)*M[1,i]*M[2,j]))
cross3(M) = sum(i,1,3, sum(j,1,3, Eijk(i,j, 3)*M[1,i]*M[2,j]))

cross(M) = (cross1(M), cross2(M), cross3(M))

A = ((1,2,3),(4,5,6))
cross(A)
```

EIGENMATH output: $[3, -6, 3]^t$

▷ Run script.

7.3.3 Curl

$\text{curl } F$ ²¹ is a vector operator that is used in vector calculus to describe the circulation of a 3D differentiable vector field $F: \mathbb{R}^3 \rightarrow \mathbb{R}^3$, given through its three component functions $F(x, y, z) = (F_x(x, y, z), F_y(x, y, z), F_z(x, y, z))$.

We cite the definition:

$$(\text{curl } F)(x, y, z) = \left(\frac{\partial F_z}{\partial y} - \frac{\partial F_y}{\partial z}, \frac{\partial F_x}{\partial z} - \frac{\partial F_z}{\partial x}, \frac{\partial F_y}{\partial x} - \frac{\partial F_x}{\partial y} \right) \quad (7.4)$$

The equation for each component $(\text{curl } F)_k$ is obtained by exchanging each occurrence of a subscript y, z, x in cyclic permutation $(y, z) \rightarrow (z, x) \rightarrow (x, y)$ by focussing at the denominators. – We implement *curl* in EIGENMATH in three ways.

1st classic vectorial definition²² of *curl* with EIGENMATH

We implement the classic definition from above straight ahead in EIGENMATH and test it with the vector field $F(x, y, z) = (xy, -\sin(z), 1)$:

²¹In classic scientific literature in Europe, the alternative notation *rot* F (read: rotation F) is used.

²²cf. Barth

```

# EIGENMATH
Curl(F) = do(
  Cx = d(F[3],y)-d(F[2],z),
  Cy = d(F[1],z)-d(F[3],x),
  Cz = d(F[2],x)-d(F[1],y),
  C  = (Cx,Cy,Cz),
  C)

F = (x*y,-sin(z),1)    -- example vector field [Marsden, p.917]

Curl(F)    -- testing the definition
curl(F)    -- check against build-in curl

```

EIGENMATH output: $[\cos(z), 0, -x]$

▷ Run script.

2nd tensorial version of *curl*

If we write $F = (F_1, F_2, F_3)$ as a function of position $x = (x^1, x^2, x^3)$ in $\mathbb{R}^3$ using index notation, then the i th component of the curl of F equals (summation suppressed according the EINSTEIN summation convention!)

$$(\text{curl } \mathbf{F})_i(\mathbf{x}) = \varepsilon^{ijk} \frac{\partial}{\partial x^j} F_k(\mathbf{x}) \quad (7.5)$$

in tensorial notation, which follows from the cross product definition in 7.3.2, substituting components of the gradient vector.

Index i is 'free' in the defining term (7.3) in the sense that it is not seen twice on the RHS of the tensor term. But index j and k are to be seen twofold on the RHS - each one once as superscript ('upper index') and once as subscript ('lower index'). Therefore there are 2 implicit hidden summation processes involved in accordance with the EINSTEIN summation convention.

So we have two phases $A \rightarrow B$ to construct the term in EIGENMATH:

$$\varepsilon_{ijk} \cdot \frac{\partial}{\partial x^j} F^k(\mathbf{x})$$

$\textcolor{red}{A} \downarrow \text{sum over } k, \quad \text{i.e. 'contract'}$
 $\textcolor{red}{B} \downarrow \text{sum over } j \quad \text{i.e. 'contract'}$
 $\bullet \quad \quad \quad \text{result}$

We then have *Implementation* N°**2**, where we do not need any memorizing help.

```
# EIGENMATH
F = (x*y, -sin(z), 1)  -- differential vector field in R^3
X = (x,y,z)           -- coordinates in R^3

Eijk(i,j,k) = (j-i)*(k-i)*(k-j)/2 -- permutation tensor

Curl(F,i) = sum(j,1,3, -- ith component of curl
               sum(k,1,3, Eijk(i,j,k) * d(F[k],X[j]) ))

Curl(F,1)           -- 1st component of curl F
Curl(F,2)
Curl(F,3)

CurlF = (Curl(F,1), Curl(F,2), Curl(F,3))
CurlF           -- curl F as vector field
```

EIGENMATH output: $[\cos(z), 0, -x]$

▷ Run script.

3rd fully tensorial version of *curl* using tensor operators

The following wonderful compact implementation is by G. WEIGT. He use again the *i*th component tensor formula of the curl of F , i.e. $(\text{curl } \mathbf{F})_i(\mathbf{x}) = \varepsilon_{ijk} \frac{\partial}{\partial x^j} F^k(\mathbf{x})$. Because we call the tensor operators `contract(.)` and `outer(.)`, we have to use the (multidimensional) matrix table version `eijk` of the permutation tensor. We explain the following two constructions `curl1` and `curl2` later, but would like to whet your appetite for the following chapter, where these tensor operators incl. the Kronecker product $\otimes$ are themed.

```
# EIGENMATH
eijk = zero(3,3,3)  -- define permutation tensor as TABLE
eijk[1,2,3] = 1
eijk[2,3,1] = 1
eijk[3,1,2] = 1
eijk[3,2,1] = -1
```

```

eijk[1,3,2] = -1
eijk[2,1,3] = -1

F = (x*y, -sin(z), 1)  -- differential vector field in R^3

curl1(F) = contract(contract(outer(eijk, d(F,(x,y,z))), 3,4),2,3)
curl1(F)

curl2(F) = contract(dot(eijk, d(F,(x,y,z))), 2,3)
curl2(F)

```

EIGENMATH output: $[\cos(z), 0, -x]$

▷ Run script.

Exercise 20. Calculate the the curl of the following vector fields [Mardsen, p. 914 ff].

- $F(x, y, z) = (e^z, -\cos(xy), z^3y)$.
- $F(x, y, z) = (xy \cos(x), -yz \sin(x), -xy \tan(y))$.
- $F(x, y, z) = (yz, -xz, xy) \cdot 1/(x^2 + y^2 + z^2)$.
- $F(x, y, z) = (ye^z, xe^z, xye^z)$.

Exercise 21. Let $f: \mathbb{R}^3 \rightarrow \mathbb{R}$ be defined through $f(x, y, z) = x^2yz^3$ and $A: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ be defined through $A(x, y, z) = (xz, -y^2, 2x^2y)$.

Calculate $\text{curl}(A)$ and $\text{curl}(f \cdot A)$, cf. [28, p. 151].

Exercise 22. Let f be $f(x, y, z) = xy + yz + zx$ and $A(x, y, z) = (x^2y, y^2z, z^2)$.

Calculate $\text{curl}(A)$ and $\text{curl}(f \cdot A)$ at the point $P = (3, -1, 2)$, cf. [28, p. 151].

Exercise 23. If $A: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ is defined through $A(x, y, z) = (3xz^2, -yz, x + 2z)$, calculate $\text{curl} \text{curl}(A)$, cf. [28, p. 158].

8 The KRONECKER product and the outer product

8.1 outer

EIGENMATH's `outer` function implements the tensor operator $\overset{\text{outer}}{\otimes}$ of tensor algebra. Given two tensors (i.e. multidimensional 'arrays' or tables of numbers), their outer product is again a tensor. The outer product of the tensors T and S is a tensor with dimension $(\dim(T), \dim(S))$. We define the outer product in the special case of vectors, leave the definition of the outer product of matrices for the reader, but give the definition for the general case of two tensors, which includes the case of matrices, cf. [40].

8.1.1 Definition : outer

We quote WIKIPEDIA's outer definition.

Given two vectors $\mathbf{u} = \begin{bmatrix} u_1 \\ u_2 \\ \vdots \\ u_m \end{bmatrix}$ and $\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix}$ of size $m \times 1$ and $n \times 1$, their *outer product*,

denoted $\mathbf{u} \overset{\text{outer}}{\otimes} \mathbf{v}$ is defined as the $m \times n$ matrix obtained by multiplying each element of $\mathbf{u}$ by each element of $\mathbf{v}$:

$$\mathbf{u} \overset{\text{outer}}{\otimes} \mathbf{v} := \begin{bmatrix} u_1 v_1 & u_1 v_2 & \dots & u_1 v_n \\ u_2 v_1 & u_2 v_2 & \dots & u_2 v_n \\ \vdots & \vdots & \ddots & \vdots \\ u_m v_1 & u_m v_2 & \dots & u_m v_n \end{bmatrix} \quad (8.1)$$

In tensorial index notation: $(\mathbf{u} \overset{\text{outer}}{\otimes} \mathbf{v})_{ij} = u_i v_j$

- The outer product $\mathbf{u} \otimes \mathbf{v}$ is equivalent to a matrix multiplication $\mathbf{u} \mathbf{v}^T$, provided that $\mathbf{u}$ is represented as a $m \times 1$ column vector and $\mathbf{v}$ as a $n \times 1$ column vector (which makes $\mathbf{v}^T$ a row vector).

The concept of the outer product $\overset{\text{outer}}{\otimes}$ is best understood by looking at several examples.

8.1.2 Examples of the use of outer

Example 10. (the outer product of two vectors)

```

u=(1,2,3)
v=(3,2,1)
outer(u,v)

```

▷ Run script.

EIGENMATH output:

$$\begin{bmatrix} 3 & 2 & 1 \\ 6 & 4 & 2 \\ 9 & 6 & 3 \end{bmatrix}$$

Example 11. (the outer product of two matrices with *arbitrary* dimensions)

```

A = ((1, 2, 3),( 4, 5, 6))  -- a 2-by-3 matrix
A
B = ((1,1),(1,1))           -- ones(2), a 2-by-2 matrix
B
outer(A,B)                  -- a (2,3,2,2) tensor (multidim. matrix)

```

▷ Run script.

EIGENMATH output:

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$$

$$\begin{bmatrix} \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix} \begin{bmatrix} 2 & 2 \\ 2 & 2 \end{bmatrix} \begin{bmatrix} 3 & 3 \\ 3 & 3 \end{bmatrix} \\ \begin{bmatrix} 4 & 4 \\ 4 & 4 \end{bmatrix} \begin{bmatrix} 5 & 5 \\ 5 & 5 \end{bmatrix} \begin{bmatrix} 6 & 6 \\ 6 & 6 \end{bmatrix} \end{bmatrix}$$

Obviously `outer` respects the block structure of both input matrices.**Example 12.** (the outer product of two multi-dim arrays with *arbitrary* dimensions)

```

# EIGENMATH
i = quote(i)          -- clear name i (otherwise i^2=1 ;)

T = zero(2,2,2)       -- a (2,2,2) tensor
for(i,1,2, for(j,1,2, for(k,1,2, T[i,j,k] = i^2*j*k))) -- initialize T

S = zero(2,2,2,2)     -- a (2,2,2,2) tensor
for(i,1,2, for(j,1,2, for(k,1,2, for(l,1,2, S[i,j,k,l] = i^2*j*k*l))))

```


Exercise 24. Let $A = ((x, y), (z, u))$ and $B = ((a, b), (c, d))$.

Calculate $A \overset{\text{outer}}{\otimes} B$, $B \overset{\text{outer}}{\otimes} (A \overset{\text{outer}}{\otimes} B)$ and $(A \overset{\text{outer}}{\otimes} B) \overset{\text{outer}}{\otimes} (A \overset{\text{outer}}{\otimes} B)$ using the build-in function `outer`. Imagine by mind what output is to be expected.

Exercise 25. (The inner product is the contract of the outer product.)

Given two vectors $\mathbf{u} = (u_1, u_2, \dots, u_m)$ and $\mathbf{v} = (v_1, v_2, \dots, v_n)$.

Verify: The dot product $\mathbf{u} \bullet \mathbf{v} = \text{dot}(\mathbf{u}, \mathbf{v})$ is the contract (trace) of the outer product.

Exercise 26. Think about the algebraic properties of the outer product $\overset{\text{outer}}{\otimes}$.

Check, if it is associative, commutative, bilinear etc.

8.2 kronecker

EIGENMATH's `kronecker` function implements the tensor operator $\otimes$ of tensor algebra, i.e. `kronecker(A,B)` returns the Kronecker tensor product of tensors (e.g. matrices) A and B . The Kronecker product is best understood by thinking about block matrices. If A is a $n \times p$ matrix and B is a $m \times q$ matrix, then the **Kronecker product** $A \otimes B$ is a block matrix that is build from blocks of B , where each block is multiplied by an element of A , cf. [37].

8.2.1 Definition of kronecker

Let A be an $m \times n$ matrix and let B an $p \times q$ matrix.

Then the **Kronecker product** of A and B , symbolically written $A \otimes B$, is the $m \cdot p \times n \cdot q$ matrix explicitly defined by

$$A \otimes B := (a_{ij} \cdot B) = \begin{pmatrix} a_{11}B & \cdots & a_{1n}B \\ \vdots & \ddots & \vdots \\ a_{m1}B & \cdots & a_{mn}B \end{pmatrix} \quad (8.2)$$

$$= \text{kronecker}(A, B) \quad \text{in EIGENMATH} \quad (8.3)$$

In words: If A is an m -by- n matrix and B is a p -by- q matrix, then `kronecker(A,B)` is an $m \cdot p$ -by- $n \cdot q$ matrix, which is calculated by taking all possible products between the elements of A and those of B , i.e. the Kronecker tensor product of A and B is a large matrix formed by multiplying B by each element of A .

8.2.2 Examples of the use of kronecker()

Example 13.

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix} \otimes \begin{pmatrix} 7 & 8 \\ 9 & 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot \begin{pmatrix} 7 & 8 \\ 9 & 0 \end{pmatrix} & 2 \cdot \begin{pmatrix} 7 & 8 \\ 9 & 0 \end{pmatrix} \\ 3 \cdot \begin{pmatrix} 7 & 8 \\ 9 & 0 \end{pmatrix} & 4 \cdot \begin{pmatrix} 7 & 8 \\ 9 & 0 \end{pmatrix} \\ 5 \cdot \begin{pmatrix} 7 & 8 \\ 9 & 0 \end{pmatrix} & 6 \cdot \begin{pmatrix} 7 & 8 \\ 9 & 0 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} 7 & 8 & 14 & 16 \\ 9 & 0 & 18 & 0 \\ 21 & 24 & 28 & 32 \\ 27 & 0 & 36 & 0 \\ 35 & 40 & 42 & 48 \\ 45 & 0 & 54 & 0 \end{pmatrix}$$

```
# EIGENMATH input:
kronecker( ((1,2),(3,4),(5,6)), ((7,8),(9,0)) )
```

EIGENMATH output:

cf. example 13.

We now repeat the examples of `outer` to contrast them with the results of `kronecker`.

Example 14. (the Kronecker product of two vectors)

```
# EIGENMATH
u=(1,2,3)
v=(3,2,1)
kronecker(u,v)
```

• Mark & Copy the blue code lines. ▷ Then Click here and Paste it into the input box `□RUN`. Press RUN.
EIGENMATH output:

$$\begin{bmatrix} 3 \\ 2 \\ 1 \\ 6 \\ 4 \\ 2 \\ 9 \\ 6 \\ 3 \end{bmatrix}$$

In contrast to `outer` the output of `kronecker` has stacked the rows to one long column.

Example 15. (the Kronecker product of two matrices with *arbitrary* dimensions)

```
# EIGENMATH
A = ((1, 2, 3),( 4, 5, 6))  -- a 2-by-3 matrix
A
B = ((1,1),(1,1))          -- ones(2), a 2-by-2 matrix
B
kronecker(A,B)              -- a (2,3,2,2) tensor (multidim. matrix)
```

▷ Run script.

EIGENMATH output:

$$\begin{bmatrix} 1 & 1 & 2 & 2 & 3 & 3 \\ 1 & 1 & 2 & 2 & 3 & 3 \\ 4 & 4 & 5 & 5 & 6 & 6 \\ 4 & 4 & 5 & 5 & 6 & 6 \end{bmatrix}$$

In contrast to `outer` the output of `kronecker` has cleared the brackets of the inner matrices to give one single matrix. Obviously `kronecker` 'flattens' the block structure of both input matrices.

Remark. The KRONECKER product of two tensors with *arbitrary* dimensions is currently not implemented in EIGENMATH. So there is no return value e.g for the tensor product $(T[i, j, k] = i^2 * j * k) \otimes (S[i, j, k, l] = i^2 * j * k * l)$.

Exercise 27. Let $A = \begin{bmatrix} x & y \\ z & u \end{bmatrix}$ and $B = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$. Calculate $A \otimes B$, $B \otimes (A \otimes B)$ and $(A \otimes B) \otimes (A \otimes B)$

Exercise 28. Write a function `ones(n)`, which returns a n -by- n matrix whose all entries are 1, e.g. `ones(2)` is $\begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}$. EIGENMATH output: $((1,1),(1,1))$

8.3 Applications of kronecker

8.3.1 The direct sum $\oplus$

Let A be an $m \times n$ matrix and B an $p \times q$ matrix. Then the *direct sum* of A and B , symbolically written $A \oplus B$, is the $m \cdot p \times n \cdot q$ matrix explicitly defined by

$$A \oplus B := \begin{pmatrix} A & O \\ O & B \end{pmatrix} \quad (8.4)$$

i.e. for any arbitrary matrices A and B the direct sum (aka tensor sum) is defined as the block diagonal matrix of A and B . Both zero matrices O are of appropriate dimension.

Example 16. (cf. [30, p. 7], [29, p. 28])

$$\begin{pmatrix} 1 & 2 & 3 \end{pmatrix} \oplus \begin{pmatrix} 4 & 5 \\ 6 & 7 \end{pmatrix} = \begin{pmatrix} \begin{pmatrix} 1 & 2 & 3 \end{pmatrix} & \begin{pmatrix} 0 & 0 \end{pmatrix} \\ \begin{pmatrix} 0 & 0 & 0 \end{pmatrix} & \begin{pmatrix} 4 & 5 \\ 6 & 7 \end{pmatrix} \end{pmatrix} = \begin{pmatrix} 1 & 2 & 3 & 0 & 0 \\ 0 & 0 & 0 & 4 & 5 \\ 0 & 0 & 0 & 6 & 7 \end{pmatrix}$$

We show how the direct sum could be used in EIGENMATH using function `kronecker`.

```
# EIGENMATH

A=((1,2),(3,4))
B=((4,5),(6,7))
U=((1,0),    -- Upper
   (0,0))
L=((0,0),    -- Lower
   (0,1))

kronecker(U,A)

directSum(A,B)= kronecker(U,A) + kronecker(L,B)
directSum(A,B)
```

▷ Run script.

EIGENMATH output:

$$\begin{bmatrix} 1 & 2 & 0 & 0 \\ 3 & 4 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 2 & 0 & 0 \\ 3 & 4 & 0 & 0 \\ 0 & 0 & 4 & 5 \\ 0 & 0 & 6 & 7 \end{bmatrix}$$

The first matrix is the result of the call `kron(U,A)` and the second matrix is the result of the call `directSum(A,B)`

Exercise 29. Calculate the direct sum of A and B from example 9 using `directSum(A,B)`

8.3.2 Construction of the basis of $\mathbb{R}^4$ from the basis of $\mathbb{R}^2$, cf. [30, p. 7]

Let $u = (1, 0), v = (0, 1)$ be the canonical standard basis in $\mathbb{R}^2$.

Verify, that $(u \otimes u, u \otimes v, v \otimes u, u \otimes u)$ is the standard basis in $\mathbb{R}^4$.

We use EIGENMATH's `kron(.)`.

```
# EIGENMATH
kron(A,B)=kron(A,B)

u=(1,0)
v=(0,1)

-- Basis R^4
kron(u,u)
kron(u,v)
kron(v,u)
kron(v,v)
```

▷ Run script.

EIGENMATH output: $[1, 0, 0, 0], \dots, [0, 0, 0, 1]$

Exercise 30. Construct an orthonormal basis of $\mathbb{C}^4$ with the help of `kron`.

Hint: start with $u = 1/\sqrt{2} \cdot (1, i)$ and $u = 1/\sqrt{2} \cdot (1, -i)$.

Exercise 31. (cf. [?, p. 112]) A basis in the Hilbert space $M_2(\mathbb{C})$ of 2-by-2 matrices is given by the PAULI spin matrices

$$\sigma_0 = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \sigma_1 = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}, \sigma_2 = \begin{bmatrix} 0 & -i \\ i & 0 \end{bmatrix}, \sigma_3 = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}$$

including the 2-by-2 identity matrix. Use this orthonormal basis and the Kronecker products $\sigma_j \otimes \sigma_k$ with $j, k = 0, 1, 2, 3$ to find a basis in the Hilbert space $M_4(\mathbb{C})$.

Solution:

```
# EIGENMATH
kron(A,B)=kronecker(A,B)
-- Pauli spins is basis in M2(C)
s(0) = ((1,0),(0,1))
s(1) = ((0,1),(1,0))
s(2) = ((0,-i),(i,0))
s(3) = ((1,0),(0,-1))

for(j,0,3,
  for(k,0,3,
    print( kron(s(j),s(k)) )    -- Basis M4(C)
  ))
```

▷ Run script.

Exercise 32. Do some more of the examples 2.1 ... 2.12 in [29] using EIGENMATH.

9 contract

The *contraction* of a tensor acts by setting equal a pair of indices²³ of the tensor to each other and summed over. In Einstein notation this summation is implicit in the notation. The result is a new tensor with order reduced by 2, cf. [41].

Contraction of tensors is heavily used in Differential Geometry or in General Relativity. Tensor contraction is a generalization of the trace of a matrix.

9.1 Definition of contract

Let $\mathbf{T} = T^i_j$ be a tensor. Then its *contraction* is defined by

$$T^i_j \delta_i^j = T^j_j = \sum_{j=1}^n T^j_j = T^1_1 + \cdots + T^n_n \quad (9.1)$$

In EIGENMATH this is expressed with the call of `contract(T,i,j)`.

- A general contraction is denoted by labeling one (covariant, lower) index and one (contravariant, upper) index with the same letter, summation over that index being implied by the summation convention. The resulting contracted tensor inherits the remaining indices of the original tensor, cf. [41].

- For example, contracting a tensor T of type (2,2) on the second and third indices (i.e. *b*) to create a new tensor U of type (1,1) is written as

$$T^{ab}_{bc} = \sum_b T^{ab}_{bc} = T^{a1}_{1c} + T^{a2}_{2c} + \cdots + T^{an}_{nc} = U^a_c$$

9.2 Examples of contract

Example 17.

```
# EIGENMATH
u=(1)
contract(u)

A = ((1, 2, 3),( 4, 5, 6))
-- contract(A) -- ERROR: unequal tensor dimensions !

B = ((1,2),(3,4))          -- contract = trace
contract(B)
```

EIGENMATH output:

1 error 5

▷ Run script.

²³one a subscript (upper) index, the other a superscript (lower) index

Comment. `contract(B)` contracts ('adds') the diagonal elements of B , i.e. $1 + 4 = 5$. Tensor B (a matrix of shape 2-by-2) is reduced into a new tensor of type 1-by-1, i.e. a number 5.

Example 18.

```
# EIGENMATH
i = quote(i)          -- clear i

T = zero(2, 2,2)
for(i,1,2, for(j,1,2, for(k,1,2, T[i,j,k] = i^2*j*k)))
T
contract(T,1,2)      -- (1)
contract(T,2,3)      -- (2)
```

▷ Run script.

EIGENMATH output:

$$T = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} \\ \begin{bmatrix} 4 & 8 \\ 8 & 16 \end{bmatrix} \end{bmatrix}$$

$$\begin{bmatrix} 9 \\ 18 \end{bmatrix}$$

$$\begin{bmatrix} 5 \\ 20 \end{bmatrix}$$

Comment. In (1) the command `contract(T,1,2)` adds $(1+8, 2+16) = (9, 18) = \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 8 \\ 16 \end{bmatrix}$, i.e. it adds the 1st column of 1st submatrix to the 2nd column of the second submatrix. In (2) the command `contract(T,2,3)` adds $(1 + 4, 4 + 16) = (5, 20) = \begin{bmatrix} 1 \\ 4 \end{bmatrix} + \begin{bmatrix} 4 \\ 16 \end{bmatrix}$, i.e. it adds the trace of 1st submatrix to the trace of the second submatrix. Tensor T (an array of shape 2-by-2-by-2) is reduced into a new tensor of type 2-by-1, i.e. a vector.

Let's elaborate a bit on it. If we fill tensor T with arbitrary values we are able to follow the path of picking the right entries of T by `contract(.,1,2)`:

```
# EIGENMATH
T = zero(2, 2,2)  -- an multi-array with 2^3=8 general elements
"contract(T,1,2):"
-- tensor index
-- i j k
-- 1 2 3
-- . .      (.,1,2) : choose all entries with 'i=j' aka '1=2'
--                  i.e. all entries of type 11x and 22x.
T[1,1,1] = a -- .
```

```

T[1,1,2] = b  -- ..
T[1,2,1] = c
T[1,2,2] = d
T[2,1,1] = e
T[2,1,2] = f
T[2,2,1] = g  -- .  = a + g
T[2,2,2] = h  -- .. = b + h

```

That is explicit: $\text{contract}(T, 1, 2) = \begin{bmatrix} \circ + \circ \\ \circ\circ + \circ\circ \end{bmatrix} = \begin{bmatrix} T[1,1,1] + T[2,2,1] \\ T[1,1,2] + T[2,2,2] \end{bmatrix}$.

Now look at $\text{contract}(\cdot, 2, 3)$:

```

"contract(T,2,3):"
T = zero(2, 2,2)
-- tensor index
-- i j k
-- 1 2 3
-- . .      (..,2,3) : choose all entries with 'j=k' aka '2=3'
--                      i.e. all entries of type    x11  and  x22.
T[1,1,1] = a  -- .
T[1,1,2] = b
T[1,2,1] = c
T[1,2,2] = d  -- .  = a + d
T[2,1,1] = e  -- ..
T[2,1,2] = f
T[2,2,1] = g
T[2,2,2] = h  -- .. = e + h

```

That is explicit: $\text{contract}(T, 2, 3) = \begin{bmatrix} \circ + \circ \\ \circ\circ + \circ\circ \end{bmatrix} = \begin{bmatrix} T[1,1,1] + T[1,2,2] \\ T[2,1,1] + T[2,2,2] \end{bmatrix}$.

Example 19.

```

# EIGENMATH
S = zero(2, 2,2,2)
for(i,1,2, for(j,1,2, for(k,1,2, for(l,1,2,
    S[i,j,k,l] = i^2*j*k*l))))

S
contract(S)          -- (1), i.e. contract(S) = contract(S,1,2)
contract(S,2,3)      -- (2)
contract(S,1,3)      -- (3)

```

▷ Run script.

EIGENMATH output:

$$S = \begin{bmatrix} \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix} & \begin{bmatrix} 2 & 4 \\ 4 & 8 \end{bmatrix} \\ \begin{bmatrix} 4 & 8 \\ 8 & 16 \end{bmatrix} & \begin{bmatrix} 8 & 16 \\ 16 & 32 \end{bmatrix} \end{bmatrix}$$

$$\begin{bmatrix} 9 & 18 \\ 18 & 36 \end{bmatrix}$$

$$\begin{bmatrix} 5 & 10 \\ 20 & 40 \end{bmatrix}$$

$$\begin{bmatrix} 9 & 18 \\ 18 & 36 \end{bmatrix}$$

Comment. In (1) and (3) `contract(S)≡contract(S,1,2)=contract(S,1,3)` adds $(1 + 8, 2 + 16) = (9, 18) = \begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 8 \\ 16 \end{bmatrix}$, i.e. it adds the 1st column of 1st 'sub'matrix to the 2nd column of the second 'sub'matrix.

In (2) the command `contract(S,2,3)` picks the trace of each matrix-part of S and puts it as entry at the corresponding position in the new reduced tensor of type 2-by-2, i.e. a matrix.

Exercise 33. Watch the output of `contract` by looking at an arbitrary $(2,2,2,2)$ tensor S and elaborate a bit on it as in example 11. Therefore fill tensor S with arbitrary values and follow the path of picking the right entries of S by `contract(.,1,2)` and `contract(.,2,3)`:

```
# EIGENMATH
S = zero(2, 2,2,2)  -- we need 2^4=16 elements
S[1,1,1,1] =a
S[1,1,1,2] =b
-- ...
S[2,2,2,1] =o
S[2,2,2,2] =p
```

Example 20. Excurs: Basic Contraction Process with `itensor` package from MAXIMA

We demonstrate in short how one may use the contraction operator from MAXIMA's `itensor` package.

CONTRACTION	<code>itensor</code>
1 Load the package.	<code>load("itensor");</code>
2 inform itensor that g is the metric tensor.	<code>imetric(g);</code>
3 enable index raising/lowering via metric.	<code>defcon(g);</code>
4 let contract performs the EINSTEIN-summation.	<code>contract(expression);</code>

Let's contract a tensor (vector) using the metric:

```
load("itensor")$
imetric(g)$           /* set metric name to g */
defcon(g)$            /* define contraction for g */
ishow(g([i,j],[ ])*v([ ],[j]))$ /* display g_ij * v^j */
contract(%);          /* contract index j : g_ij * v^j -> v_i */
```

▷ Run script.

YAMWI output:

```
[%i4] ishow(g([i,j],[ ])*v([ ],[j]));
[%t4]      j
          v  g
           i j
[%o4]      v([ ], [j]) g([i, j], [ ])
[%i5] contract(%);
[%o5]      v([i], [ ])
[%i6] ishow(%);
[%t6]      v
           i
```


9.3 Case study : construction of the *curl*

In chapter 7.3.2 *3rd* we looked at the wonderful compact EIGENMATH implementation of the *curl* by G. WEIGT. We started with an arbitrary vector-valued function $F: \mathbb{R}^3 \rightarrow \mathbb{R}^3$ defined through its component functions $F = (F_1, F_2, F_3)$ in the coordinates (x^1, x^2, x^3) . Then we used the *i*th component tensorial formula of the curl of F , i.e.

$$(\text{curl } \mathbf{F})_i(\mathbf{x}) = \varepsilon_{ijk} \frac{\partial}{\partial x^j} F^k(\mathbf{x}) \quad (9.2)$$

Task: How to translate formula (9.2) to EIGENMATH?

Solution: Analyse formula (9.2). Index i is 'free' in the defining term in the sense that it is not seen twice on the RHS of (9.2). But index j and k are to be seen twofold on the RHS - each one once as superscript ('upper index') and once as subscript ('lower index'). Therefore there are 2 implicit hidden summation processes involved in accordance with the EINSTEIN summation convention. So we have three phases $A \rightarrow B \rightarrow C$ to construct the term in EIGENMATH:

$$\varepsilon_{ijk} \cdot \frac{\partial}{\partial x^j} F^k(\mathbf{x})$$

A ↓ outer to build products

B ↓ contract over k , i.e. sum

C ↓ contract over j , i.e. sum

We are able to use EIGENMATH, because we know the ingredients, i.e. the tensor operators *contract*, *outer* and the (multidimensional) table version *eijk* of the permutation tensor.

```
# EIGENMATH
F = (F1(),F2(),F3())  -- the vector-valued generic function F=(F1,F2,F3)

eijk = zero(3,3,3)    -- the permutation tensor eijk
eijk[1,2,3] = 1
eijk[2,3,1] = 1
eijk[3,1,2] = 1
eijk[3,2,1] = -1
eijk[1,3,2] = -1
eijk[2,1,3] = -1

eijk

d(F,(x1,x2,x3))      -- the JACOBI matrix of F of type 3x3
```

$$\begin{bmatrix} d[F_{10},x_1] & d[F_{10},x_2] & d[F_{10},x_3] \\ d[F_{20},x_1] & d[F_{20},x_2] & d[F_{20},x_3] \\ d[F_{30},x_1] & d[F_{30},x_2] & d[F_{30},x_3] \end{bmatrix}$$

```

-- ----- A -> B -> C becomes the curl of F

A = outer(eijk, d(F,(x1,x2,x3)))
--      |
--      eijk , dF
--      |
--      outer
--      |
--      (eijk * dF_k /dx_j)
--      123      4      5

-- A is now a multi-matrix of 27 elements, i.e. a 3 x 3x3 Tensor
A      -- take a look at A

-- 1st contraction on k, which is at index positions 3 and 4
--      | |
B = contract(A,3,4)      -- i.e. EINSTEIN sum with index k
--      |
--      sum over k=3=4
--      Sum eij(k) dF_(k) /dx_j
--      12 .      .      3
--      j is now at index positions 2 and 3 (k is 'killed')

B      -- take a look at B

-- 2nd Contraction on j, which lives now at index positions 2 and 3
--      | |
C = contract(B,2,3)      -- i.e. EINSTEIN sum with index j
--      |
--      sum over j=2=3
--      Sum eij(k) dF_(k) /dx_j
--      1. .      .      . (j is killed)
-- i.e. add (trace) all elements on the main diagonals of B
-- i.e. 0 + dF3/dx2 - dF2/dx3, ...

C      -- is now the curl of F

```

▷ Run script.

$$C = \begin{bmatrix} -d[F_2 0, x_3] + d[F_3 0, x_2] \\ d[F_1 0, x_3] - d[F_3 0, x_1] \\ -d[F_1 0, x_2] + d[F_2 0, x_1] \end{bmatrix}$$

Now we can give a shorted version:

```
--      ijk      k j      -- 0. index list by equation (9.2)
A = outer(eijk, d(F,(x1,x2,x3))) -- 1. do all element-wise products
A      -- ..3      4 .
B = contract(A,3,4)           -- 2. sum across k
--      ij.      . j
B      -- 12.      . 3
C = contract(B,2,3)          -- 3. sum across j
C
```

▷ Run script.

Shortest version by concatenation of processes A-B-C:

```
curlF = contract(contract(outer(eijk, d(F,(x1,x2,x3))),3,4),2,3)
curlF
```

▷ Run script.

Exercise 34. Repeat the reasoning above using the concrete function $F = (ye^z, xe^z, xye^z)$.

Exercise 35. (orbital velocity in circular motion - a second view at *cross*)

We quote from the German help for `levi_civita` function. For circular motion, the orbital velocity $\vec{w}$ is the cross product of the angular velocity $\vec{v}$ and the position vector $\vec{r}$. Thus, we have $\vec{w} = \vec{v} \times \vec{r}$.

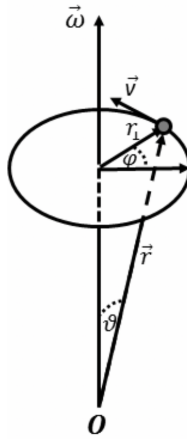


Figure 19: $\odot$: circular motion of point $\bullet$ with orbital velocity $\vec{w}$, angular velocity $\vec{v}$ and position vector $\vec{r}$.

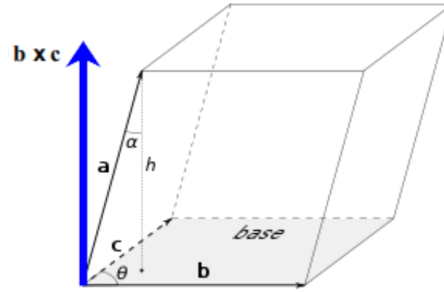
In any basis, the a -th component of the orbital velocity $\vec{w}$ is given by the tensorial formula, cf. (7.3.2):

$$w^a = \varepsilon^{abc} v_b r_c \quad (9.3)$$

Task: calculate `cross([1,2,3],[4,5,6])` using tensor technics.

Hint: First, introduce a tensorial notation for the first component of cross product using the permutaion symbol. The expression is then simplified for the first component to the familiar definition of the cross product.

Exercise 36. The scalar triple product²⁴ $\text{spat}(a, b, c) := a \cdot (b \times c)$ of three vectors $\mathbf{a}$, $\mathbf{b}$ and $\mathbf{c}$ is defined using the levi-civita symbol and then simplified using 'contract'.



□: the plane, spanned by vectors $\mathbf{b}$ and $\mathbf{c}$.

Figure 20: ↑: the cross product $(b \times c)$

[$\mathbf{a}, \mathbf{b}, \mathbf{c}$]: the spat product as volume of the feldspare

Task: calculate `spat( [1,2,3], [4,5,6], [7,8,8] )` using tensor technics.

Hint: use the defining term for function $\text{spat} : (\mathbb{R}^3)^3 \rightarrow \mathbb{R}$

$$\text{spat}(\mathbf{a}, \mathbf{b}, \mathbf{c}) := \sum_{i=1}^3 \sum_{j=1}^3 \sum_{k=1}^3 \varepsilon^{ijk} a_i b_j c_k \quad (9.4)$$

and translate it to tensor language:

$$\text{spat}(a, b, c) \stackrel{\text{Math}}{:=} a \cdot (b \times c) \longrightarrow \varepsilon^{ijk} \stackrel{\text{tensor}}{a_i b_j c_k} \longrightarrow \stackrel{\text{EigenMath}}{\text{contract}}$$

Remark. If you feel that this section closely resembles the one on determinants and cross, you are correct: the *spat* product is merely a geometric variant of the determinant – hence the comparable tensorial representation. Nevertheless, we included this section here to illustrate the use of the `contract`'ion operator through an additional, but familiar example.

²⁴Figure 20 adapted from wikipedia, $a(b \times c)$. We use the name 'spat', because it invokes the typical picture of a 'feldspare' aka parallelepiped in your head. In German speaking counties the scalar triple product is therefore often called 'spat product'.

10 The LEVI-CIVITA Tensor ϵ

The LEVI-CIVITA tensor acts like a metric and is often used to define tensorial objects, cf. [?, p. 340] or [30, p.50] for an application in the Nambu mechanics. It may per example also be used to define the curl operator or the cross product, cf. [6, p.148]. A heavy use is in the operation of lowering and *raising the index* in the so-called index juggling.

10.1 Definition

We follow GRINFELD [6, p.148].

Let Z denote the determinant of the (covariant) metric tensor $g = g_{ij}$, i.e. $Z := \det(g)$. Then we define

1. The square root of Z is called the *volume element* V of the metric, i.e.

$$V := \sqrt{\det(g)} \quad (10.1)$$

2. The **LEVI-CIVITA tensor** ϵ_{ijk} resp. E^{ijk} is defined as the volume scaled permutation symbol e_{ijk} , i.e.

$$\epsilon_{ijk} := V \cdot e_{ijk} \quad (10.2)$$

$$E^{ijk} := \frac{e^{ijk}}{V} \quad (10.3)$$

Remark. We have

LEXICON	<i>Math</i>	EIGENMATH
<i>metric tensor</i> g :	g_{ij}	g or gdd
<i>inverse of metric tensor</i> g^{-1} :	g^{ij}	ginv or guu
<i>volume element</i> V :	$\sqrt{\det(g)}$	sqrt(det(gdd))
LEVI-CIVITA <i>tensor</i> ϵ_{ijk} :	$\sqrt{\det(g)} \cdot e_{ijk}$	epsilon
LEVI-CIVITA <i>tensor</i> E^{ijk} :	$1/\sqrt{\det(g)} \cdot e^{ijk}$	Epsilon

In EIGENMATH's output window the small Greek letter *epsilon* ε is printed as ϵ and the big Greek letter *Epsilon* E is printed as E, not as E .

10.2 Examples

Example 21. The LEVI-CIVITA tensor in an affine coordinate systems in $\mathbb{R}^2$ can be calculated by EIGENMATH as follows, cf. [6, p. 148] using the volume element.

```
# EIGENMATH
-- metric tensor in affine coordinates, see example 14
gdd = zero(2,2)
      gdd[1,1]= 4
      gdd[1,2]= 1
      gdd[2,1]= 1
      gdd[2,2]= 1

-- volume element in affine coordinates
Z = det(gdd)
Z

V = sqrt(Z)
V

-- permutation tensor eij in R^2 as table
eij = zero(2,2)
      for(i,1,2, for(j,1,2, eij[i,j]=j-i ))
eij

-- LEVI-CIVITA tensor epsilon
epsilon = V * eij
epsilon

Epsilon = 1/V * eij
Epsilon
```

▷ Run script.

EIGENMATH output:

$$\begin{aligned}
 Z &= 3 \\
 V &= 3^{1/2} \\
 e_{ij} &= \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix} \\
 \varepsilon &= \begin{bmatrix} 0 & 3^{1/2} \\ -3^{1/2} & 0 \end{bmatrix} \\
 E &= \begin{bmatrix} 0 & \frac{1}{3^{1/2}} \\ -\frac{1}{3^{1/2}} & 0 \end{bmatrix}
 \end{aligned}$$

Example 22. The LEVI-CIVITA tensor in polar coordinates in $\mathbb{R}^2$ is calculated by EIGEN-MATH using the results of example 15 and using the volume element.

```
-- metric tensor in polar coordinates
gdd = zero(2,2)
      gdd[1,1]= 1
      gdd[2,2]= r^2

-- volume element in polar coordinates
Z = det(gdd)
Z
V = sqrt(Z)
V

-- permutation tensor eij in R^2 as table
eij = zero(2,2)
      for(i,1,2, for(j,1,2, eij[i,j]=j-i ))
eij

-- LEVI-CIVITA tensor epsilon
epsilon = V * eij
epsilon
Epsilon = 1/V * eij
Epsilon
```

▷ Run script.

EIGENMATH output:

$$Z = r^2$$

$$V = r$$

$$e_{ij} = \begin{bmatrix} 0 & 1 \\ -1 & 0 \end{bmatrix}$$

$$\epsilon = \begin{bmatrix} 0 & r \\ -r & 0 \end{bmatrix}$$

$$E = \begin{bmatrix} 0 & \frac{1}{r} \\ -\frac{1}{r} & 0 \end{bmatrix}$$

Exercise 37. Calculate the LEVI-CIVITA tensor in cylindrical coordinates in $\mathbb{R}^3$ by EIGEN-MATH using the results of example 16.

11 The CHRISTOFFEL Symbol Γ_{ij}^k

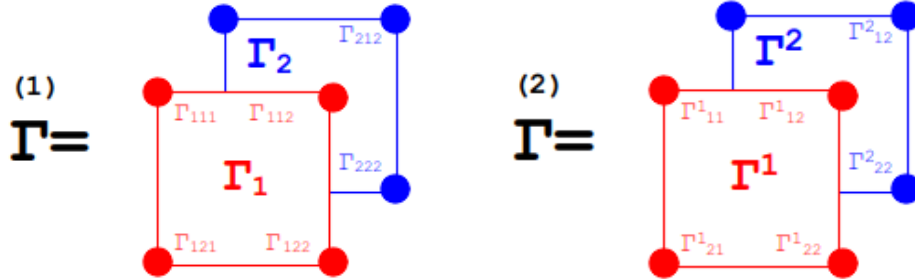


Figure 21: How to imagine the CHRISTOFFEL symbol of first kind $\Gamma^{(1)}$ and the CHRISTOFFEL symbol of second kind $\Gamma^{(2)}$ as $2^3 = 8$ numbers arranged in a tensorial manner in 2 layers. This structure is respected by the output of EIGENMATH.

The CHRISTOFFEL symbol plays a dominant role in Differential Geometry and General Relativity, where we define the RIEMANN curvature tensor R_{bcd}^a , the RICCI tensor R_{bd} and the EINSTEIN curvature scalar R with the help of the CHRISTOFFEL symbol of 1st and 2nd kind. Then it is possible to calculate curvature variants of surfaces²⁵ in space. We use the results in [6, p.69] or [30, p.62] to implement the CHRISTOFFEL symbols in EIGENMATH.

11.1 Definition

Let $g = (g_{ij})$ be a metric tensor with inverse tensor $ug = g^{ij}$ in the coordinate system (x^1, x^2, x^3) in e.g. $\mathbb{R}^3$. Let $R : U \rightarrow \mathbb{R}^n$ be the position vector of the coordinate system.

a. The **CHRISTOFFEL symbol of 1st kind, denoted $\Gamma_{ij,k}$** where $i, j, k \in \{1, 2\}$ is defined by

$$\Gamma_{ij,k} := \frac{1}{2} \cdot (g_{jk,i} + g_{ik,j} - g_{ij,k}) \stackrel{!}{=} \frac{\partial^2 R}{\partial u_i \partial u_j} \cdot \frac{\partial R}{\partial u_k} \equiv R_{u^i u^j} \bullet R_{u^k} \quad (11.1)$$

where $g_{ij,k} \equiv \frac{\partial g_{ij}}{\partial x^k}$ denotes the partial derivative of g_{ij} with respect to the coordinate x^k .

b. The **CHRISTOFFEL symbol of 2nd kind, denoted Γ_{ij}^k** is defined by using the inverse metric tensor g^{ij} and memorizing the EINSTEIN summation conventionas

$$\Gamma_{ij}^k := \frac{1}{2} \cdot g^{km} \cdot (g_{mi,j} + g_{mj,i} - g_{ij,m}) \quad (11.2)$$

²⁵This will be done in the next script in this series with the title *Elementary Differential Geometry of Surfaces*.

- Decoding the EINSTEIN summation convention and to prepare for the implementation of these formulas in EIGENMATH we write equation (11.2) explicit as

$$\Gamma_{ij}^k = \frac{1}{2} \cdot \sum_{m=1}^2 g^{km} \cdot (g_{mi,j} + g_{mj,i} - g_{ij,m}) \quad (11.3)$$

- If we denote the partial derivative with respect to the coordinate x_j in short by ∂_i , we may write formula b. in better memorizable shape, cf. [17, p.91]:

$$\Gamma_{ij}^k = \frac{1}{2} \cdot \sum_{m=1}^3 g^{km} \cdot (\partial_j g_{mi} + \partial_i g_{mj} - \partial_m g_{ij})$$

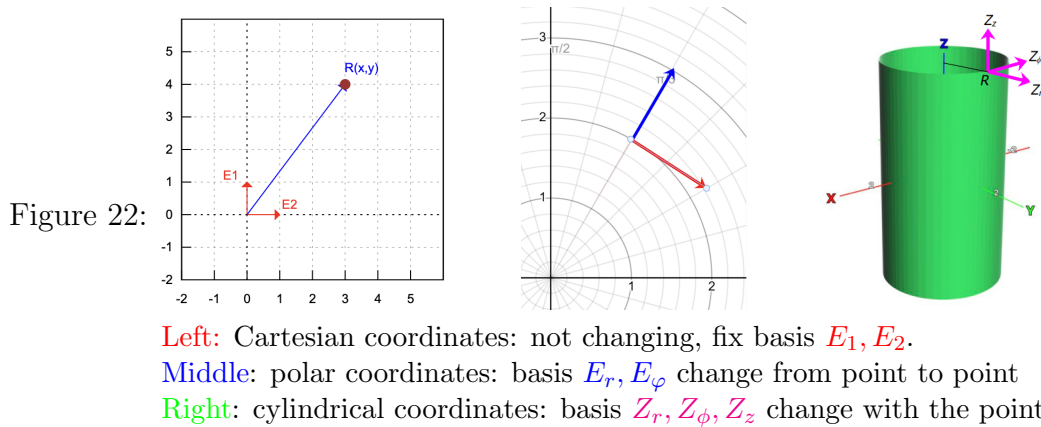
Implementing formulas a. and b. in EIGENMATH should enhance a clear understanding of these constructs.

Remark. We have

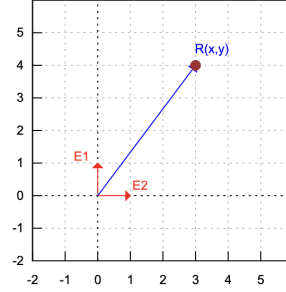
LEXICON	Math	EIGENMATH
metric tensor g :	g_{ij}	g or gdd
inverse of metric tensor g^{-1} :	g^{ij}	ginv or guu
CHRISTOFFEL tensor of 1st kind:	$\Gamma_{i,jk}$	Gammadd or GammaI
CHRISTOFFEL tensor of 2nd kind:	Γ_{ij}^k	Gammaudd or GammaII

11.2 Examples

We now demonstrate two implementations of the CHRISTOFFEL symbols: one closely oriented on formula a. and its term structure and the other using clever compact tensor techniques such as contract and index juggling, with was given by G. WEIGT. We test our code in affine, polar and cylindrical coordinates in the plane $\mathbb{R}^2$ resp. in space $\mathbb{R}^3$.



11.2.1 Calculation of the CHRISTOFFEL symbol of 2nd kind using classic loops



Example 23. The CHRISTOFFEL symbol of 2nd kind in *Cartesian coordinates* in $\mathbb{R}^2$ is calculated using results of example 15. Let's step through all coefficients of the CHRISTOFFEL symbol of 2nd kind using a for-loop.

```
-- metric tensor in Cartesian coordinates
gdd = zero(2,2)
    gdd[1,1]= 1
    gdd[1,2]= 0
    gdd[2,1]= 0
    gdd[2,2]= 1

-- use Cartesian coordinates
X = (x,y)

-- calculate inverse guu to gdd
guu = inv(gdd)
guu

-- calculate CHRISTOFFEL symbol 2nd kind
GammaII=zeros(2,2,2)    -- array shape
for( k,1,2,              -- fill Gamma
    for( l,1,2,
        for( m,1,2,
            GammaII[k,l,m] =                -- (1)
                sum(i,1,2, guu[k,i]/2 *      -- (2)
                    ( d( gdd[m,i], X[1]) +    -- (3)
                      d( gdd[l,i], X[m]) -
                      d( gdd[m,l], X[i]) )
                ))))

GammaII                -- (4)
GammaII[1,2,2]         -- (5)
GammaII[2,1,2]
```

▷ Run script.

EIGENMATH output:

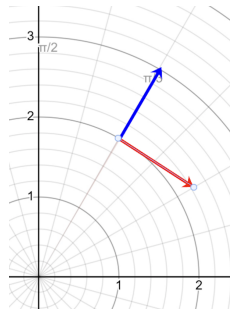
$$g_{uu} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

$$\Gamma = \begin{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \end{bmatrix}$$

$$\begin{matrix} 0 \\ 0 \end{matrix}$$

Comment. In (1) we give $\text{GammaII}[k,1,m] = \Gamma_{lm}^k$ the value in accordance with formula 11.1.b. By the way, we respect the EINSTEIN summation convention by explicit summing the three partial derivatives up. The resulting fully filled symbol is outputted in (4). (5) verifies that all components of the affine CHRISTOFFEL symbol Γ_{II} vanishes at all points. Later we will argue, that therefore this space is 'flat'.

Example 24. Calculate the CHRISTOFFEL symbol of 2^{nd} kind in a polar coordinate system in $\mathbb{R}^2$.



```
# EIGENMATH
-- metric tensor in polar coordinates
gdd = zero(2,2)
      gdd[1,1]= 1
      gdd[2,2]= r^2

-- use polar coordinates
X = (r,theta)
X

-- calculate guu = (gdd)^{-1}
guu = inv(gdd)
guu
```

```

-- calculate GammaII using a for-loop
GammaII=zero(2,2,2)      -- array of type (2,2,2)

for( k,1,2,
  for( l,1,2,
    for( m,1,2,
      GammaII[k,l,m] =
        sum(i,1,2, guu[k,i]/2 *
          ( d(gdd[m,i],X[l]) +
            d(gdd[l,i],X[m]) -
            d(gdd[m,l],X[i]) )
        ))))

GammaII      --(1)
GammaII[1,2,2]  --(2)
GammaII[2,1,2]

```

▷ Run script.

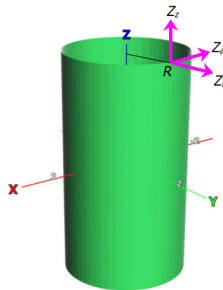
EIGENMATH output:

$$\Gamma = \begin{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & -r \end{bmatrix} \\ \begin{bmatrix} 0 & \frac{1}{r} \\ \frac{1}{r} & 0 \end{bmatrix} \end{bmatrix}$$

$-r$
 $\frac{1}{r}$

Comment. In (1) we see that the fully filled CHRISTOFFEL symbol Γ_{II} has non-zero components. (6) picks two of these non-zero components of Γ_{II} , namely Γ_{22}^1 and Γ_{12}^2 .

Example 25. Calculate the CHRISTOFFEL symbol of 2^{nd} kind in cylindrical coordinates in $\mathbb{R}^3$.



```
# EIGENMATH
-- metric tensor in cylindrical coordinates
gdd = zero(3,3)
      gdd[1,1]= 1
      gdd[2,2]= r^2
      gdd[3,3]= 1

gdd

-- X is cylindrical coordinates, t means theta
X = (r,t,z)
X

-- calculate guu, i.e. the inverse of gij
guu = inv(gdd)
guu

n=3    -- we are now in a 3-dimensional world
GammaII=zero(n,n,n)    -- array of type (3,3,3)

for( k,1,n,
    for( l,1,n,
        for( m,1,n,
            GammaII[k,l,m] =
                sum(i,1,2, guu[k,i]/2 *
                    ( d(gdd[m,i],X[l]) +
                      d(gdd[l,i],X[m]) -
                      d(gdd[m,l],X[i]) )
                ))))

GammaII          --(1)
GammaII[1,2,2]   --(2)
GammaII[2,1,2]
GammaII[2,2,1]
```

▷ Run script.

Comment. Here we have metric g and CHRISTOFFEL symbol Γ^{II} of 3 dimensions. Therefore the CHRISTOFFEL symbol is stacked with three 3-by-3 matrices. Only the first two $\Gamma_{..}^1$ and $\Gamma_{..}^2$ has non-zero entries. We pick three of them in (2).

EIGENMATH output:

$$\Gamma = \begin{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0 & -r & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & \frac{1}{r} & 0 \\ \frac{1}{r} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{bmatrix}$$

$-r$
 $\frac{1}{r}$
 $\frac{1}{r}$

11.2.2 *Calculation of the CHRISTOFFEL symbol's of both kinds using tensor techniques*

Example 26. Calculate the CHRISTOFFEL symbol in affine coordinates in $\mathbb{R}^2$ using the results of example 15.²⁶

```
# EIGENMATH
" metric tensor in affine coordinates"
gdd = zero(2,2)
    gdd[1,1]= 4
    gdd[1,2]= 1
    gdd[2,1]= 1
    gdd[2,2]= 1
-- use Cartesian coordinates
X = (x,y)
-- calculate guu
guu = inv(gdd)
guu

" calculate all partial derivatives w.r.t. x and y "
gddd = d( gdd, X)
gddd                                     -- an array with 2 Jacobi matrices

--- Calculate so-called connection coefficients aka the
"-- CHRISTOFFEL symbol of 1st kind along formula 11.1.a "

-- Gamma      = 1/2 (g      + g      - g      )
--      abc      ab,c      ac,b      bc,a
--      123      12 3      13 2      23 1
--
--
Gammaddd = 1/2*( gddd +                -- Gab,c: correct order
                 transpose(gddd,2,3) -  -- Gac,b: transpose c and b
                 transpose(gddd,2,3,1,2)) -- Gbc,a: transpose c and a,
--                                     -- then b and a

Gammaddd                                     --(1)

-- Raise first index a ('index juggling') using guu, see 11.1.a
--
--      a      au
-- Gamma      = g      Gamma
--      bc      ubc
--
--                                     --(2)

"-- CHRISTOFFEL symbol of 2nd kind along formula 11.1.b "
Gammaudd = dot(guu,Gammaddd)                                     --(3)
Gammaudd                                     --(4) CHRISTOFFEL symbol of 2nd kind
```

²⁶The tensor techniques in the next pages are inspired by code from G. WEIGT.

▷ Run script.

EIGENMATH output:

$$\Gamma_{ddd} = \left[\begin{array}{c} \left[\begin{array}{cc} 0 & 0 \end{array} \right] \\ \left[\begin{array}{cc} 0 & 0 \end{array} \right] \\ \left[\begin{array}{cc} 0 & 0 \end{array} \right] \end{array} \right] \quad \Gamma_{udd} = \left[\begin{array}{c} \left[\begin{array}{cc} 0 & 0 \end{array} \right] \\ \left[\begin{array}{cc} 0 & 0 \end{array} \right] \\ \left[\begin{array}{cc} 0 & 0 \end{array} \right] \end{array} \right]$$

Comment. In (1) we give $\mathbf{Gammadd} = \Gamma_{abc}$ the value in accordance with formula 11.1.b. In formula (2) index u is both upper and lower to be seen, ergo there is an implicit summation over u ('contraction') w.r.t. the EINSTEIN summation convention. This is done in (3). The resulting symbol $\mathbf{Gammaudd} = \Gamma_{bc}^a$ is outputted in (4). It is the affine CHRISTOFFEL symbol tensor Γ with the same result as in example 21.

Example 27. Calculate the CHRISTOFFEL symbols in polar coordinates in $\mathbb{R}^2$

```
# EIGENMATH
-- metric tensor in polar coordinates
gdd = zero(2,2)
      gdd[1,1]= 1
      gdd[2,2]= r^2

-- use polar coordinates
X = (r, theta)

-- calculate guu
guu = inv(gdd)

-- calculate all partial derivations w.r.t. r and theta
gddd = d( gdd, X)
gddd                                     --(1)

" calculate CHRISTOFFEL symbol of 1st kind:"
-- see explanation in previous example 26

Gammadd = 1/2*(gddd +
               transpose(gddd,2,3) -
               transpose(gddd,2,3,1,2))

Gammadd                                     --(2)

" calculate CHRISTOFFEL symbol of 2nd kind:"
-- raise first index using guu, i.e. .ddd -> .udd
Gammaudd = dot(guu,Gammadd) -- (3)
Gammaudd
```

▷ Run script.

EIGENMATH output:

$$g_{ddd} = \begin{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 \\ 2r & 0 \end{bmatrix} \end{bmatrix} \quad \Gamma_{ddd} = \begin{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & -r \end{bmatrix} \\ \begin{bmatrix} 0 & r \\ r & 0 \end{bmatrix} \end{bmatrix} \quad \Gamma_{udd} = \begin{bmatrix} \begin{bmatrix} 0 & 0 \\ 0 & -r \end{bmatrix} \\ \begin{bmatrix} 0 & \frac{1}{r} \\ \frac{1}{r} & 0 \end{bmatrix} \end{bmatrix}$$

Comment. In (1) we collect all partial derivatives w.r.t. r and θ , we then calculate Gammadd, which is our notation for the CHRISTOFFEL symbol Γ_{abc} of 1st kind.

Raising the first index u (see previous example) results in an implicit summation over u in (3). The resulting CHRISTOFFEL symbol $\text{Gammaudd} = \Gamma_{bc}^a$ of 2nd kind is outputted giving the same result as in example 24.

Example 28. Calculate the CHRISTOFFEL symbol in cylindrical coordinates in $\mathbb{R}^3$

```
# EIGENMATH
-- metric tensor in cylindrical coordinates
gdd = zero(3,3)
      gdd[1,1]= 1
      gdd[2,2]= r^2
      gdd[3,3]= 1

-- use cylindrical coordinates, t means theta
X = (r, t, z)

-- Calculate guu, i.e. the inverse of gij
guu = inv(gdd)

-- calculate all partial derivations w.r.t. r,t and z
gddd = d( gdd, X)
gddd

-- calculate CHRISTOFFEL symbol of 1st kind
Gammadd = 1/2*( gddd
                +
                transpose(gddd,2,3)
                -
                transpose(gddd,2,3,1,2))

Gammadd

-- Raise first index
Gammaudd = dot( guu, Gammadd )
Gammaudd
```

▷ Run script.

EIGENMATH output:

$$g_{ddd} = \begin{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 \\ 2r & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{bmatrix} \quad \Gamma_{ddd} = \begin{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0 & -r & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & r & 0 \\ r & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{bmatrix} \quad \Gamma_{udd} = \begin{bmatrix} \begin{bmatrix} 0 & 0 & 0 \\ 0 & -r & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & \frac{1}{r} & 0 \\ \frac{1}{r} & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \\ \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \end{bmatrix}$$

Comment. We repeat the same tensorial calculation as in the examples before. In contrast to the classic solution using for-loops the higher dimension of Γ is automatically detected, no special dimension marker like ' $n = 3$ ' is necessary: the dimension of CHRISTOFFEL symbol $\Gamma_{..}$ is taken from the declaration $X = (r, t, z)$, which is a 3-vector.

Exercise 38. Calculate the CHRISTOFFEL tensor Γ_{abc} of 1st kind for the last three examples using the classical way via for-loops. Verify your calculation with a check of the examples 24, 25 and 26.



This ends our journey to the first elements of Tensor Calculus. See you again doing the first steps into the *Elementary Differential Geometry of Surfaces*.

12 Appendix: source code of tensorBox

This is a collection of the relevant definitions from the booklet. This file should be loaded to do calculations with tensors using EIGENMATH.

```
##### (2023) Dr.W.G. Lindner, Leichlingen DE
###      tensorBox      Tensor box
#####      for calculations with tensors

-- KRONECKER delta function d(i,j)
delta(i,j) = test( i=j, 1, 0)

- KRONECKER delta 2-tensor dij
deltaij = zero(2,2)
    deltaij[1,1] = 1
    deltaij[2,2] = 1

-- 2D permutation tensor als FUNCTION
Eij(i,j) = j-i
Eij(1,2)
Eij(1,1)

-- 2D permutation tensor eij as TABLE
eij = zero(2,2)
for(i,1,2, for(j,1,2, eij[i,j]=j-i ))

-- 3D permutation tensor as FUNCTION
Eijk(i,j,k) = test(
    or( (i,j,k)==(1,2,3), (i,j,k)==(2,3,1), (i,j,k)==(3,1,2)), +1,
    or( (i,j,k)==(3,2,1), (i,j,k)==(1,3,2), (i,j,k)==(2,1,3)), -1,
    or( i==j, j==k, k==i), 0 )

-- 3D permutation tensor as TABLE
eijk = zero(3,3,3)
    eijk[1,2,3] = 1
    eijk[2,3,1] = 1
    eijk[3,1,2] = 1
    eijk[3,2,1] = -1
    eijk[1,3,2] = -1
    eijk[2,1,3] = -1

-- 4D permutation tensor as FUNCTION
Eijkl(i,j,k,l) = (i-j)*(i-k)*(i-l)*(j-k)*(j-l)*(k-l)/12

-- 4D permutation tensor as TABLE
```

```

eijkl=zero(4,4,4,4)
for(i,1,4, for(j,1,4, for(k,1,4, for(l,1,4,
    eijkl[i,j,k,l] = (i-j)*(i-k)*(i-l)*(j-k)*(j-l)*(k-l)/12 ) )))

-- metric tensor g in 2D, an example
gdd = zero(2,2)
    gdd[1,1]= 4
    gdd[1,2]= 1
    gdd[2,1]= 1
    gdd[2,2]= 1

-- inverse guu to gdd
guu = inv(gdd)

-- volume element
Z = det(gdd)
V = sqrt(Z)

-- LEVI-CIVITA tensor epsilon in 2D
epsilon = V * eij

Epsilon = 1/V * eij

-- 2D/3D/nD CHRISTOFFEL tensor 2nd kind
-- specify dimension n = 2,3,..
n=3
Gamma=zero(n,n,n)
for( k,1,n,
    for( l,1,n,
        for( m,1,n,
            Gamma[k,l,m] =
                sum(i,1,2, guu[k,i]/2 *
                    ( d(gdd[m,i],X[1]) +
                      d(gdd[l,i],X[m]) -
                      d(gdd[m,l],X[i]) )
                ))))

```

- [Click here to get the code.](#)

13 grad-div-curl in different coordinate systems

To use formula's for the vector analysis 'operators' *grad*, *div*, *curl* (=rot), *laplacian* it is recommended to remember and work with general coordinates for the definitions and afterwards specify the concrete coordinate system instead of memorize many (≥ 12) special formula's for special coordinate systems e.g. *cartesian*, *cylindrical*, *spherical*, ..see (1')..(4'), (1^C)..(4^C) and (1^S)..(4^S).

Theorem [Vector Analysis in $\mathbb{R}^3$ with general coordinates]

Let $u = (u_1, u_2, u_3)$ be a general ('curvilinear') orthogonal coordinate system.

Let $\mathbf{F} = (F_1, F_2, F_3): D \subset \mathbb{R}^3 \rightarrow \mathbb{R}^3$ be a C^1 "vector" function with coordinates u .

Let $f(u_1, u_2, u_3): D \subset \mathbb{R}^3 \rightarrow \mathbb{R}$ be a C^1 "scalar" function with coordinates u .

Let $R: (x, y, z) \mapsto (u_1, u_2, u_3)$ be a transformation between (x, y, z) and (u_1, u_2, u_3) .

Let $U := (\frac{\partial R}{\partial u}) = (U_1, U_2, U_3)$ be the *Jacobian* matrix of R with columns U_i .

Let $h = (h_1, h_2, h_3)$ with $h_i := \sqrt{U_i \bullet U_i}$ be the *scale factors* of the transformation R .

Let $\mathbf{e}_i := \frac{1}{h_i} U_i = \frac{1}{h_i} \frac{\partial R}{\partial u_i}$ be the unit vector in direction U_i .

Then we define the 'symbolic operators' $\nabla f, \nabla \cdot \mathbf{F}, \nabla \times \mathbf{F}, \Delta f$ alias *grad*, *div*, *curl* (=rot), *laplacian* through the following explicit formular's on the right hand side:

$$\nabla f(u_1, u_2, u_3) := \text{grad}(f(u_1, u_2, u_3)) := \frac{1}{h_1} \frac{\partial f}{\partial u_1} \mathbf{e}_1 + \frac{1}{h_2} \frac{\partial f}{\partial u_2} \mathbf{e}_2 + \frac{1}{h_3} \frac{\partial f}{\partial u_3} \mathbf{e}_3 \quad (13.1)$$

$$\nabla \cdot \mathbf{F} := \text{div } \mathbf{F} := \frac{1}{h_1 h_2 h_3} \left[\frac{\partial}{\partial u_1} (h_2 h_3 F_1) + \frac{\partial}{\partial u_2} (h_3 h_1 F_2) + \frac{\partial}{\partial u_3} (h_1 h_2 F_3) \right] \quad (13.2)$$

$$\begin{aligned} \nabla \times \mathbf{F} := \text{curl } \mathbf{F} &:= \frac{1}{h_1 h_2 h_3} \begin{vmatrix} h_1 \mathbf{e}_1 & h_2 \mathbf{e}_2 & h_3 \mathbf{e}_3 \\ \frac{\partial}{\partial u_1} & \frac{\partial}{\partial u_2} & \frac{\partial}{\partial u_3} \\ h_1 F_1 & h_2 F_2 & h_3 F_3 \end{vmatrix} \\ &:= \frac{1}{h_2 h_3} \left(\frac{\partial(h_3 F_3)}{\partial u_2} - \frac{\partial(h_2 F_2)}{\partial u_3} \right) + \frac{1}{h_3 h_1} \left(\frac{\partial(h_1 F_1)}{\partial u_3} - \frac{\partial(h_3 F_3)}{\partial u_1} \right) + \frac{1}{h_1 h_2} \left(\frac{\partial(h_2 F_2)}{\partial u_1} - \frac{\partial(h_1 F_1)}{\partial u_2} \right) \\ \Delta f &:= \text{Laplacian}(f(u_1, u_2, u_3)) \\ &:= \frac{1}{h_1 h_2 h_3} \left[\frac{\partial}{\partial u_1} \left(\frac{h_2 h_3}{h_1} \frac{\partial f}{\partial u_1} \right) + \frac{\partial}{\partial u_2} \left(\frac{h_3 h_1}{h_2} \frac{\partial f}{\partial u_2} \right) + \frac{\partial}{\partial u_3} \left(\frac{h_1 h_2}{h_3} \frac{\partial f}{\partial u_3} \right) \right] \end{aligned} \quad (13.3)$$

$$(13.4)$$

Verify using EIGENMATH:

1. Get the usual standard expressions for the 4 operators using **rectangular cartesian coordinates** (x, y, z) for (u_1, u_2, u_3) in the formular's (1)..(4):

$$(1') \quad \nabla f = \text{grad}(f) = \frac{\partial f}{\partial x} \mathbf{i} + \frac{\partial f}{\partial y} \mathbf{j} + \frac{\partial f}{\partial z} \mathbf{k} \equiv \left(\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y}, \frac{\partial f}{\partial z} \right)$$

$$(2') \quad \nabla \cdot \mathbf{F} = \text{div } \mathbf{F} = \frac{\partial F_1}{\partial x} + \frac{\partial F_2}{\partial y} + \frac{\partial F_3}{\partial z}$$

$$(3') \quad \nabla \times \mathbf{F} = \text{curl}(\mathbf{F}) = (F_{3,y} - F_{2,z}) \mathbf{i} + (F_{1,z} - F_{3,x}) \mathbf{j} + (F_{2,x} - F_{1,y}) \mathbf{k} \quad \square^{27}$$

$$= \left(\frac{\partial F_3}{\partial y} - \frac{\partial F_2}{\partial z} \right) \mathbf{i} + \left(\frac{\partial F_1}{\partial z} - \frac{\partial F_3}{\partial x} \right) \mathbf{j} + \left(\frac{\partial F_2}{\partial x} - \frac{\partial F_1}{\partial y} \right) \mathbf{k}$$

$$(4') \quad \Delta f \equiv \nabla^2 f \equiv \text{Laplacian}(f) := \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} + \frac{\partial^2 f}{\partial z^2}$$

- a. If $f(x, y, z) = x^2 y z^3$ and $\mathbf{F} = xz \mathbf{i} - y^2 \mathbf{j} + 2x^2 y \mathbf{k}$ find $\nabla f, \nabla \cdot \mathbf{F}, \nabla \times \mathbf{F}, \text{div}(f\mathbf{F}), \text{curl}(f\mathbf{F}), \Delta f$.

- b. Check 1a. by hand.

2. Get the following expressions for the 4 operators using **cylindrical coordinates** (r, φ, z) for (u_1, u_2, u_3) in the formular's (1)..(4):

$$(1^C) \quad \nabla f = \text{grad}(f) = f_{,r} \mathbf{e}_r + \frac{f_{,\varphi}}{r} \mathbf{e}_\varphi + f_{,z} \mathbf{e}_z$$

$$\nabla f(r, \varphi, z) = \frac{\partial f}{\partial r} \mathbf{e}_r + \frac{1}{r} \frac{\partial f}{\partial \varphi} \mathbf{e}_\varphi + \frac{\partial f}{\partial z} \mathbf{e}_z$$

$$(2^C) \quad \nabla \cdot \mathbf{F} = \text{div } \mathbf{F} = \frac{1}{r} \frac{\partial}{\partial r} (r F_r) + \frac{1}{r} F_{\varphi,\varphi} + F_{z,z}$$

$$= \frac{1}{r} \frac{\partial}{\partial r} (r F_r) + \frac{1}{r} \frac{\partial F_\varphi}{\partial \varphi} + \frac{\partial F_z}{\partial z}$$

$$(3^C) \quad \nabla \times \mathbf{F} = \left[\frac{1}{r} \frac{\partial F_z}{\partial \varphi} - \frac{\partial F_\varphi}{\partial z} \right] \mathbf{e}_r + \left[\frac{\partial F_r}{\partial z} - \frac{\partial F_z}{\partial r} \right] \mathbf{e}_\varphi + \frac{1}{r} \left[\frac{\partial}{\partial r} (r \cdot F_\varphi) - \frac{\partial F_r}{\partial \varphi} \right] \mathbf{e}_z$$

$$(4^C) \quad \Delta f = \nabla^2 f = \text{Laplacian}(f) = \frac{f_{,r}}{r} + f_{,rr} + \frac{f_{,\varphi\varphi}}{r^2} + f_{,zz}$$

- a. If $f(r, \varphi, z) = r^2 z^3$ and $\mathbf{F} = z \mathbf{e}_r - \varphi^2 \mathbf{e}_\varphi + 2r^2 z \mathbf{e}_z$ find $\nabla f, \nabla \cdot \mathbf{F}, \nabla \times \mathbf{F}, \text{div}(f\mathbf{F}), \text{curl}(f\mathbf{F}), \Delta f$.

- b. Check 2a. by hand.

3. Get the following expressions for the 4 operators using **spherical coordinates** (r, θ, φ) for (u_1, u_2, u_3) in the formular's (1)..(4):

$$(1^S) \quad \nabla f = \text{grad}(f) = f_{,r} \mathbf{e}_r + \frac{f_{,\theta}}{r} \mathbf{e}_\theta + \frac{f_{,\varphi}}{r \sin(\theta)} \mathbf{e}_\varphi$$

$$\nabla f(r, \theta, \varphi) = \frac{\partial f}{\partial r} \mathbf{e}_r + \frac{1}{r} \frac{\partial f}{\partial \theta} \mathbf{e}_\theta + \frac{1}{r \sin \theta} \frac{\partial f}{\partial \varphi} \mathbf{e}_\varphi$$

$$(2^S) \quad \nabla \cdot \mathbf{F} = \text{div } \mathbf{F} = \frac{1}{r^2} \frac{\partial}{\partial r} (r^2 F_r) + \frac{1}{r \sin \theta} \frac{\partial}{\partial \theta} (\sin \theta F_\theta) + \frac{1}{r \sin \theta} \frac{\partial F_\varphi}{\partial \varphi}$$

$$(3^S) \quad \nabla \times \mathbf{A} = \frac{1}{r \sin \theta} \left[\frac{\partial}{\partial \theta} (F_\varphi \sin \theta) - \frac{\partial F_\theta}{\partial \varphi} \right] \mathbf{e}_r + \frac{1}{r} \left[\frac{1}{\sin \theta} \frac{\partial F_r}{\partial \varphi} - \frac{\partial}{\partial r} (r F_\varphi) \right] \mathbf{e}_\theta + \frac{1}{r} \left[\frac{\partial}{\partial r} (r F_\theta) - \frac{\partial F_r}{\partial \theta} \right] \mathbf{e}_\varphi$$

$$(4^S) \quad \Delta f = \frac{1}{r^2} \frac{\partial}{\partial r} \left(r^2 \frac{\partial f}{\partial r} \right) + \frac{1}{r^2 \sin(\theta)} \frac{\partial}{\partial \theta} (\sin(\theta) \frac{\partial f}{\partial \theta}) + \frac{1}{r^2 \sin^2(\theta)} \frac{\partial^2 f}{\partial \varphi^2}$$

²⁷this is a usual compact short notation for the partial derivative: $\frac{\partial F_3}{\partial y} \equiv F_{3,y}$

$$= \frac{1}{r} \frac{\partial^2}{\partial r^2}(rf) + \frac{1}{r^2 \sin \theta} \frac{\partial}{\partial \theta} \left(\sin \theta \frac{\partial f}{\partial \theta} \right) + \frac{1}{r^2 \sin^2 \theta} \frac{\partial^2 f}{\partial \varphi^2}$$

- a. If $f(r, \theta, \varphi) = \frac{1}{2}(2 \cos \theta + 3 \sin^2 \theta \cos \varphi)$ and $\mathbf{F} = \mathbf{e}_r + \frac{2}{\theta} \mathbf{e}_\theta + \varphi r \sin(\theta) \mathbf{e}_\varphi$
find $\nabla f, \nabla \cdot \mathbf{F}, \nabla \times \mathbf{F}, \operatorname{div}(f\mathbf{F}), \operatorname{curl}(f\mathbf{F}), \Delta f$ in *spherical coordinates*.
- b. Check 3a. by hand.
4. Let $\mathbf{F}(x, y, z) = z\mathbf{e}_1 - 2x\mathbf{e}_2 + y\mathbf{e}_3 = (z, -2x, y)$. Write F in cylindrical coordinates.
Then calculate $\nabla \cdot \mathbf{F}$ and $\nabla \times \mathbf{F}$ in *cylindrical coordinates*.
5. Derive formular's for $\nabla f, \nabla \cdot \mathbf{F}, \nabla \times \mathbf{F}, \Delta f$ in *parabolic coordinates*,
i.e $(x, y, z) = (\frac{1}{2}(u_1^2 - u_2^2), u_1 u_2, u_3)$.
- a. If $f(u_1, u_2, u_3) = u_1^2 u_3^3$ and $\mathbf{F} = u_3 \mathbf{e}_1 - u_2^2 \mathbf{e}_2 + 2u_1^2 u_2 \mathbf{e}_3$
find $\nabla f, \nabla \cdot \mathbf{F}, \nabla \times \mathbf{F}, \operatorname{div}(f\mathbf{F}), \operatorname{curl}(f\mathbf{F}), \Delta f$ in general 'curvilinear' coordinates..
- b. Check 3a. by hand.
6. Get expressions for the operators $\nabla f, \nabla \cdot \mathbf{F}, \nabla \times \mathbf{F}, \Delta f$ in **polar coordinates** in $\mathbb{R}^2$.
7. Poof the formular's (1)..(4). See: [27, p. 137], [31, p. 62 ff].
 - (1) Grad: see [34, p. 22], [27, p. 148], [31, p. 64], [15, §6.9]
 - (2) Divergence: [34, p. 30], [27, p. 150], [31, p. 64], [15, §6.10]
 - (3) Curl: [34, p. 31], [27, p. 150], [15, §6.11]
 - (4) Laplacian: [34, p. 23], [27, p. 151], [15, §6.12]

14 Bibliography

References

- [1] BANCHOFF, T. F. & LOVETT, S. (³2022): *Differential Geometry of Curves and Surfaces*. London: Chapman and Hall.
- [2] BISHOP, R. L. & GOLDBERG, S. I. (1980): *Tensor Analysis on Manifolds*. New York: Dover Publications.
- [3] COLLIER, P. (2021): *A Beginner's Guide to Differential Forms*. Incomprehensible Books.
- [4] COLLIER, P. (³2021): *A Most Incomprehensible Thing*. Incomprehensible Books.
- [5] FLEISCH, D. (2012): *A Student's Guide to Vectors and Tensors*. Cambridge UK: Cambridge University Press.
- [6] GRINFELD, P. (2013): *Introduction to Tensor Analysis and the Calculus of Moving Surfaces*. New York: Springer.
- [7] KREYSZIG, E. (²1968): *Differentialgeometrie*. Leipzig: Akademische Verlagsgesellschaft Geest & Portig.
- [8] LINDNER, W. (2023): *Elementary Differential Geometry of Surfaces using CAS EIGENMATH*.
url: <https://lindnerdrwg.github.io/Differential-Geometry-of-Surfaces.pdf>
- [9] LINDNER, W. (2020): *Didactical APOS theory and other papers*.
url: <https://lindnerdrwg.github.io/>
- [10] LINDNER, W. (2020): *Linear Algebra interactiv (LAI.5): Determinants*.
url: <https://lindnerdrwg.github.io/LAI5-Determinants.pdf>
- [11] LINDNER, W. (2020): *Linear Algebra interactiv (LAI.3): Singular Systems and Analytical Geometry*.
url: <https://lindnerdrwg.github.io/LAI3-Singular-Systems-Analytical-Geometry.pdf>
- [12] LIPSCHUTZ, M. M. (1969): *Differential Geometry*. New York: McGraw-Hill
- [13] LOVELOCK, D. & RUND, H. (1989): *Tensors, Differential Forms and Variational Principles*. New York: Dover Publications.
- [14] MARSDEN, J. & WEINSTEIN, A. (²1985): *Calculus I, II, III*. New York: Springer.
- [15] MURRAY, D. (2018): *Vector Operators*.
<https://www.lehman.edu/faculty/anchordoqui/VC-3.pdf>

- [16] NEEDHAM, T. (2021): *Visual Differential Geometry and Forms*. Princeton: Princeton University Press.
- [17] OLOFF, R. (⁶2018): *Geometrie der Raumzeit*. [engl.: Geometry of Spacetime.] Berlin: Springer Spectrum.
- [18] RECKZIEGEL, H. ET AL. (1998): *Elementare Differentialgeometrie mit Maple*. Braunschweig: Vieweg.
- [19] REJBRAND, A. (2023): *The Rejbrand Encyclopaedia of Curves and Surfaces*.
url: <https://treks.se/surfaces.php>
- [20] ROOLFS, G. (2026): *Trägheitsmoment*.
url: <http://groolfs.de/Verschiedenespdf/Traegheitsmoment.pdf>
- [21] ROOLFS, G. (2026): *Was ist ein Tensor?*
url: <http://groolfs.de/Verschiedenespdf/Tensor2.pdf>
- [22] ROOLFS, G. (2026): *Tensoren*.
url: <http://groolfs.de/Verschiedenespdf/Tensor.pdf>
- [23] SCHULTZ-PISZACHICH, W. (1977): *Tensoralgebra und -analysis*. Leipzig: BSB B.G. Teubner.
- [24] SNYGG, J. (2001): *A New Approach to Differential Geometry using Clifford's Geometric Algebra*. New York: Springer-Birkhäuser.
- [25] SOCHI, T. (2017): *Tensor Calculus Made Simple*.
?: CreativeSpace. ISBN 9781541013636.
- [26] SOCHI, T. (2017): *Principles of Tensor Calculus*.
?: CreativeSpace. ISBN 9781974401390.
- [27] SPIEGEL, M.R. (1959): *Vector Analysis and an Introduction to Tensor Analysis*. New York: McGraw-Hill.
- [28] SPIEGEL, M. R. (1963): *Advanced Calculus*.
New York: McGraw-Hill (Schaums' Outline Series)
- [29] STEEB, W.-H. (1991): *Kronecker Product of Matrices and Applications*. Mannheim: Bibliographisches Institut.
- [30] STEEB, W.-H., HARDY, Y. (2019): *Kronecker Product*. In: Matrix Calculus, Kronecker Product and Tensor Product.
url: https://www.worldscientific.com/doi/pdf/10.1142/9789811202520_0002
- [31] TONG, A.(2020): *Vector Analysis*.
url: <https://www.damtp.cam.ac.uk/user/tong/vc/vc3.pdf>

- [32] TOTH, V. T. (2004): *On tensors and their matrix representations*.
url: <https://www.vttoth.com/CMS/physics-notes/139>
- [33] TOTH, V. T. (2004): *Vector spaces and metric manifolds*.
url: <https://www.vttoth.com/CMS/physics-notes/394>
- [34] VVEDENSKY, D.(1994): *Partial Differential Equations with MATHEMATICA*.
Cambridge: Addison-Wesley.
- [35] WEIGT, G. (2026): *EIGENMATH Homepage*.
url: <https://georgeweigt.github.io>
- [36] WEIGT, G. (2021): *The Schwarzschild Metric*.
url: <https://georgeweigt.github.io/schwarzschild-metric.html>
- [37] WIKIPEDIA: Kronecker Product.
url: https://en.wikipedia.org/wiki/Kronecker_product
- [38] WIKIPEDIA: Levi-Civita-Symbol.
url: <https://de.wikipedia.org/wiki/Levi-Civita-Symbol>
- [39] WIKIPEDIA: Metric Tensor.
url: https://en.wikipedia.org/wiki/Metric_tensor
- [40] WIKIPEDIA: Outer Product.
url: https://en.wikipedia.org/wiki/Outer_product
- [41] WIKIPEDIA: Tensor Contraction.
url: https://en.wikipedia.org/wiki/Tensor_contraction
- [42] WIKIPEDIA: Polar coordinate system.
url: https://en.wikipedia.org/wiki/Polar_coordinate_system
- [43] WOLFRAM mathworld: Cylinder.
url: <https://mathworld.wolfram.com/Cylinder.html>
- [44] WOLFRAM mathworld: Sphere.
url: <https://mathworld.wolfram.com/Sphere.html>
- [45] WOLFRAM mathworld: Coordinate System.
url: <https://mathworld.wolfram.com/CoordinateSystem.html>
- [46] WOLFRAM mathworld: Tensors.
url: <https://www.wolfram.com/broadcast/video.php?c=311&v=107>
- [47] WOLFRAM mathworld: Sphere.
url: <https://mathworld.wolfram.com/Tensor.html>

- [48] WOLFRAM.ALPHA mathworld: Sphere.
url: <https://www.wolframalpha.com/input/?i=tensor>
- [49] WOLFRAM mathworld: MetricTensor.
url: [https://resources.wolframcloud.com/FunctionRepository/resources/
MetricTensor/](https://resources.wolframcloud.com/FunctionRepository/resources/MetricTensor/)